MASTER AZURE – BECOME CLOUD READY • STUDENT NOTES

Session 04 – Live

ADF Pipelines in Practice

Parquet • Source queries & joins • Many tables in one pipeline • Dynamic folders • Dataset parameters

Hands-on: 7 pipeline labs • Azure SQL Database • ADLS Gen2

Prepared by Ir. UWAYO Jacques · Follow me on LinkedIn: **linkedin.com/in/uwayo**

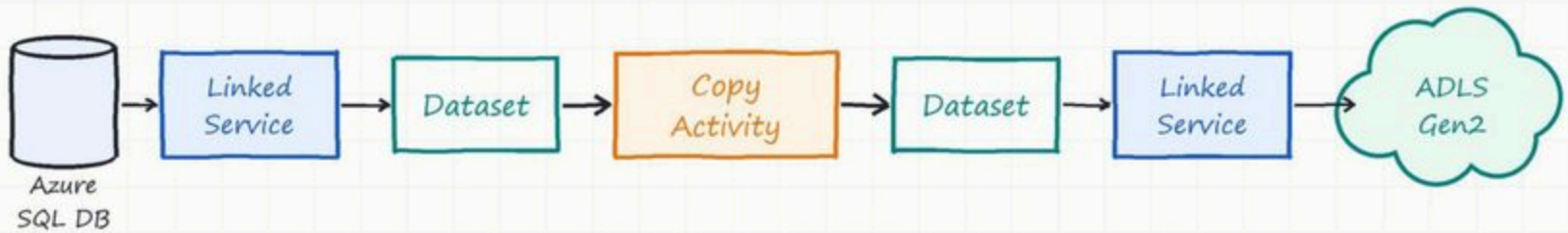
Introduction – why this session matters

- Real projects copy tens or hundreds of tables, not one
- The data lake must be organised: the right format, the right folder
- Copy only the data you need: queries, filters, joins
- Reusable building blocks (parameters) instead of copy-paste
- These patterns appear in almost every ADF interview and project

Prepared by Ir. UWAYO Jacques · Follow me on LinkedIn: linkedin.com/in/uwayo

Recap of Session 03

Recap: what we built in Session 03



Session 03: one table -> one file (CSV, JSON) - everything hard-coded

Today (Session 04): make pipelines smarter

- **Parquet** → a columnar file format for the lake
- **Query** → read only the rows / columns you need, join tables
- **Many activities** → several tables in ONE pipeline
- **Folders** → organise the lake: one folder per table
- **Parameters** → one generic dataset reused many times

Prepared by Ir. UWAYO Jacques · Follow me on LinkedIn: linkedin.com/in/uwayo

Learning objectives

- Explain structured, semi-structured and unstructured data
- Choose between CSV, JSON and Parquet – and explain why Parquet wins in a lake
- Use a source Query: select columns, filter rows, join tables
- Put several Copy activities in one pipeline; chain them or run them in parallel
- Use the 4 dependency conditions: success, failure, completion, skip
- Write each table into its own folder in ADLS
- Build one parameterised dataset and pass values at run time

Prepared by Ir. UWAYO Jacques · Follow me on LinkedIn: linkedin.com/in/uwayo

Session agenda

#	Topic	Type
1	Types of data and file formats: CSV, JSON, Parquet	Theory
2	Lab 1 – CustomerAddress table → ADLS as Parquet	Hands-on
3	Source queries – Lab 2 (Query → JSON) and Lab 3 (Join)	Hands-on
4	Lab 4 – Many tables in one pipeline, dependency conditions	Hands-on
5	Lab 5 – Dynamic folder creation in the lake	Hands-on
6	Lab 6 – Dataset parameters (dynamic parameterisation)	Hands-on
7	Lab 7 – Practice: ProductDescription (with a real error)	Hands-on
8	Troubleshooting, best practices, Q&A, homework	Discussion

Prepared by Ir. UWAYO Jacques · Follow me on LinkedIn: linkedin.com/in/uwayo

Before we start – what you need

Item	Example name used in these notes	Created in
Resource group	rg-adf-training	Session 02
ADLS Gen2 storage account + container	stdemoadls / landing	Session 02
Azure Data Factory (V2)	adf-demo-dev	Session 02 / 03
Azure SQL server + database (sample data)	sql-demo-server / sqldb-demo	Session 03
Linked services	SQL_DB_LS and ADLS_LS	Session 03
Datasets	Customer_SQL_DB_DS, ADLS_CSV_DS	Session 03

⚠ **The serverless database may be paused – the first connection can take up to a minute while it wakes up.**

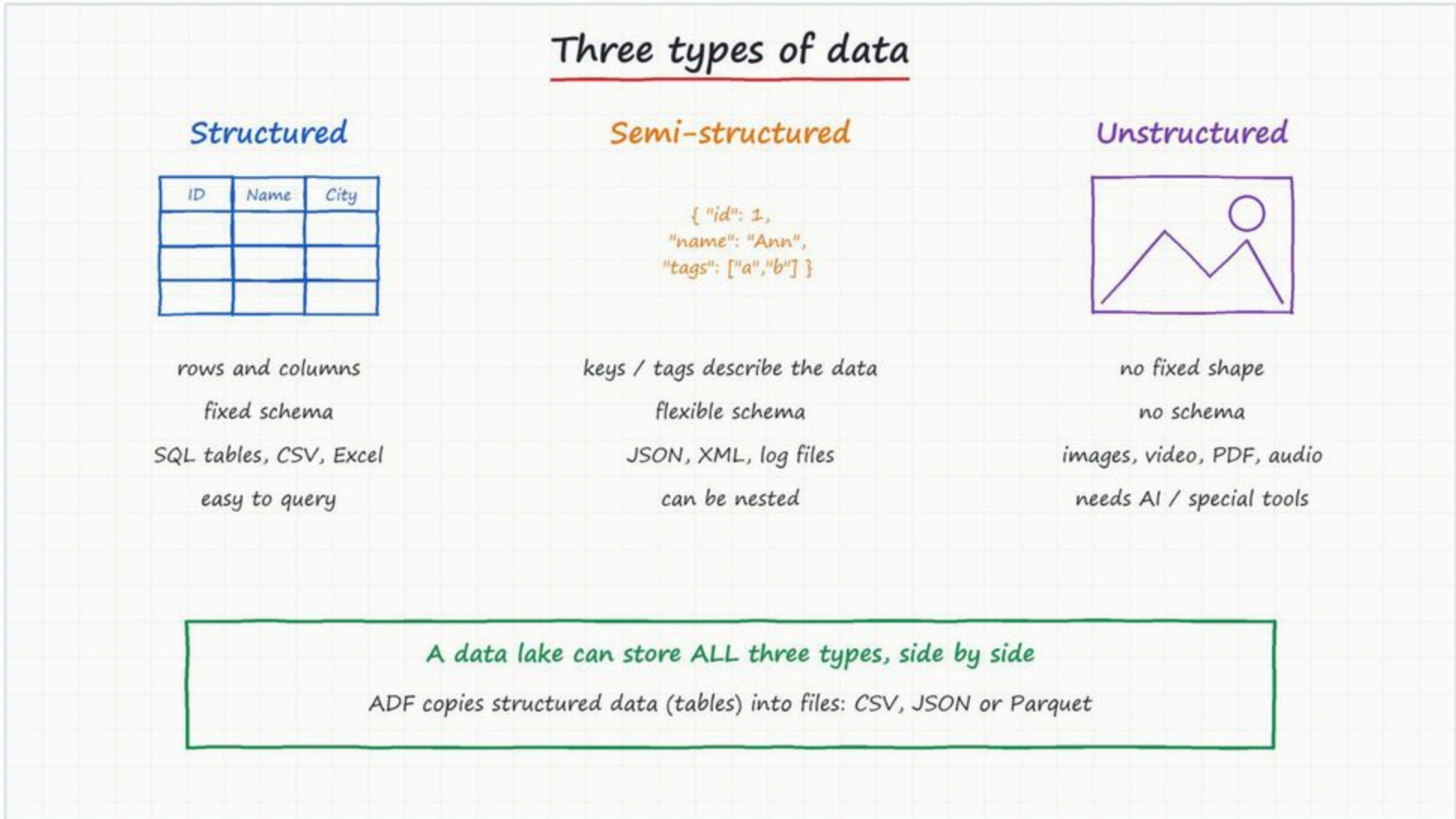
Prepared by Ir. UWAYO Jacques · Follow me on LinkedIn: linkedin.com/in/uwayo

PART 1

Types of data and file formats

Structured • Semi-structured • Unstructured • CSV vs JSON vs Parquet

Three types of data



CSV vs JSON vs Parquet

CSV vs JSON vs Parquet

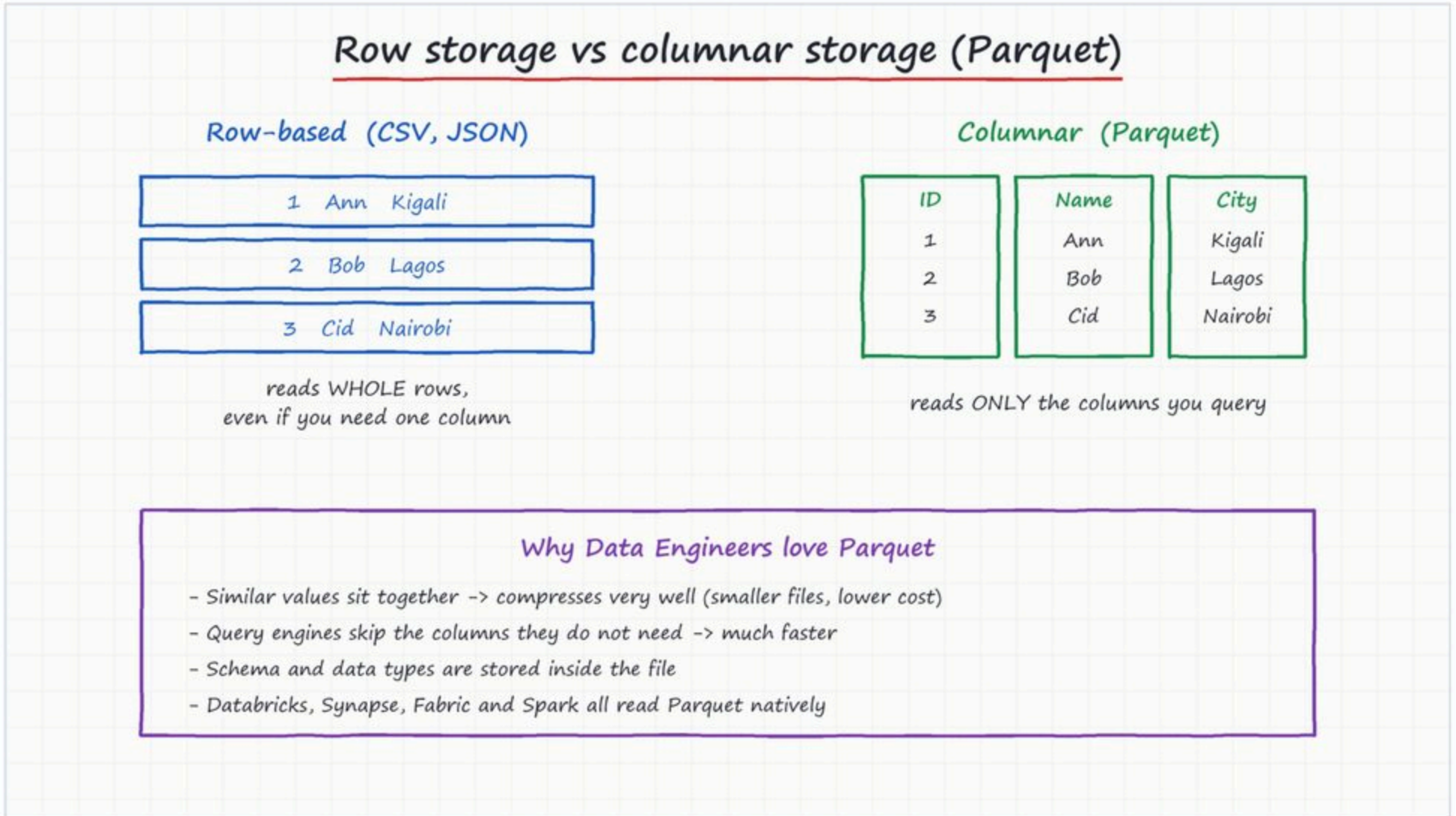
	CSV	JSON	Parquet	
Layout	row by row, text	row by row, text	column by column, binary	
Column names	once (header)	repeated in every row	stored once in metadata	
Data types	no (all text)	basic	yes (int, date, decimal...)	
Compression	none / gzip	none / gzip	built-in (snappy)	
Size	medium	largest	smallest	✓
Open in Notepad	yes	yes	no (binary)	
Best for	simple exchange	APIs, nested data	analytics / data lake	✓

Rule of thumb: Parquet (or Delta, which is built on Parquet) for the lake

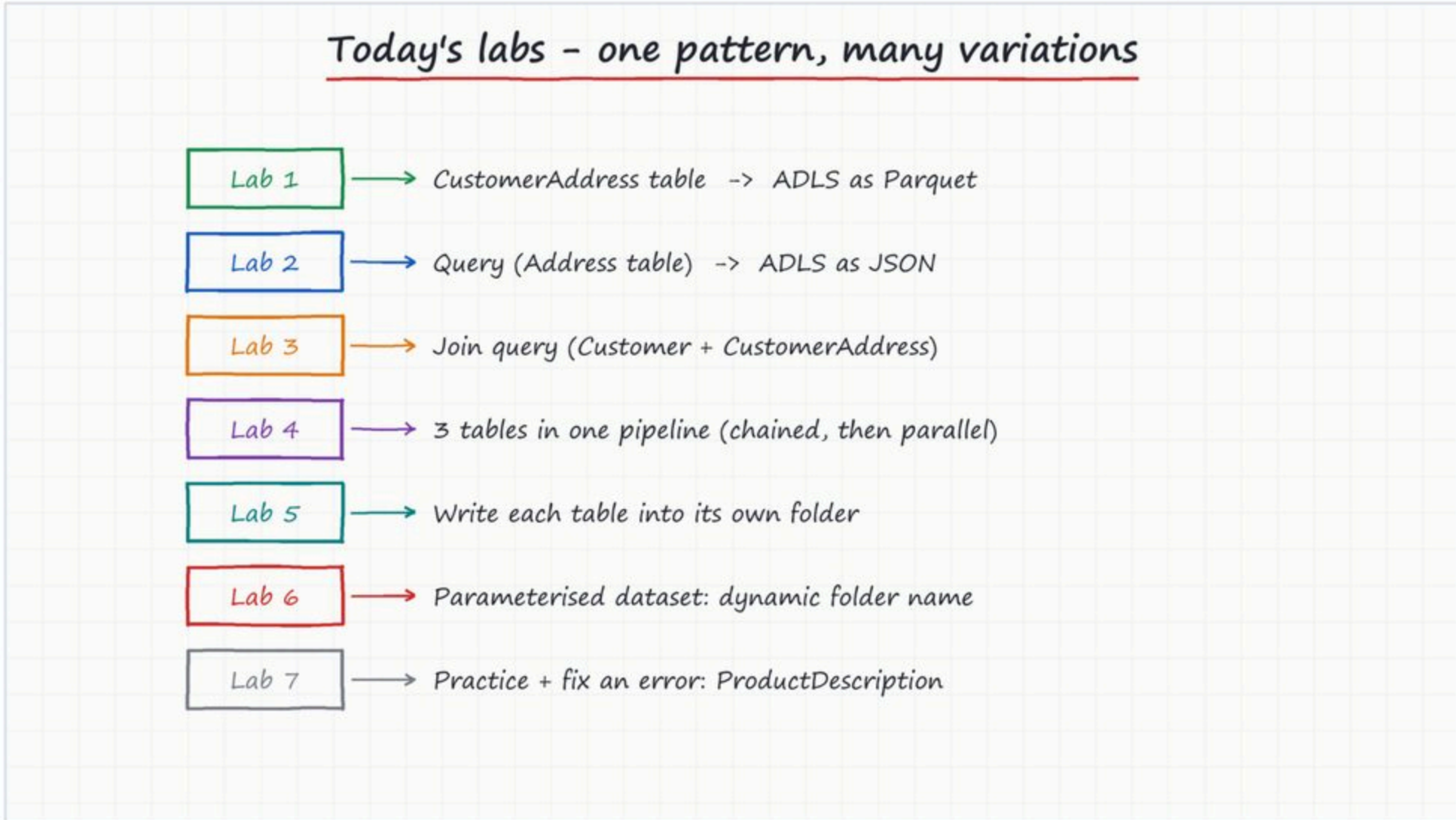
CSV / JSON when another system or a person must read the file

Prepared by Ir. UWAYO Jacques · Follow me on LinkedIn: linkedin.com/in/uwayo

Row storage vs columnar storage



Today's labs at a glance



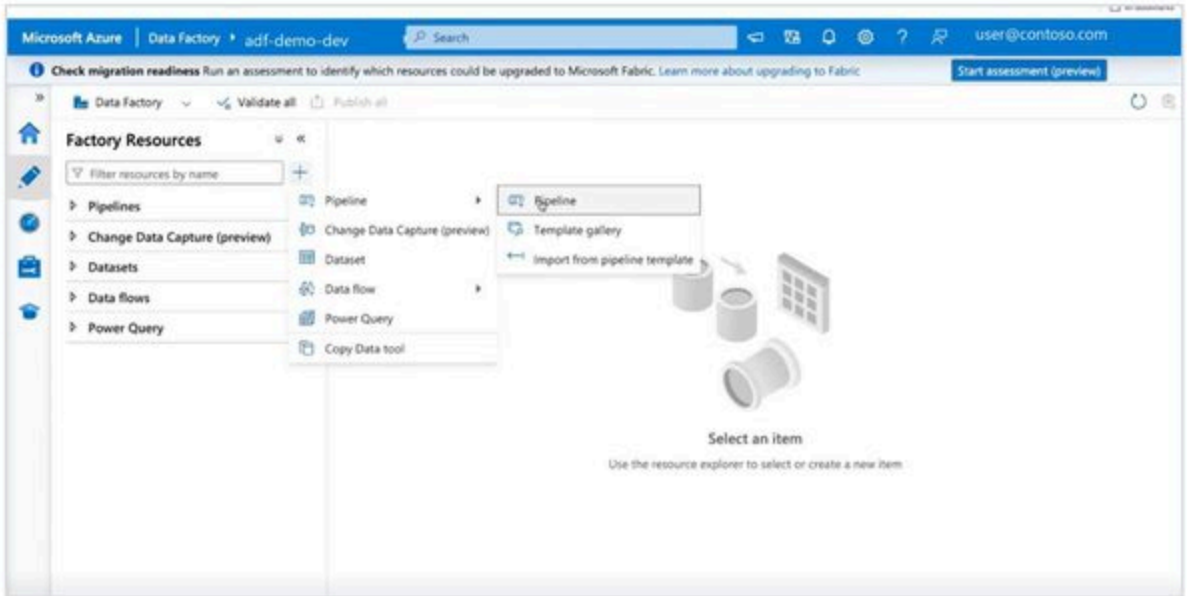
PART 2

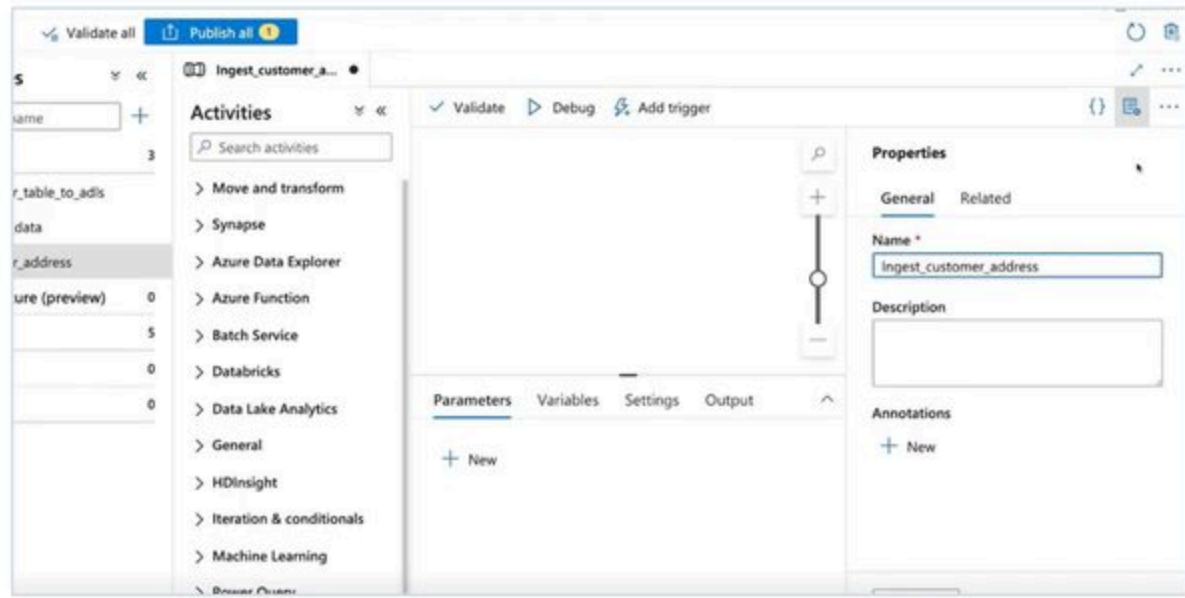
Lab 1: Table → ADLS as Parquet

SalesLT.CustomerAddress → landing/SalesLT.CustomerAddress.parquet

PART 2

Step 1 – New pipeline: Ingest_customer_address

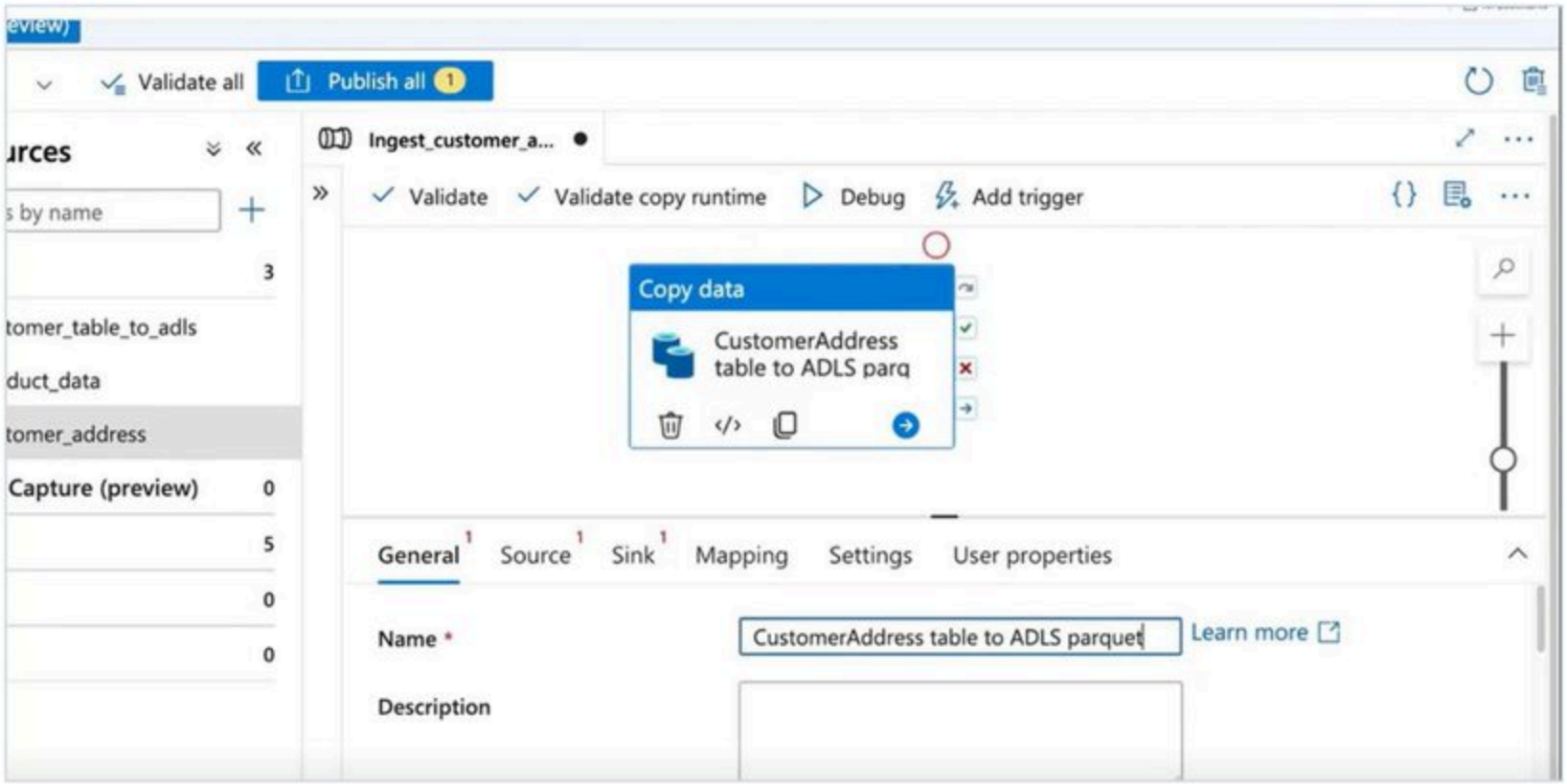




- Author → + → Pipeline → Pipeline
- Properties → Name: Ingest_customer_address
- Factory Resources already show last week's pipelines

PART 2

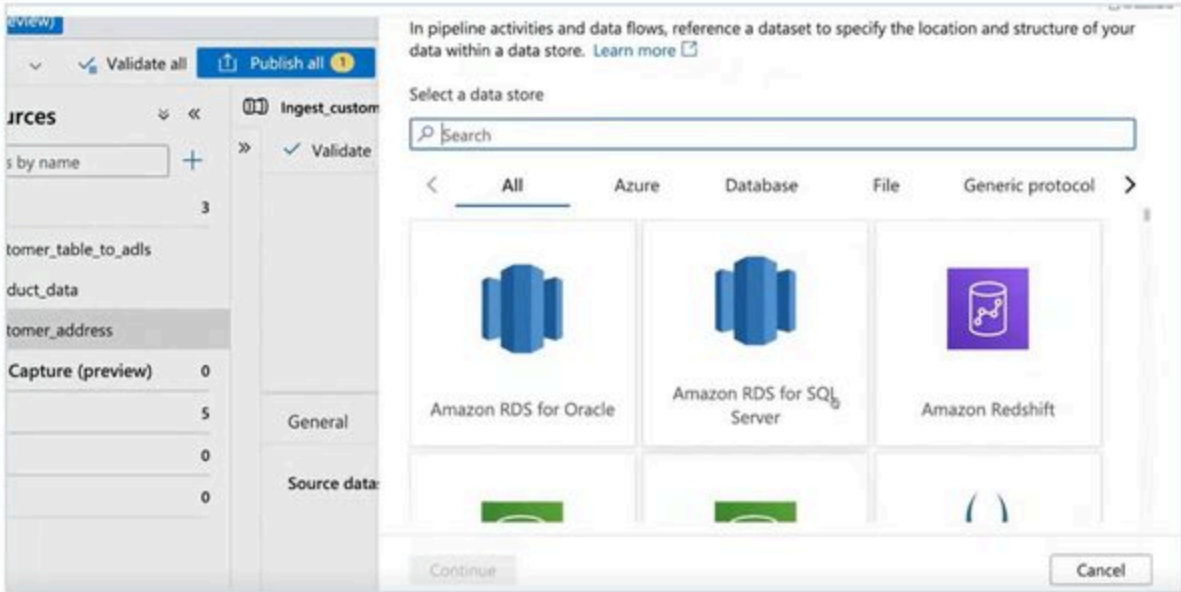
Step 2 – Add a Copy data activity

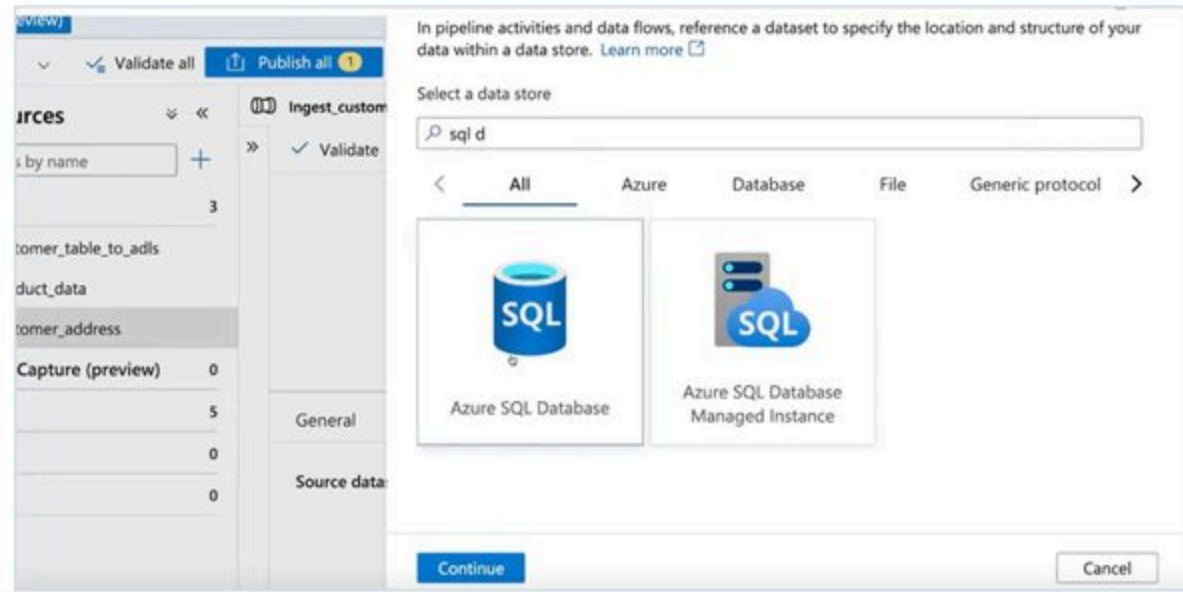


- Activities → Move and transform → Copy data
- Name: CustomerAddress table to ADLS parquet
- Red numbers on tabs = required fields missing

PART 2

Step 3 – New source dataset (Azure SQL Database)





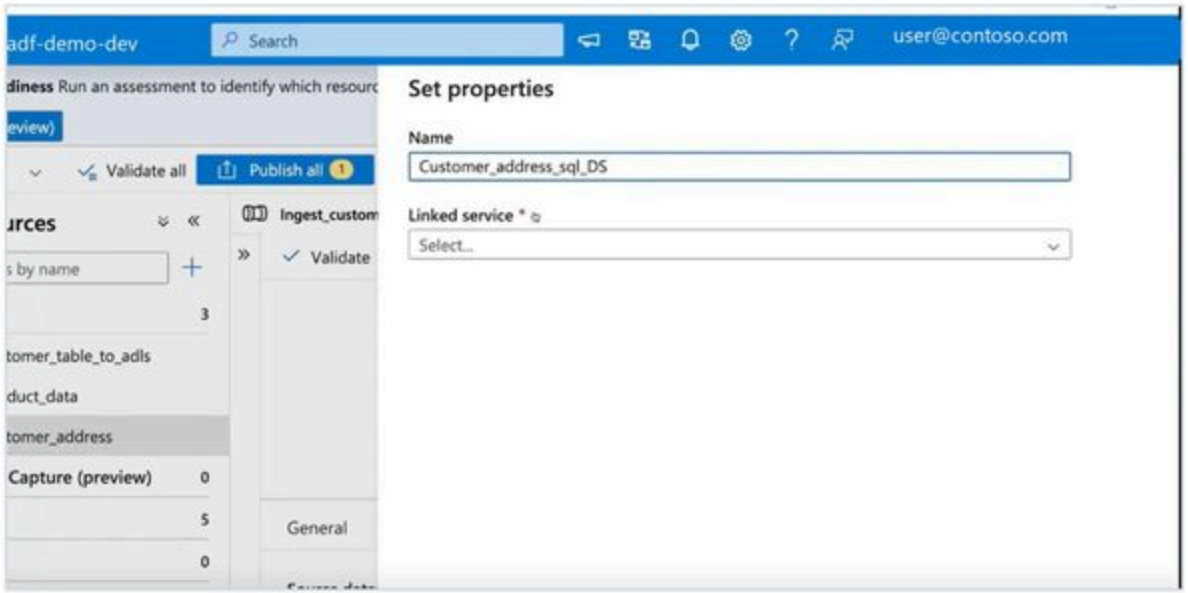
- Source tab → + New
- Search "sql" → Azure SQL Database → Continue

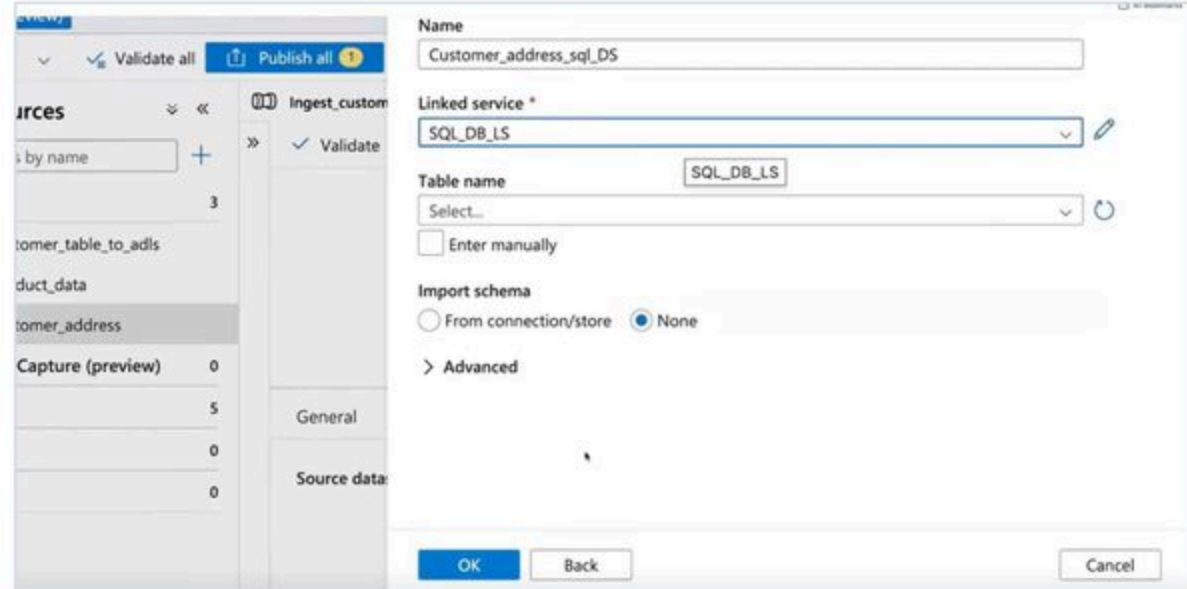
- You can create datasets from the activity – no need to leave the pipeline

Prepared by Ir. UWAYO Jacques · Follow me on LinkedIn: linkedin.com/in/uwayo

PART 2

Step 3 – Name, linked service and table

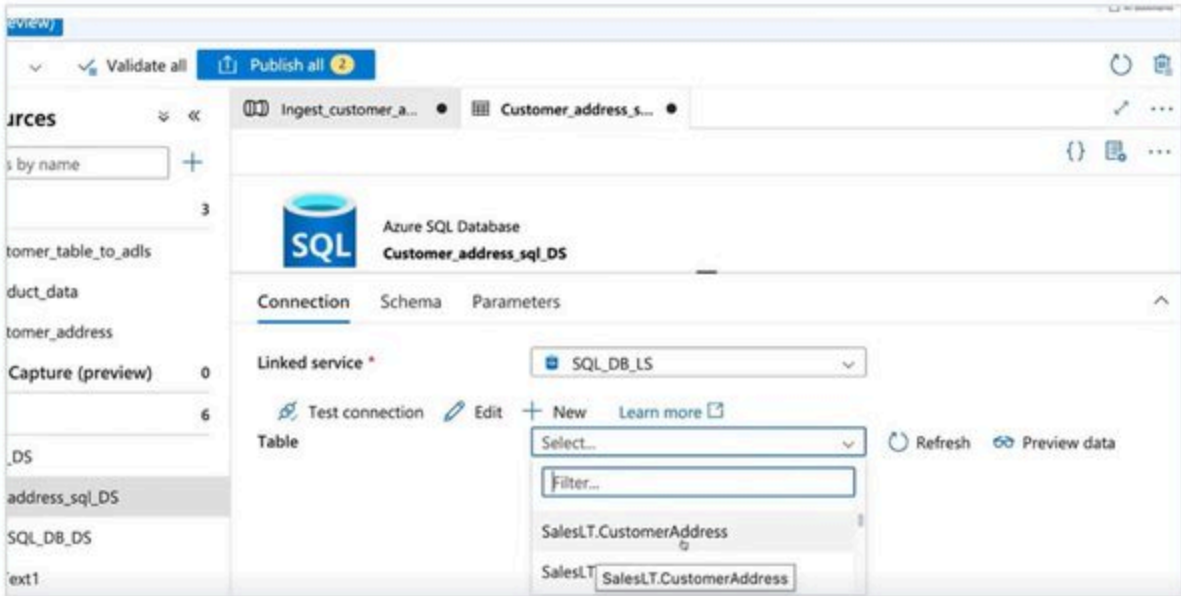


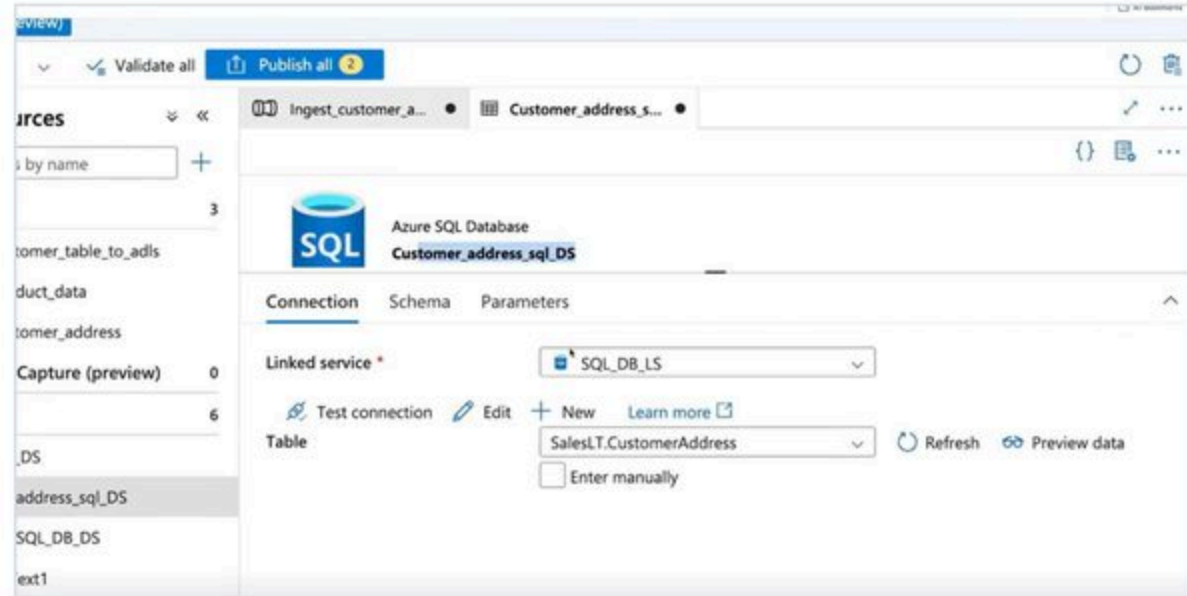


- Name: Customer_address_sql_DS
- Linked service: SQL_DB_LS (reused)
- Import schema: From connection/store

PART 2

Step 3 – Select SalesLT.CustomerAddress

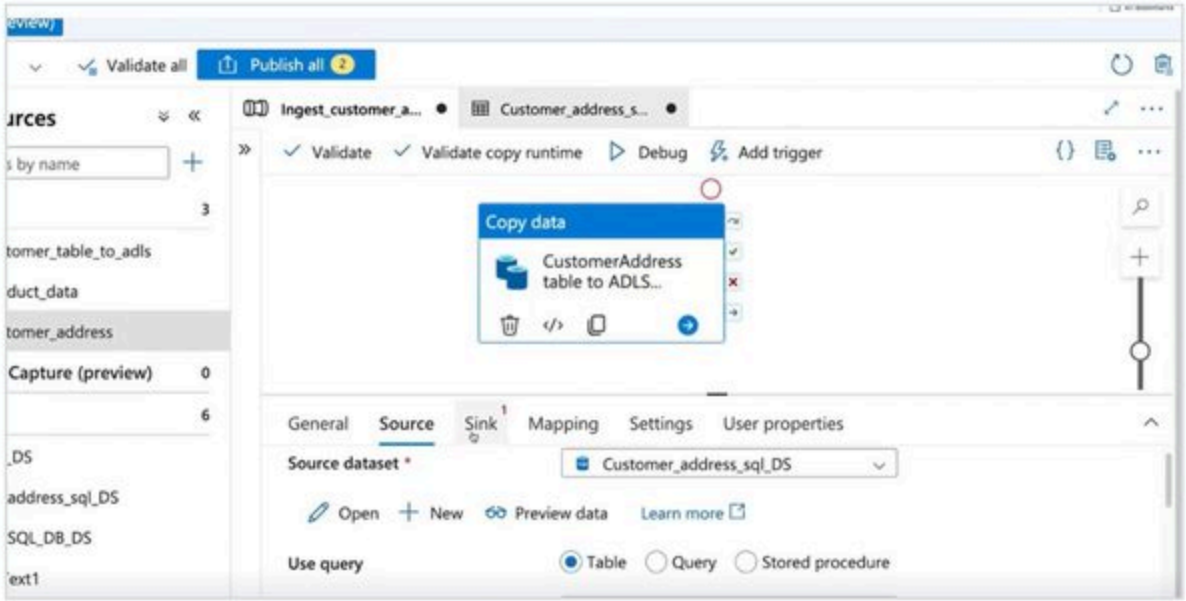


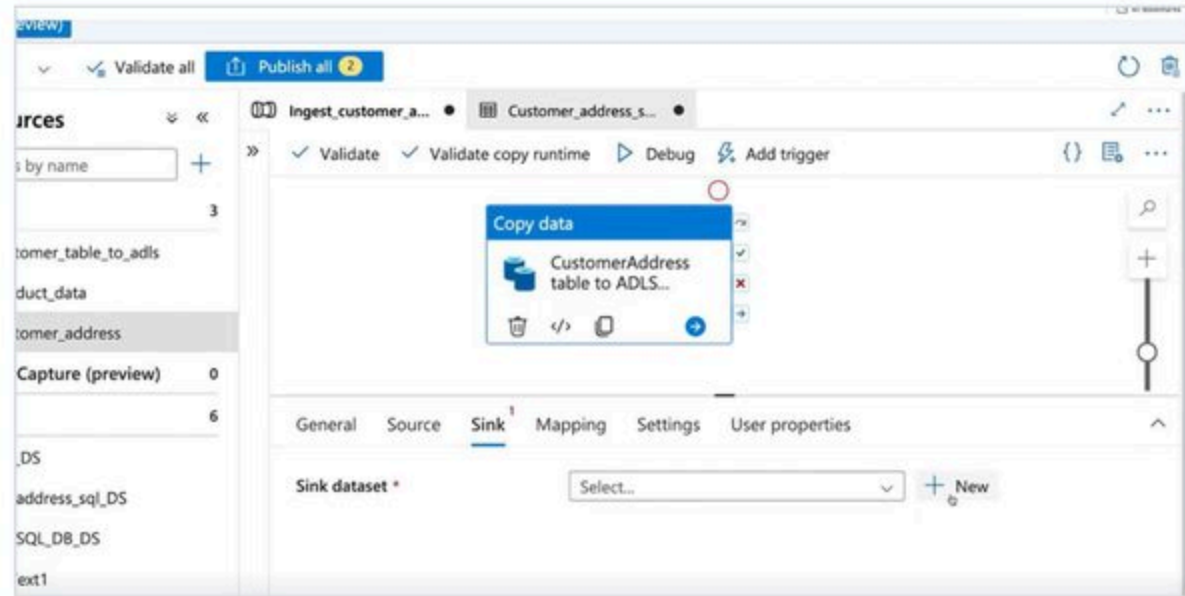


- Table drop-down → SalesLT.CustomerAddress
- Connection tab: linked service + table
- Preview data to check

PART 2

Step 4 – Sink: new ADLS Gen2 dataset

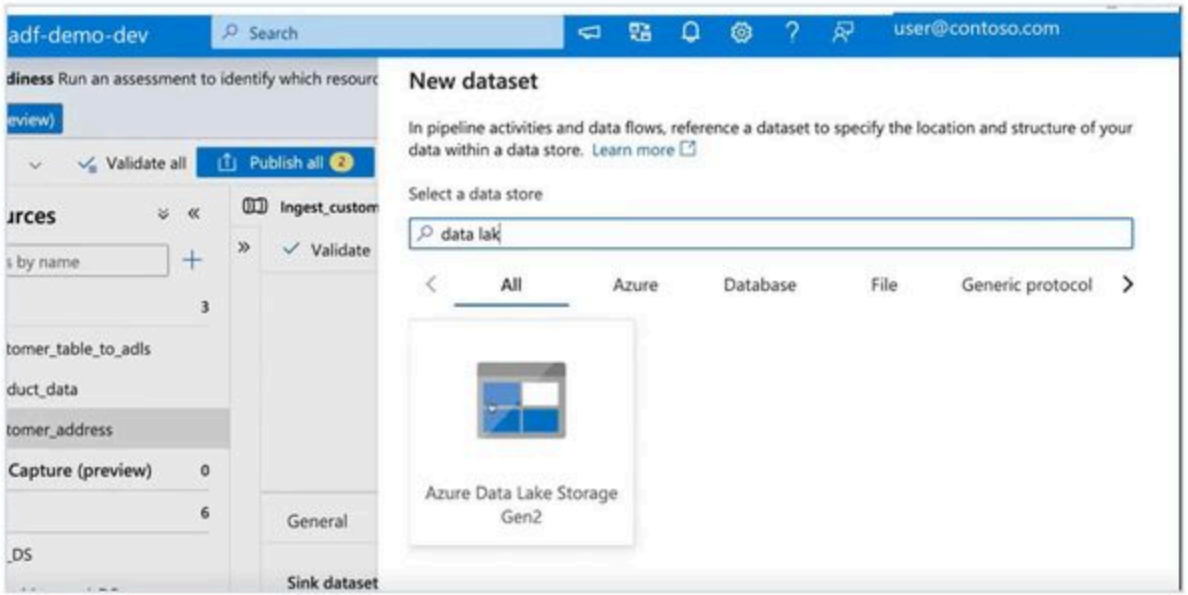


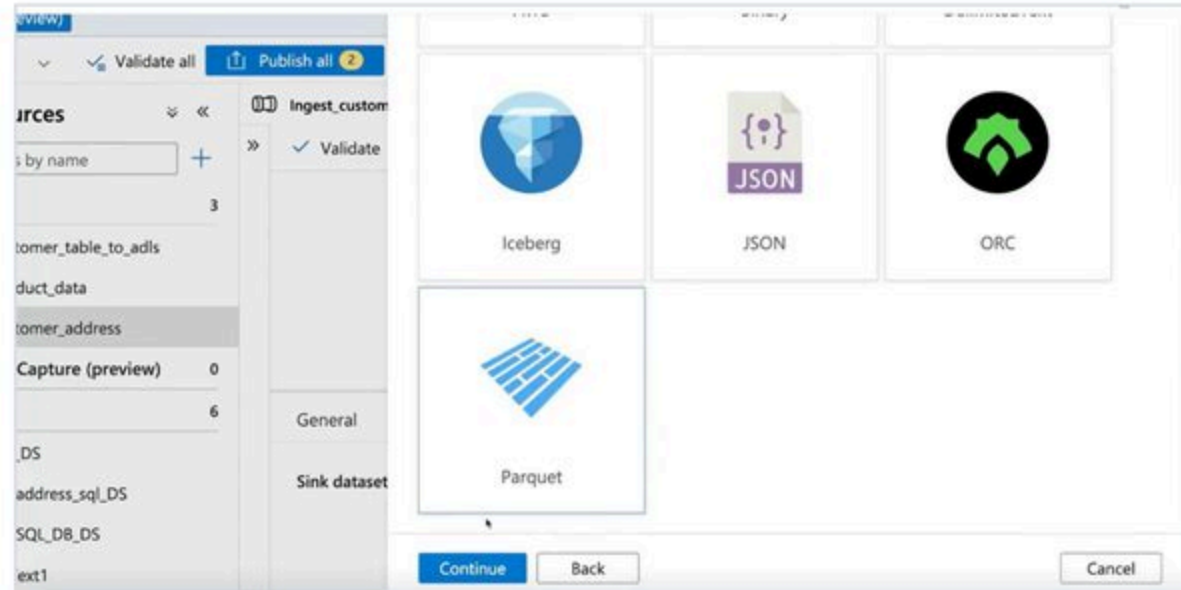


- Source dataset now selected; Use query = Table
- Sink tab → + New
- Search "data lake" → Azure Data Lake Storage Gen2

PART 2

Step 4 – Choose the Parquet format

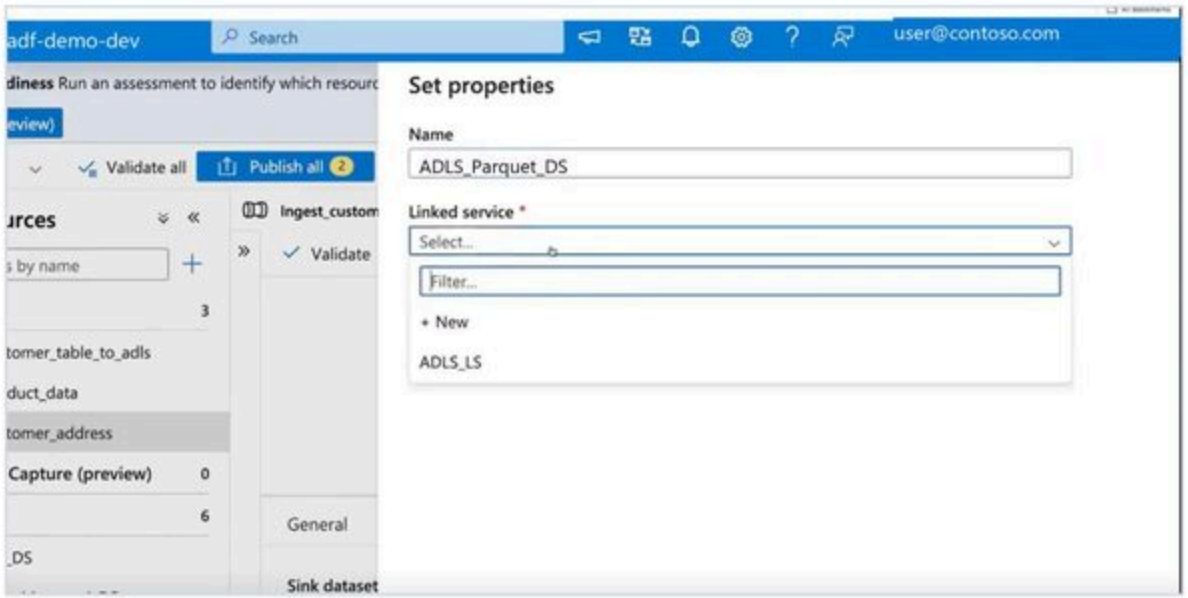


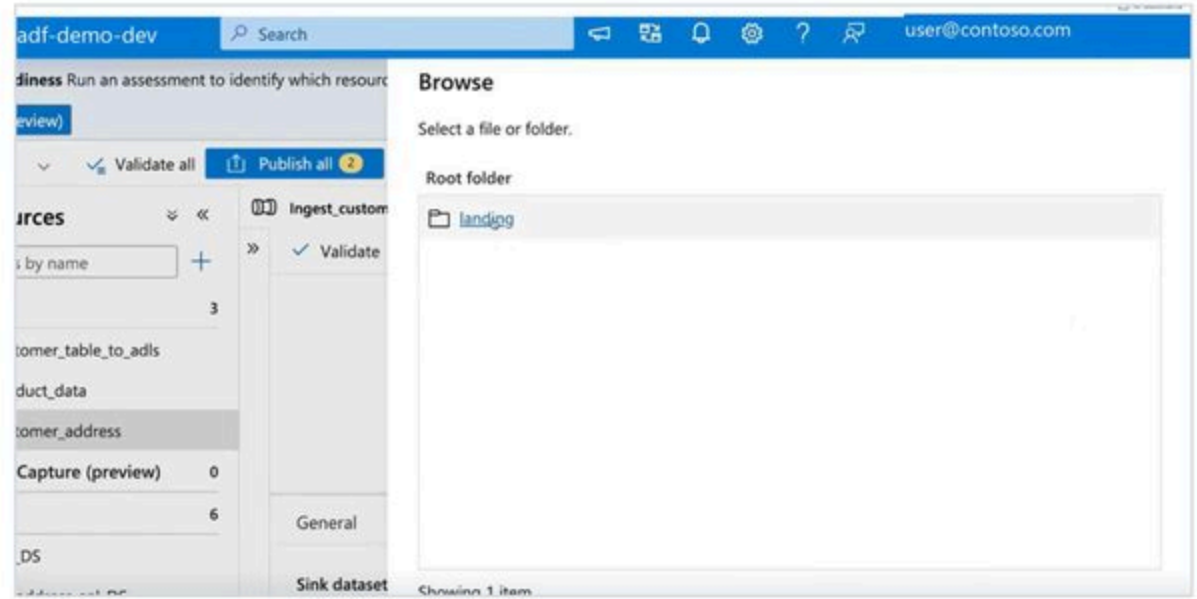


- Formats: Avro, Binary, DelimitedText, Excel, Iceberg, JSON, ORC, Parquet, XML
- Select Parquet → Continue

PART 2

Step 4 – ADLS_Parquet_DS: linked service and path

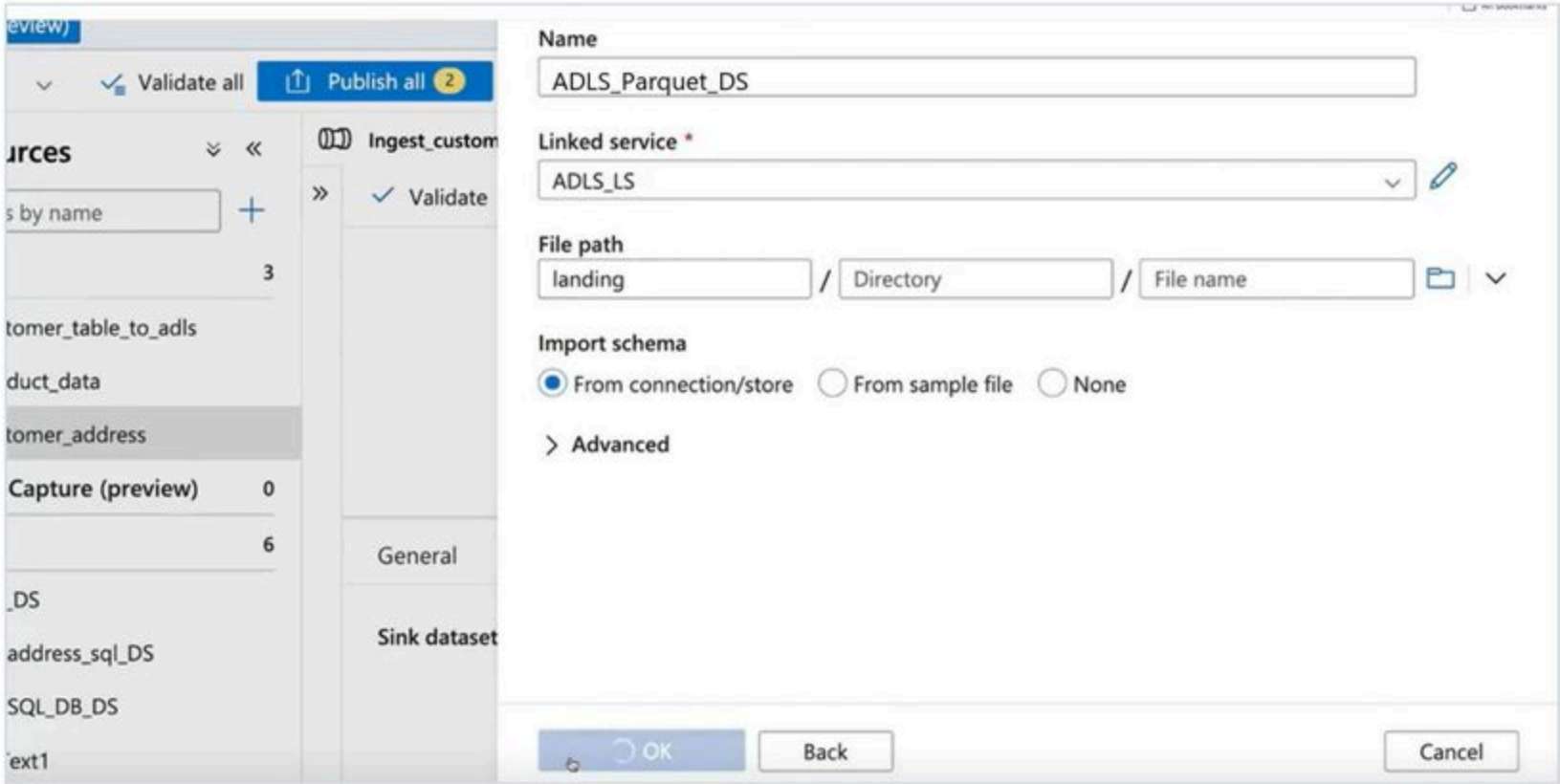




- Name: ADLS_Parquet_DS
- Linked service: ADLS_LS (reused)
- Browse → landing container

PART 2

Step 4 – File path and import schema

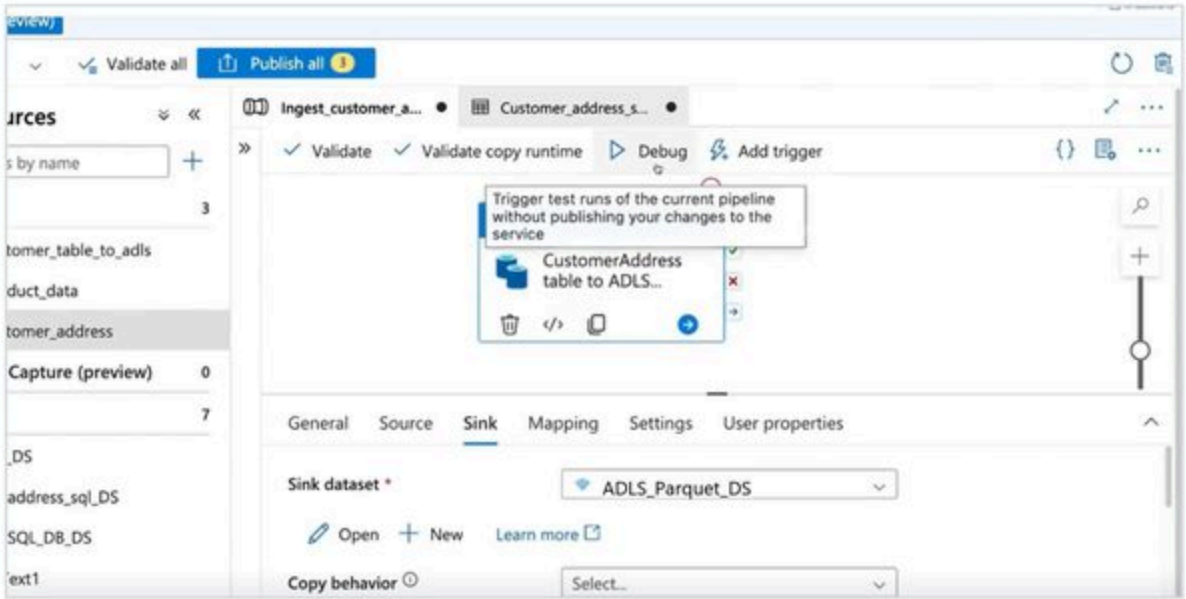


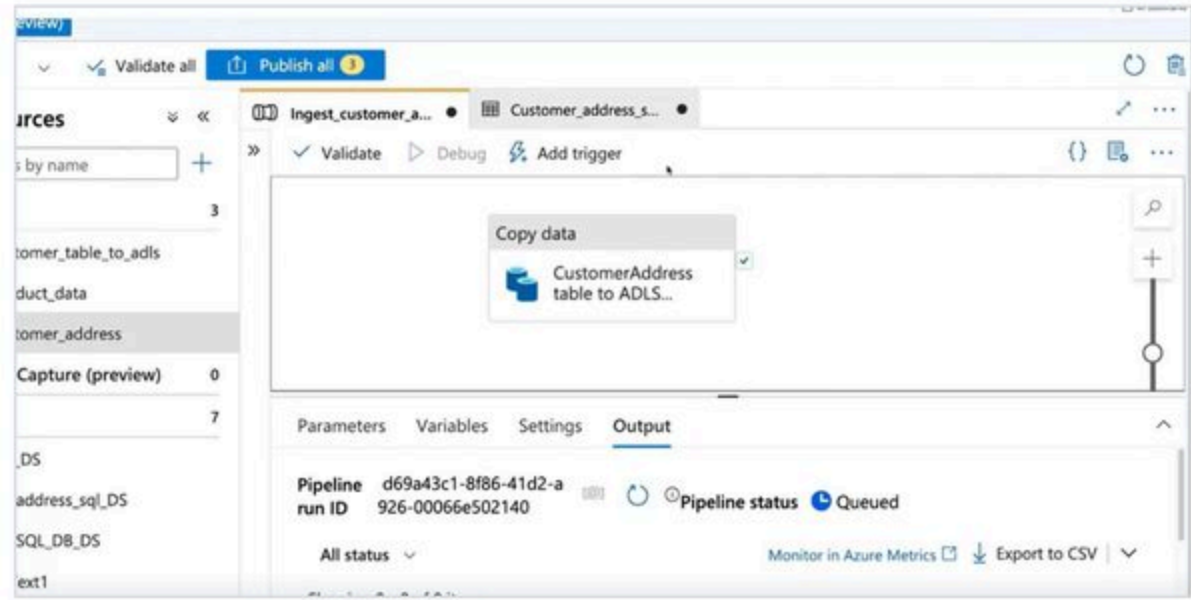
- File path: landing / (directory) / (file name)
- Leave directory and file name empty
- Import schema: None (no file yet) – then OK

⚠ Import schema "From connection/store" needs an existing file. For a new sink, choose None.

PART 2

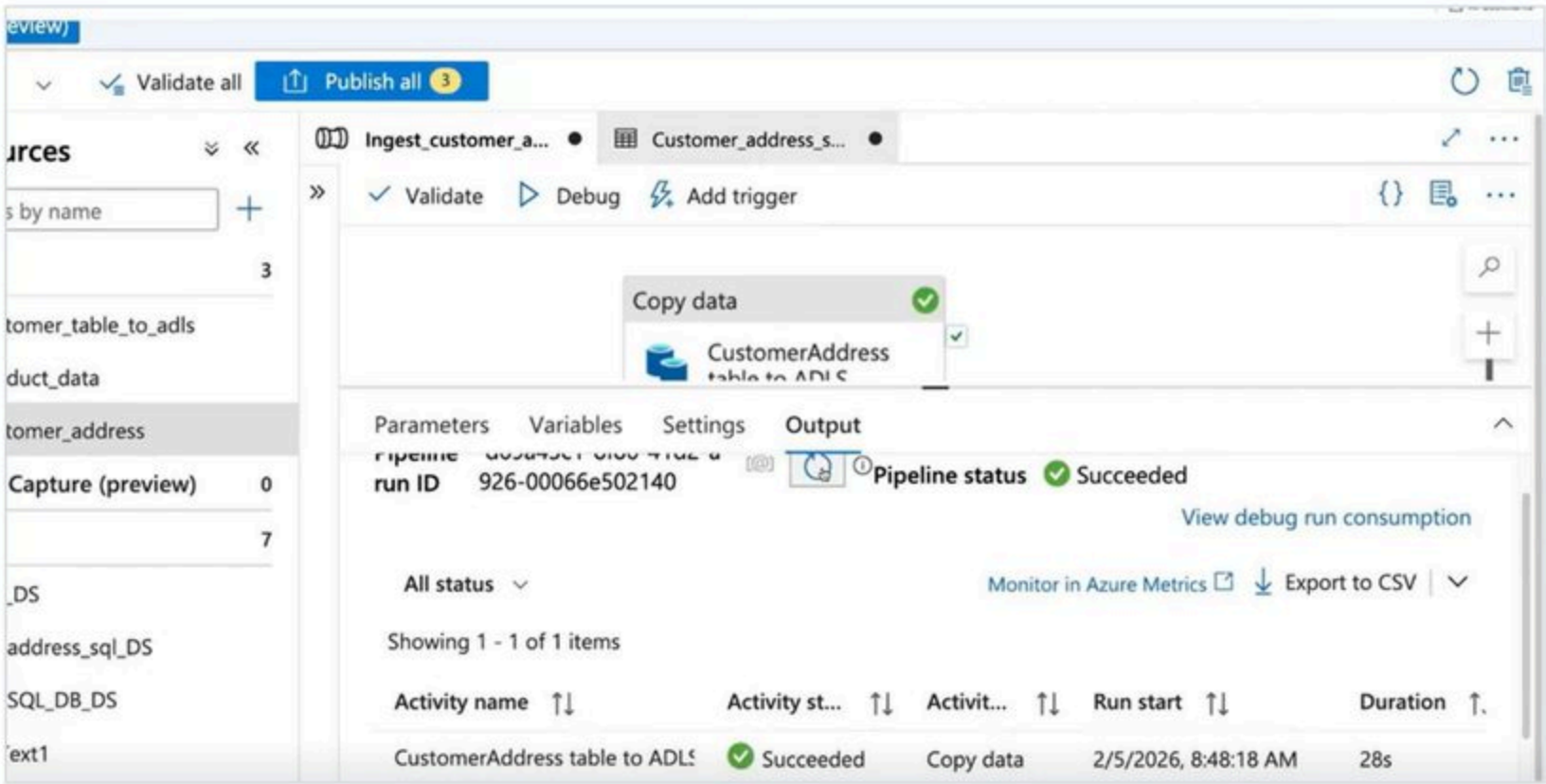
Step 5 – Debug the pipeline





- Sink dataset: ADLS_Parquet_DS
- Debug = test run without publishing
- Output tab: Queued → In progress → Succeeded

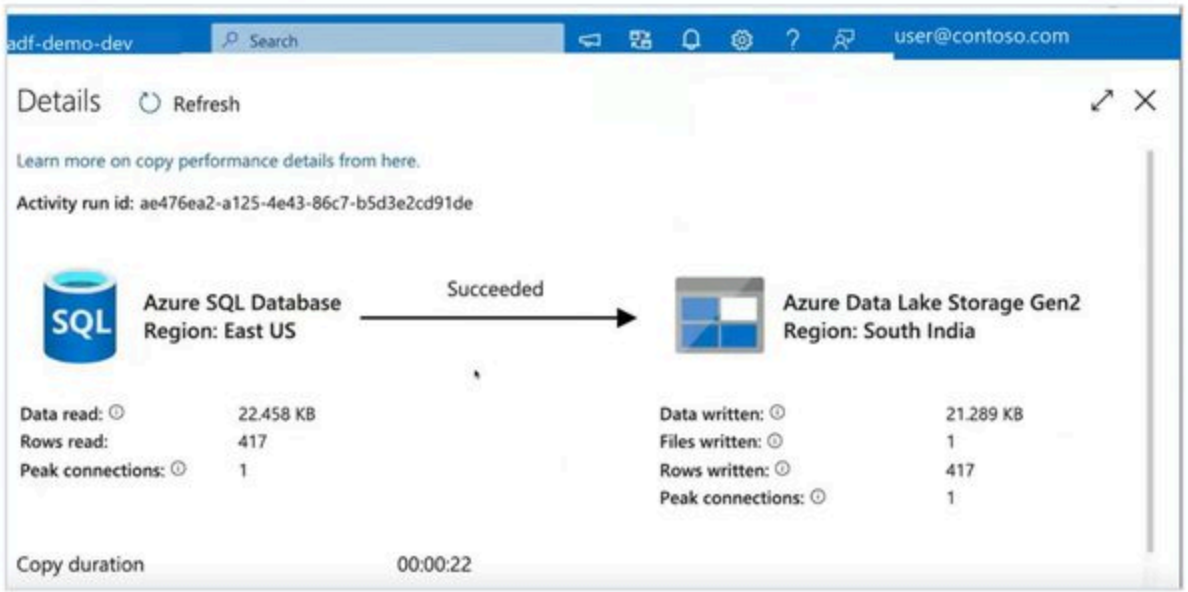
Debug succeeded

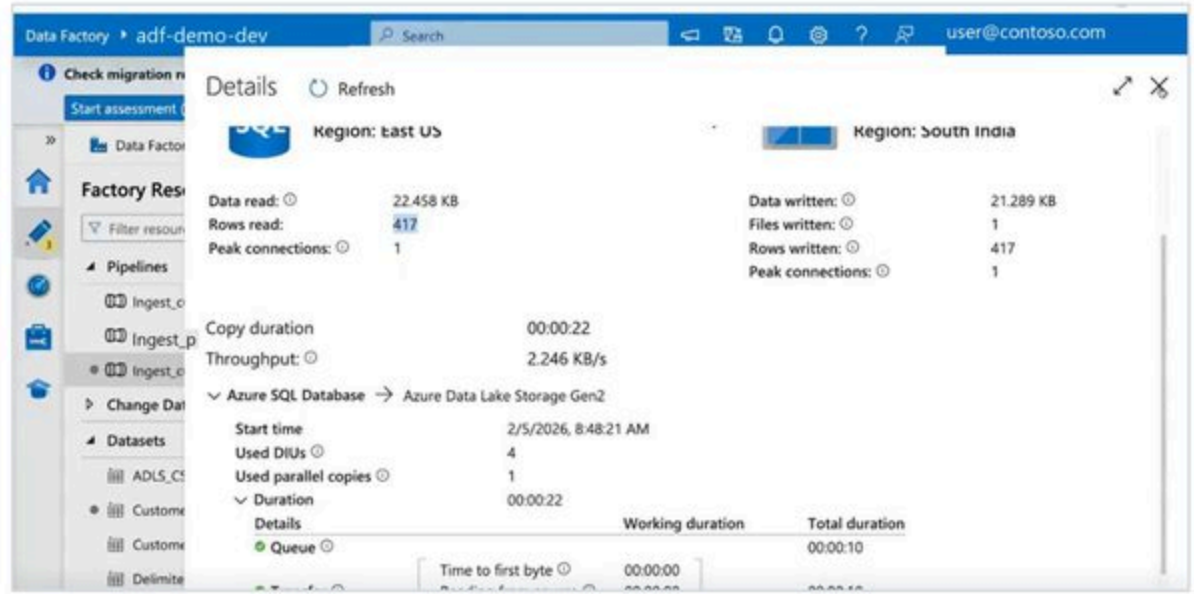


- Pipeline status: Succeeded
- Activity: CustomerAddress table to ADLS parquet
- Duration ≈ 28 s

PART 2

Copy details – rows and regions

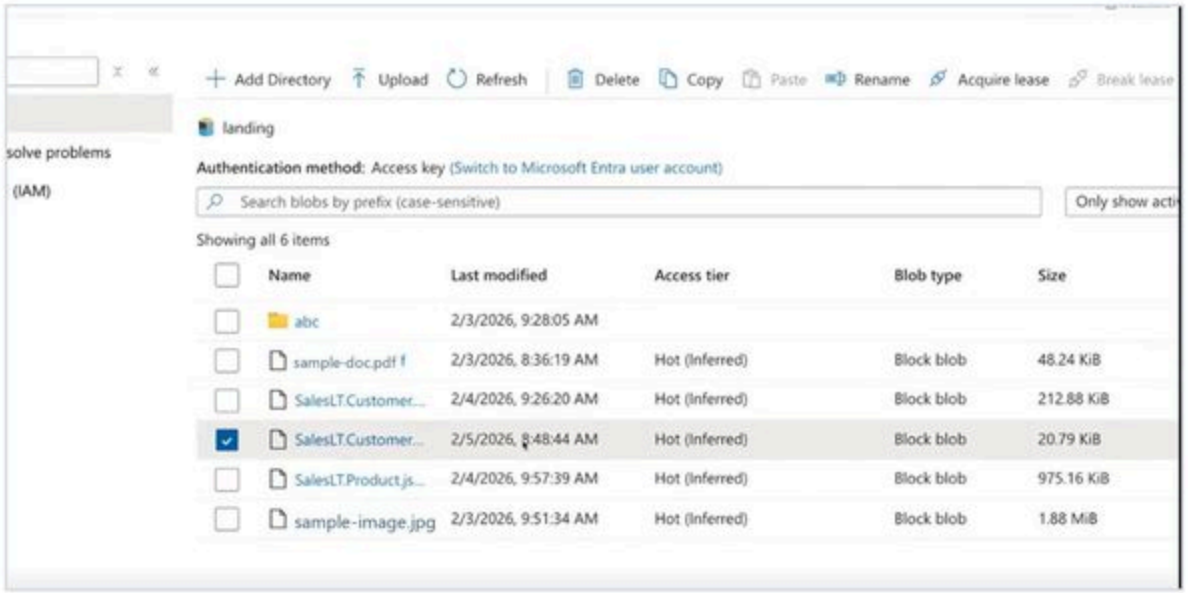


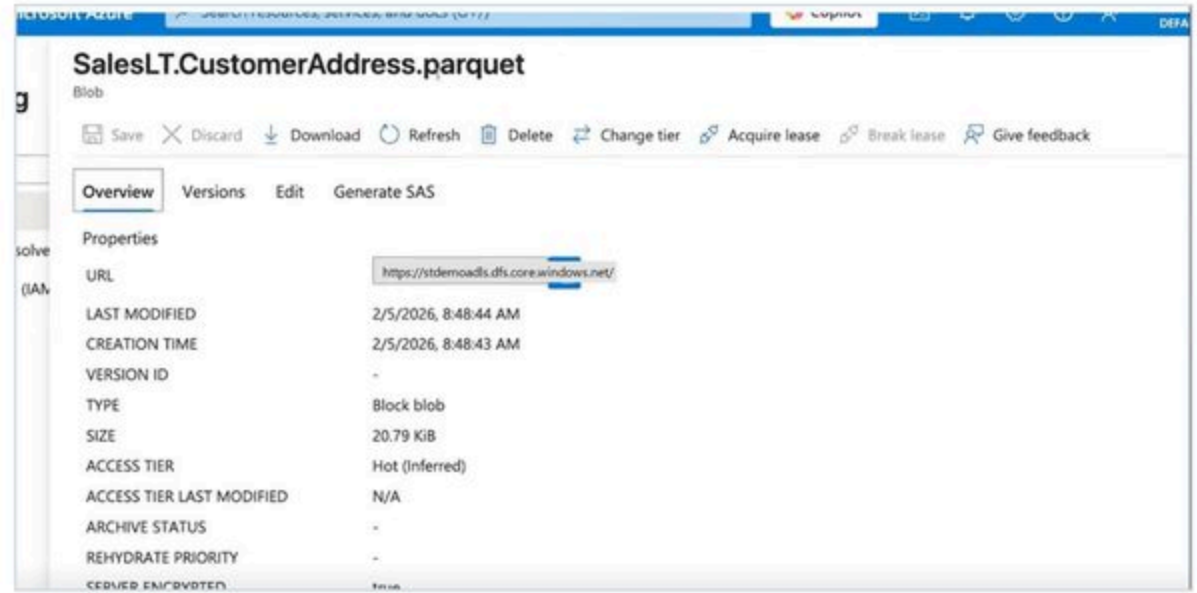


- Rows read 417 = rows written 417
- Data read ≈ 22.5 KB → written ≈ 21.3 KB (Parquet)
- Source region East US → sink region South India
- Used DIUs 4, parallel copies 1

PART 2

Validate in ADLS – the Parquet file

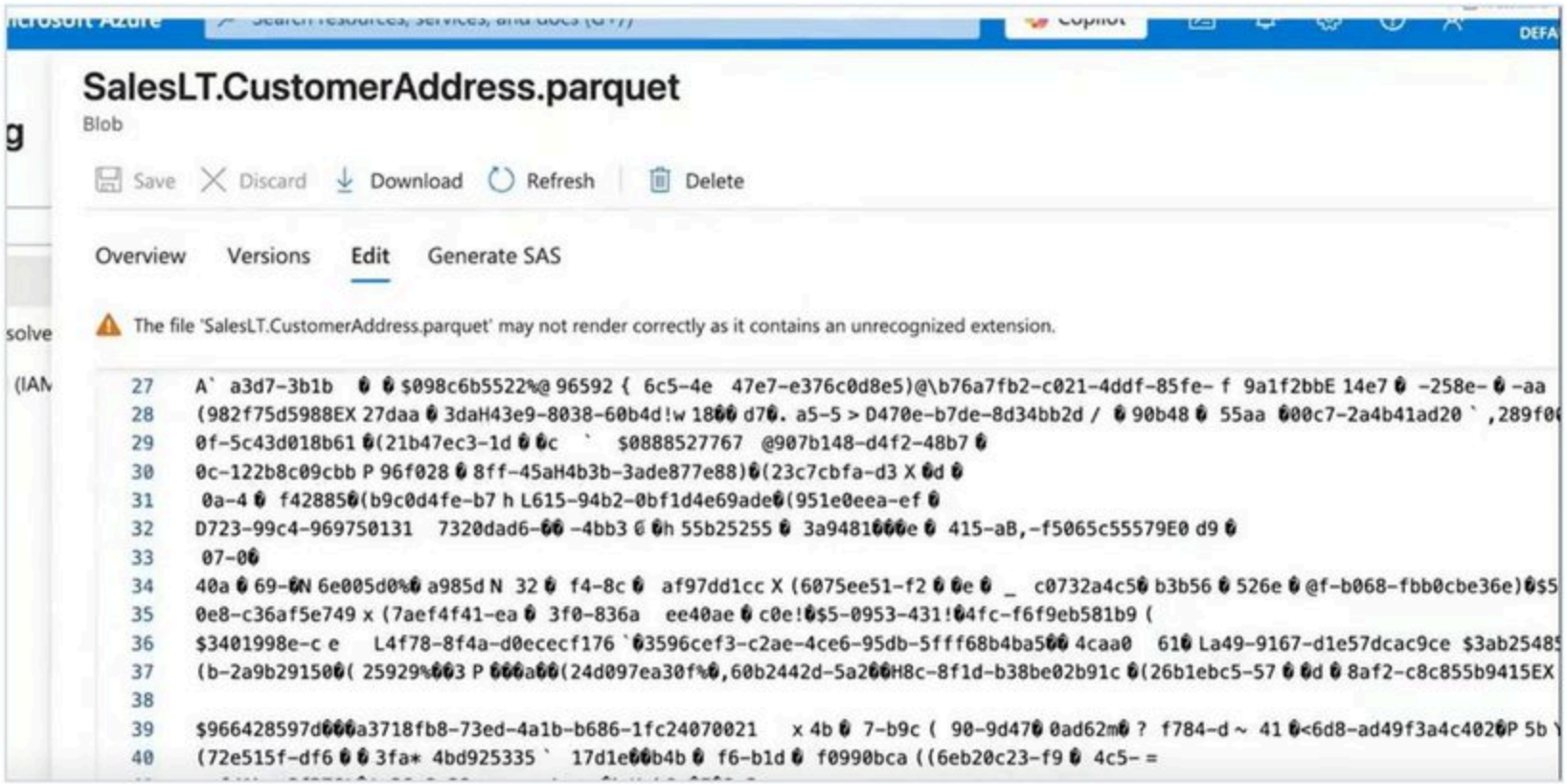




- landing → SalesLT.CustomerAddress.parquet
- Named after the source table
- Size ≈ 20.8 KiB

PART 2

Parquet is binary – you cannot read it in Notepad



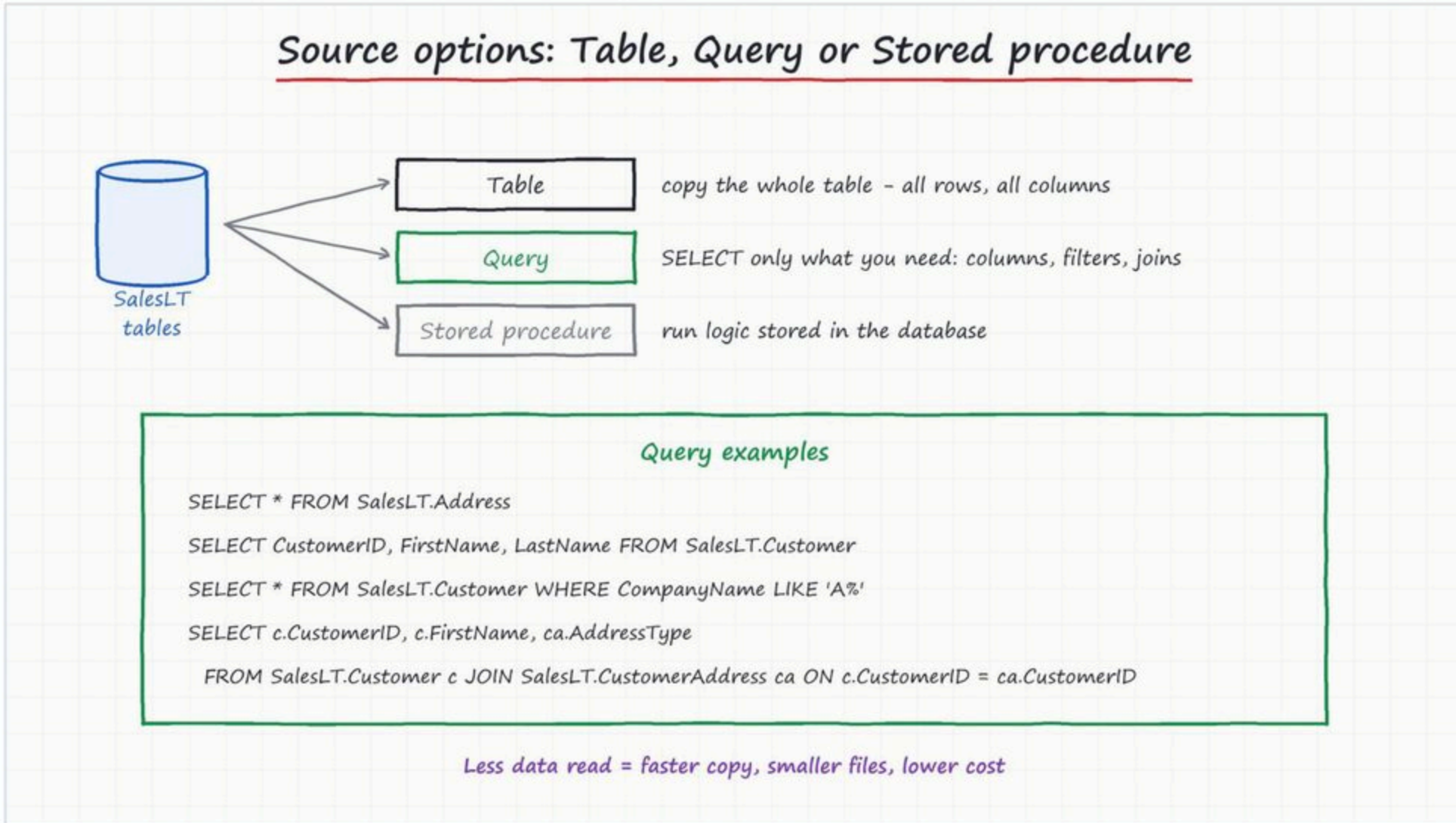
- Edit tab: "may not render correctly … unrecognized extension"
- Binary columnar content, not text
- Read it with Databricks, Synapse, Fabric, Spark, Power BI

PART 3

Source queries and joins

Lab 2: Query → JSON • Lab 3: Join two tables

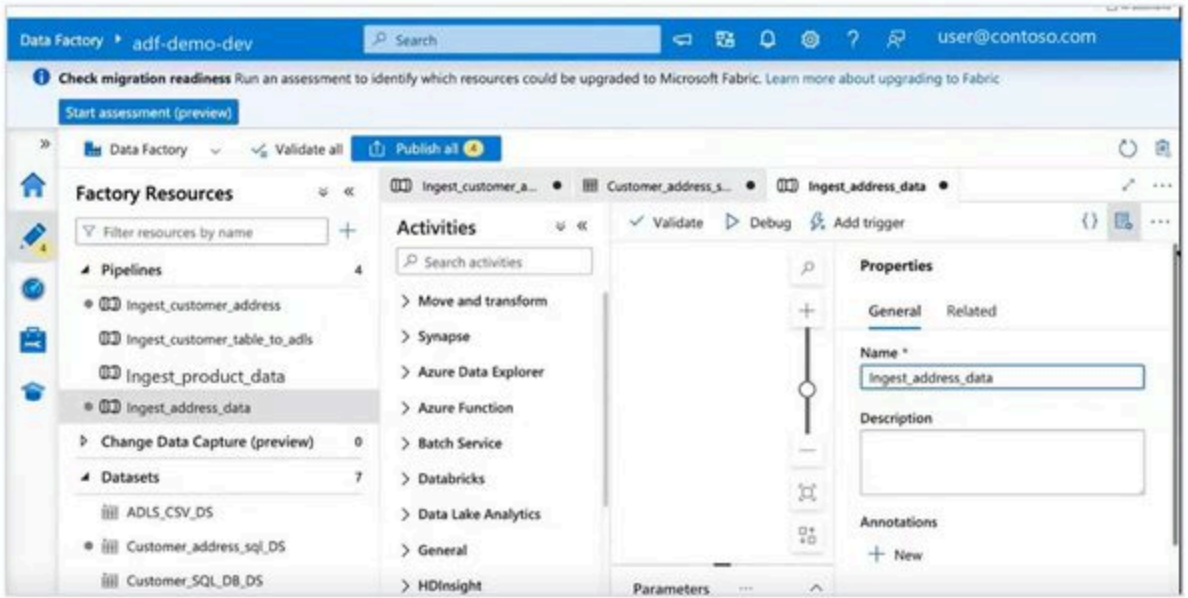
Table, Query or Stored procedure?

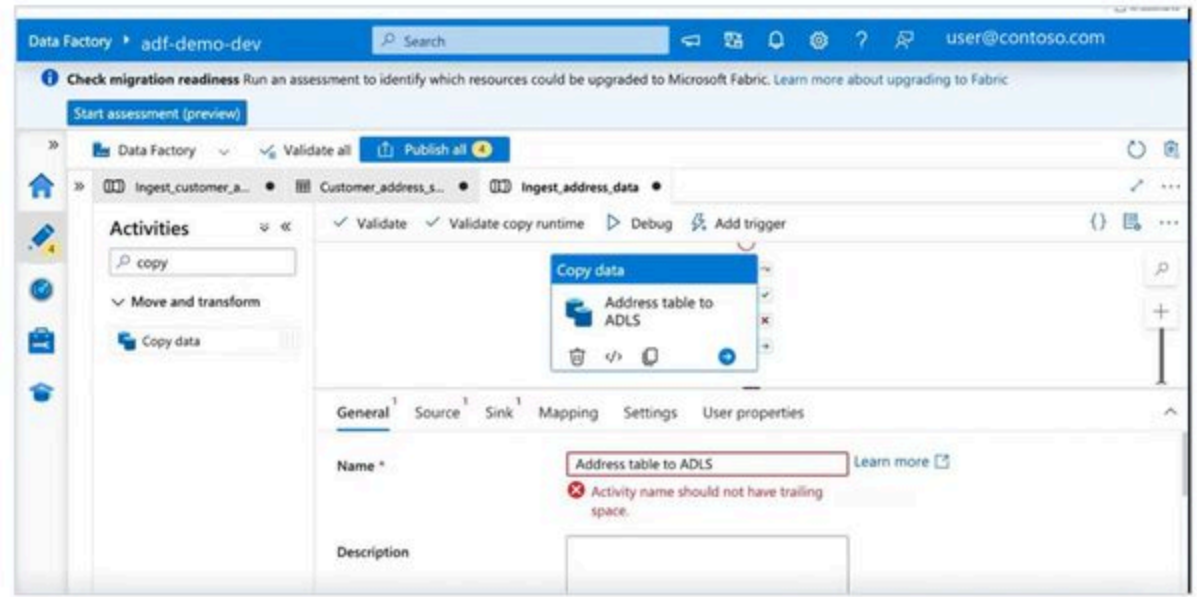


Prepared by Ir. UWAYO Jacques · Follow me on LinkedIn: linkedin.com/in/uwayo

PART 3

Lab 2 – Pipeline Ingest_address_data



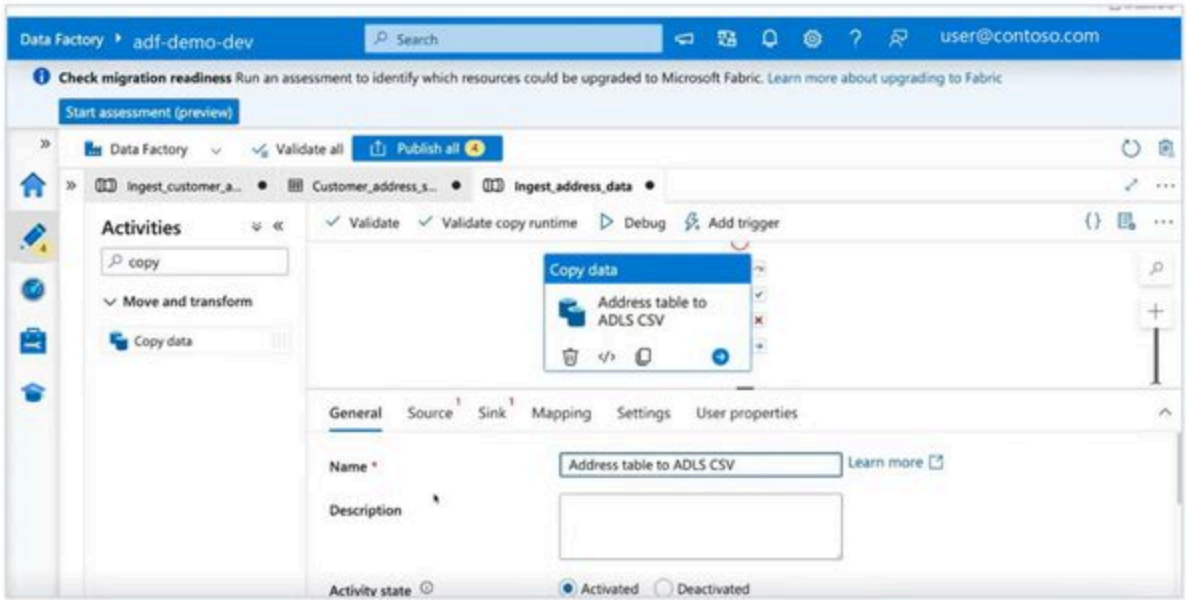


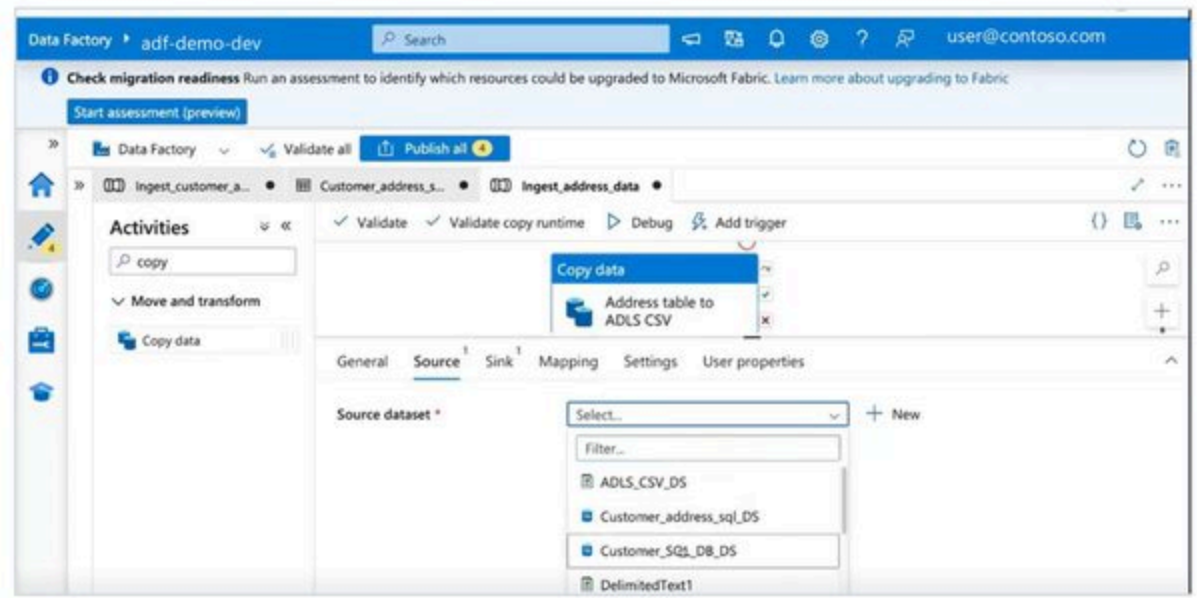
- New pipeline: Ingest_address_data
- Copy data activity

- Error: "Activity name should not have trailing space"

PART 3

Lab 2 – Reuse an existing SQL dataset



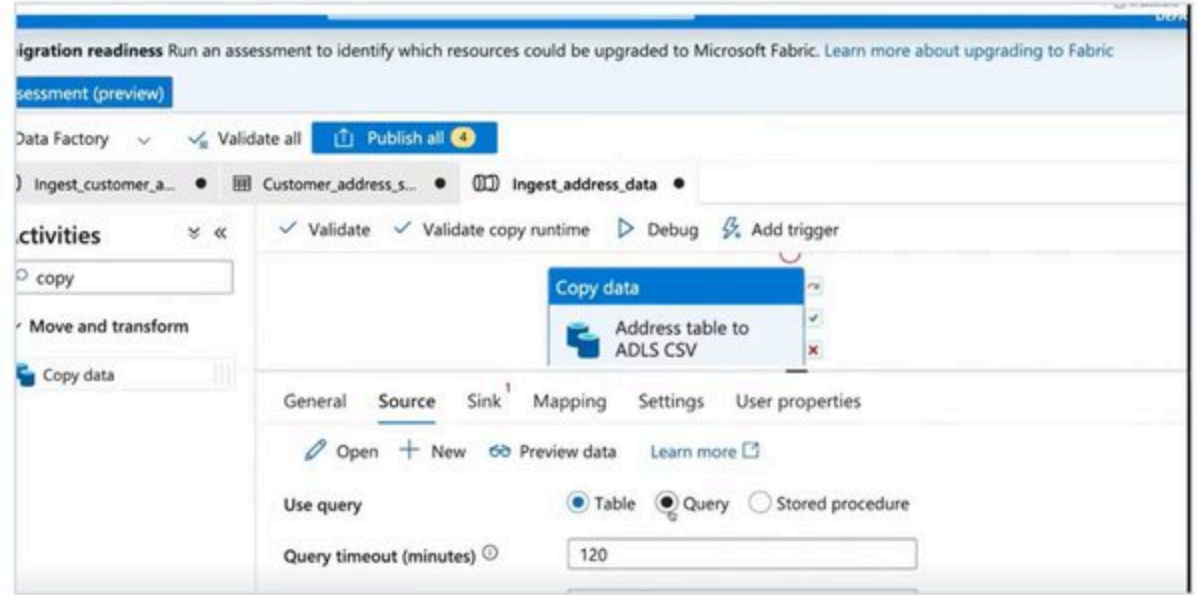


- Activity name: Address table to ADLS
- Source dataset: any dataset on SQL_DB_LS
- With Use query = Query the table setting is ignored

PART 3

Lab 2 – Use query: Query

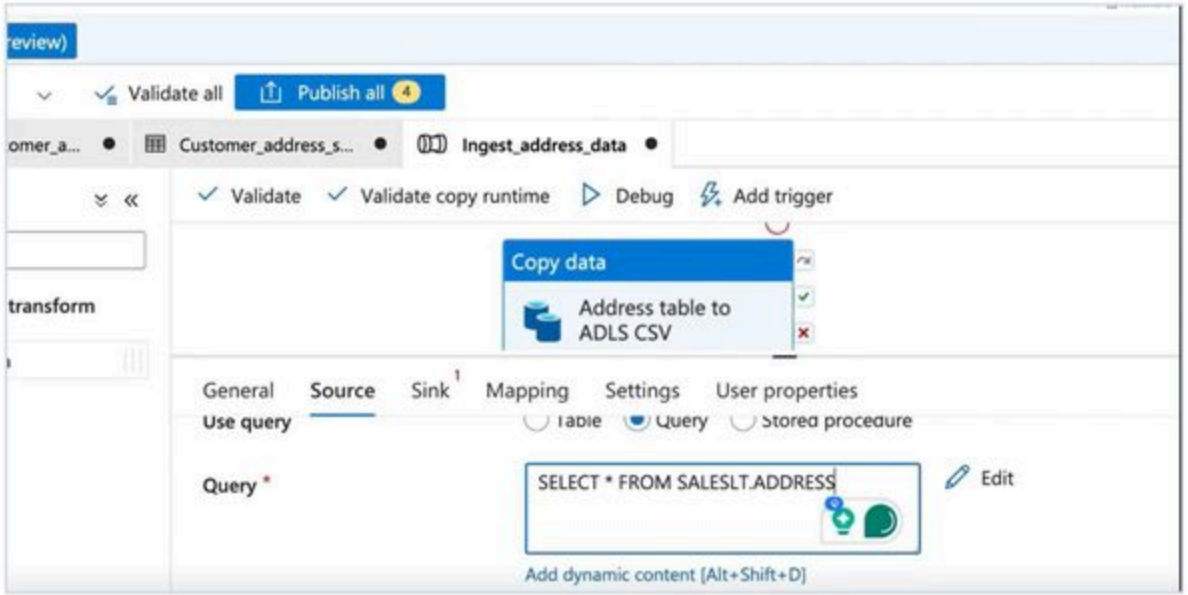


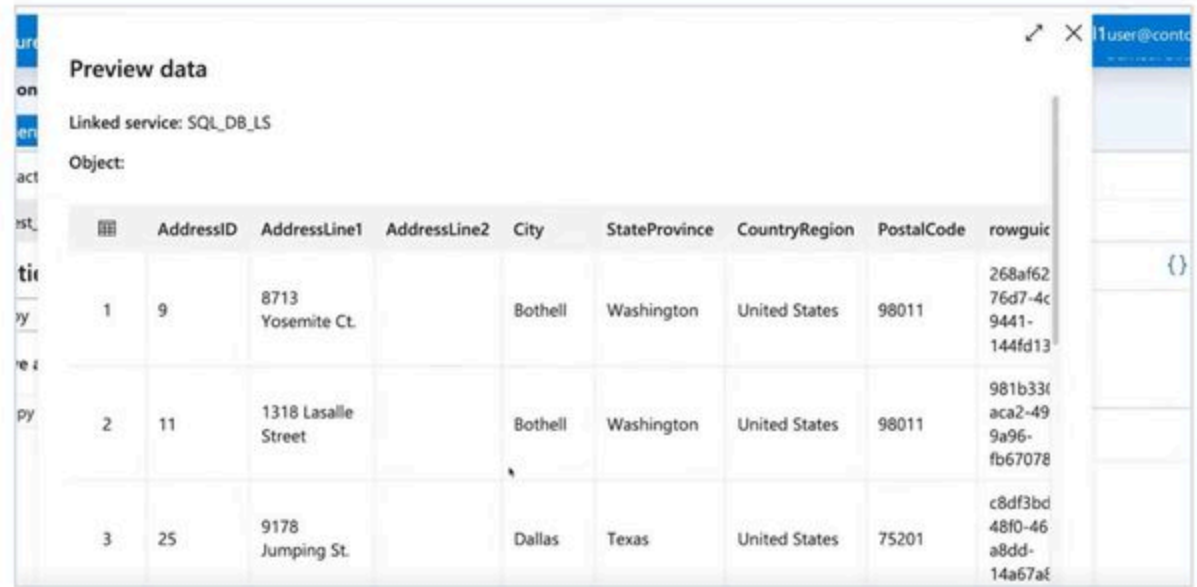


- Preview data shows the dataset's own table (Customer)
- Use query → Query
- Query timeout default: 120 minutes

PART 3

Lab 2 – Write the query

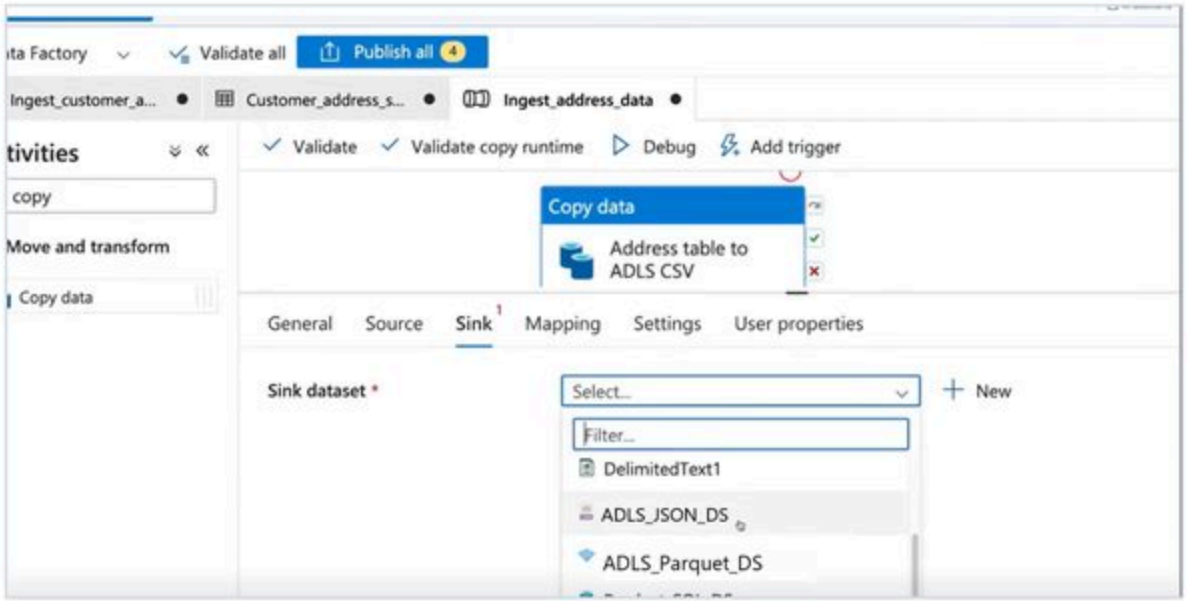


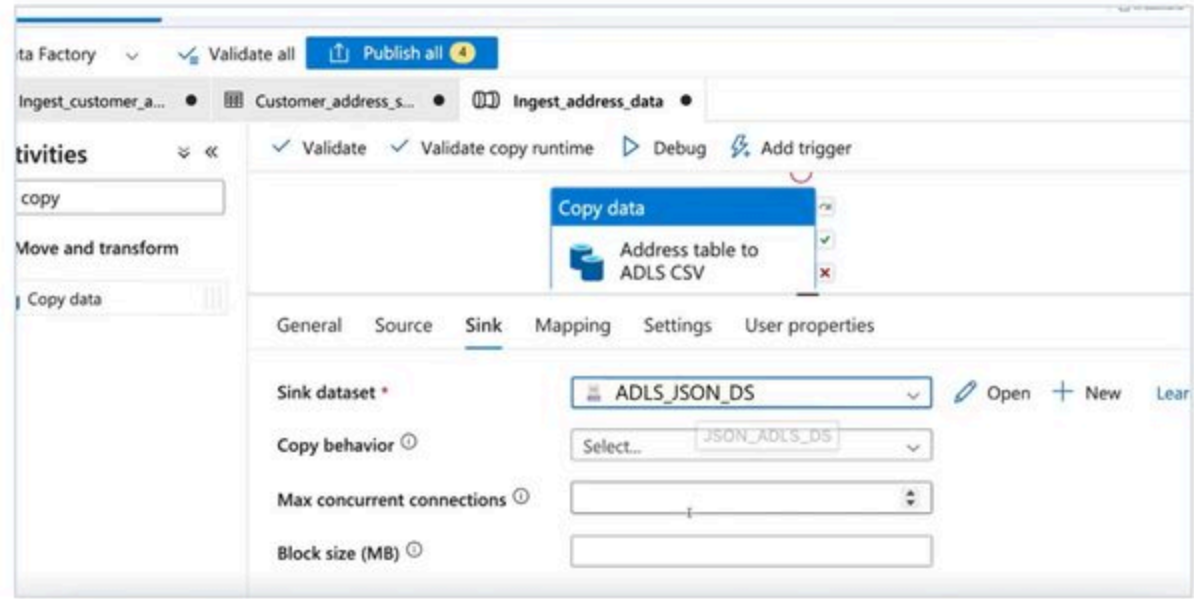


- SELECT * FROM SalesLT.Address
- Preview data now shows Address rows
- Query box supports Add dynamic content

PART 3

Lab 2 – Sink: ADLS_JSON_DS

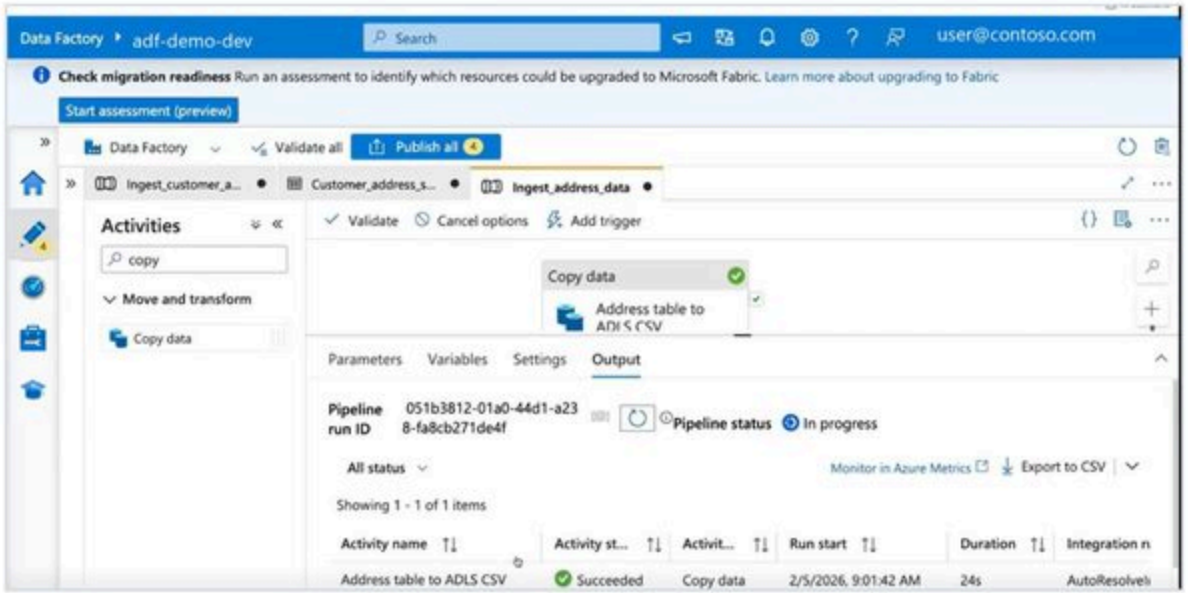


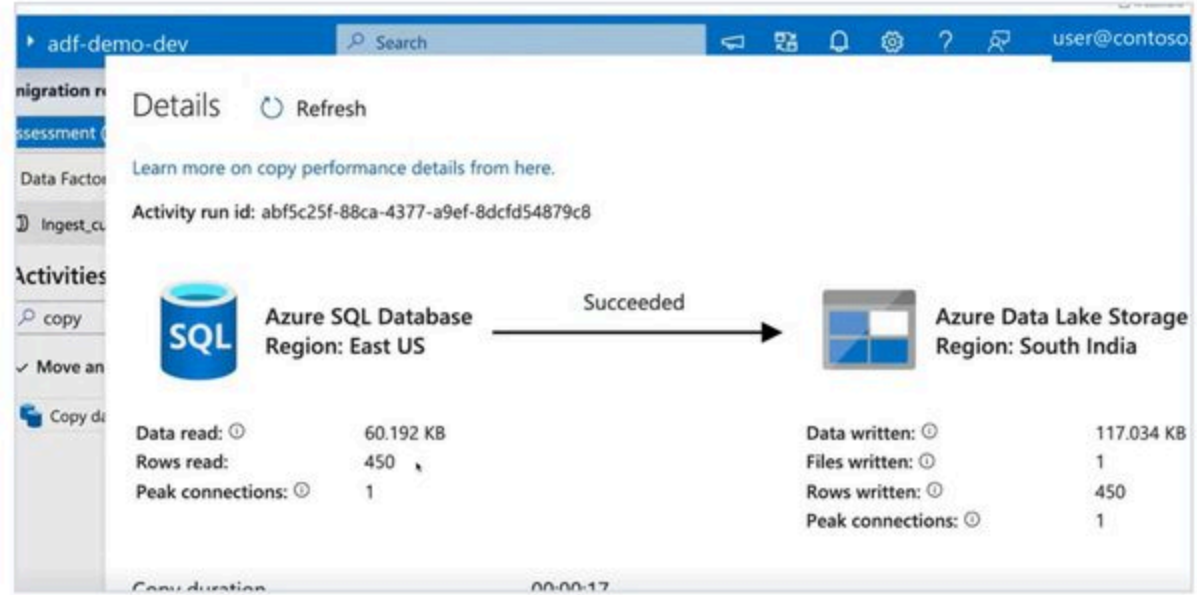


- Sink dataset list: ADLS_JSON_DS, ADLS_Parquet_DS, DelimitedText1 …
- Choose ADLS_JSON_DS (JSON in landing)
- Same sink dataset works for any table

PART 3

Lab 2 – Debug and copy details

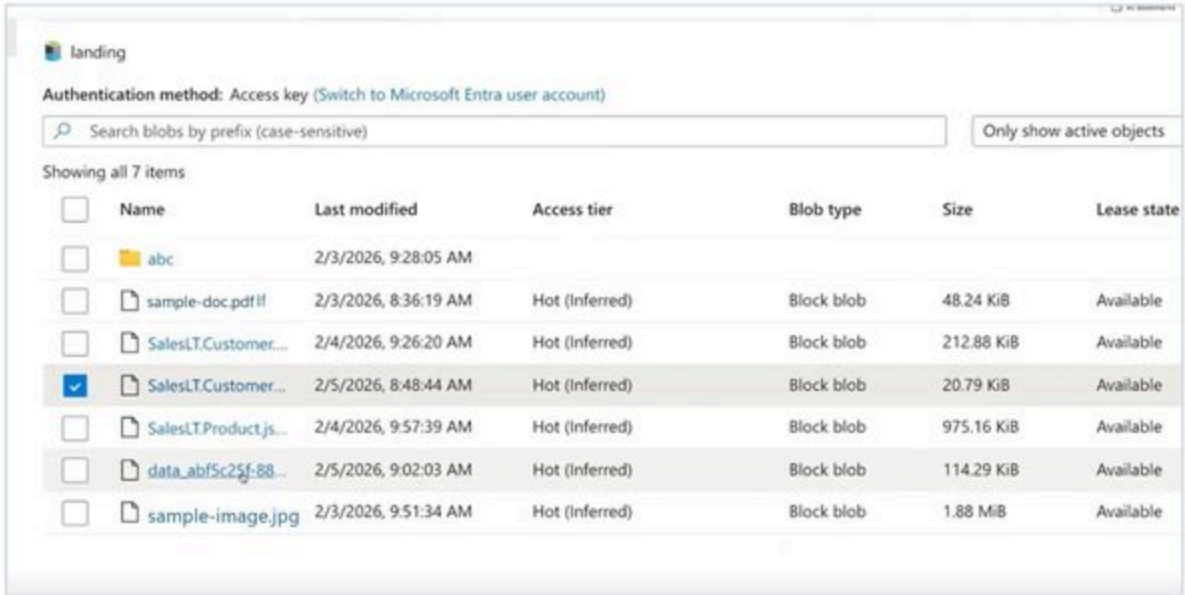


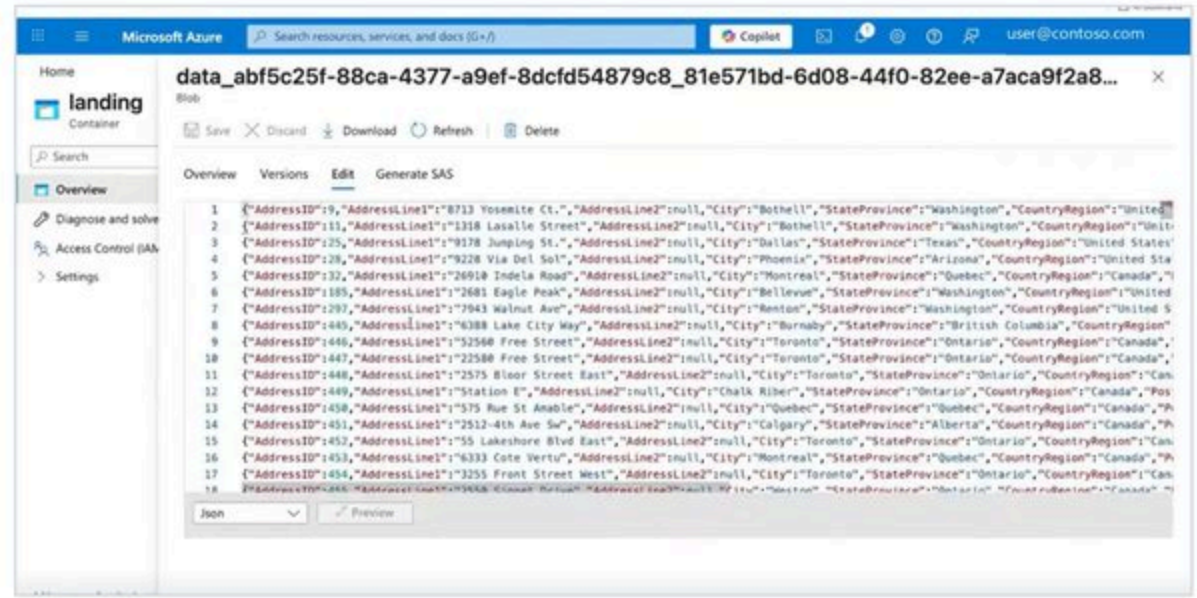


- Succeeded in ~24 s
- Rows read 450 = rows written 450
- Read ≈ 60 KB → JSON written ≈ 117 KB

PART 3

Lab 2 – Validate: the file is called data_<guid>





- New file: data_abf5c25f-… (not SalesLT.Address)
- With a query ADF cannot use a table name
- One JSON object per line

Where does my file go – and what is it called?

Where does my file go – and what is it called?

Sink dataset setting		Result in ADLS	Why
Source = Table, no file name	→	landing/SalesLT.CustomerAddress.parquet	named after the table
Source = Query, no file name	→	landing/data_<run-guid>.json	no table name to use
Directory = Customer	→	landing/Customer/data_<guid>.csv	folder created for you
File name = customer.csv	→	landing/customer.csv	you choose the name

Tip: the folder does not need to exist before the copy

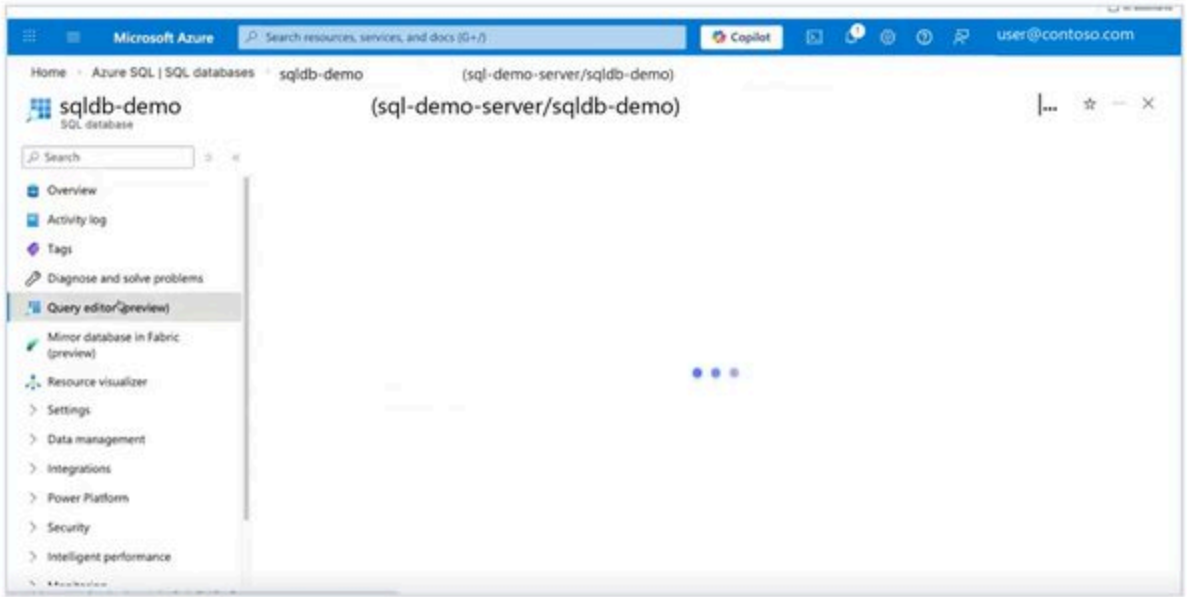
But "Import schema: From connection/store" fails on a folder that does not exist yet

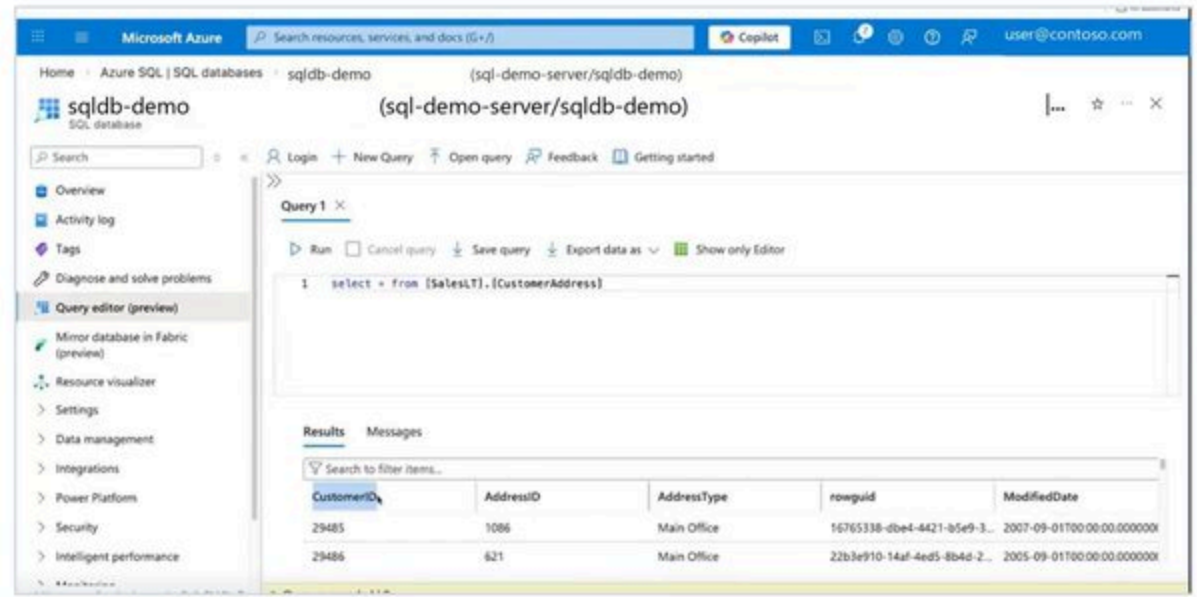
-> choose Import schema = None for new sink folders

Prepared by Ir. UWAYO Jacques · Follow me on LinkedIn: linkedin.com/in/uwayo

PART 3

Lab 3 – Explore the tables to join

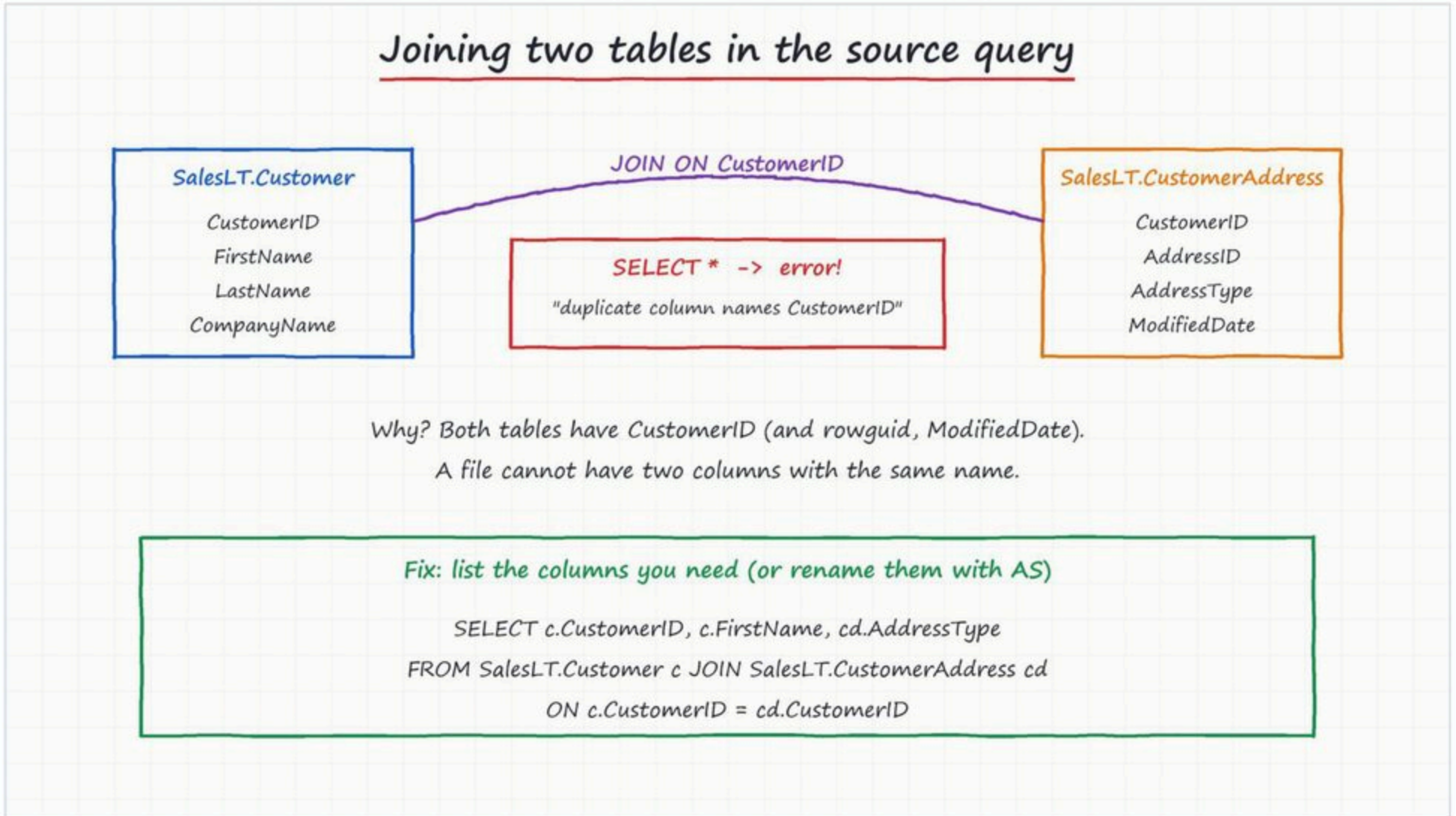




- Database → Query editor (preview)
- Firewall error? Allowlist your IP (see Session 03)

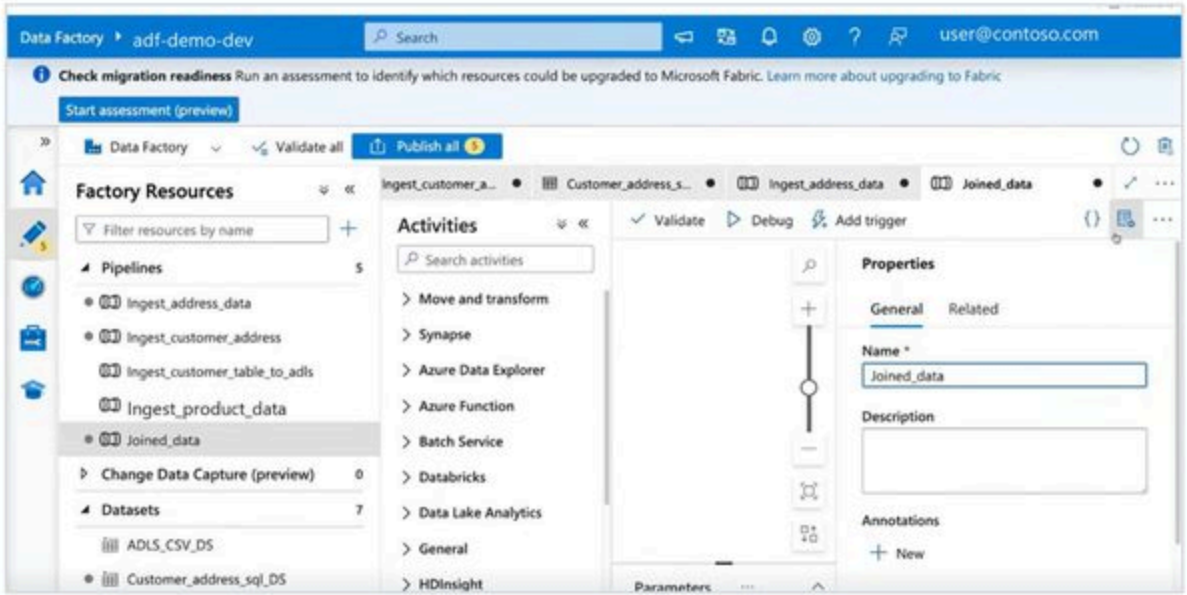
- SELECT * FROM SalesLT.CustomerAddress

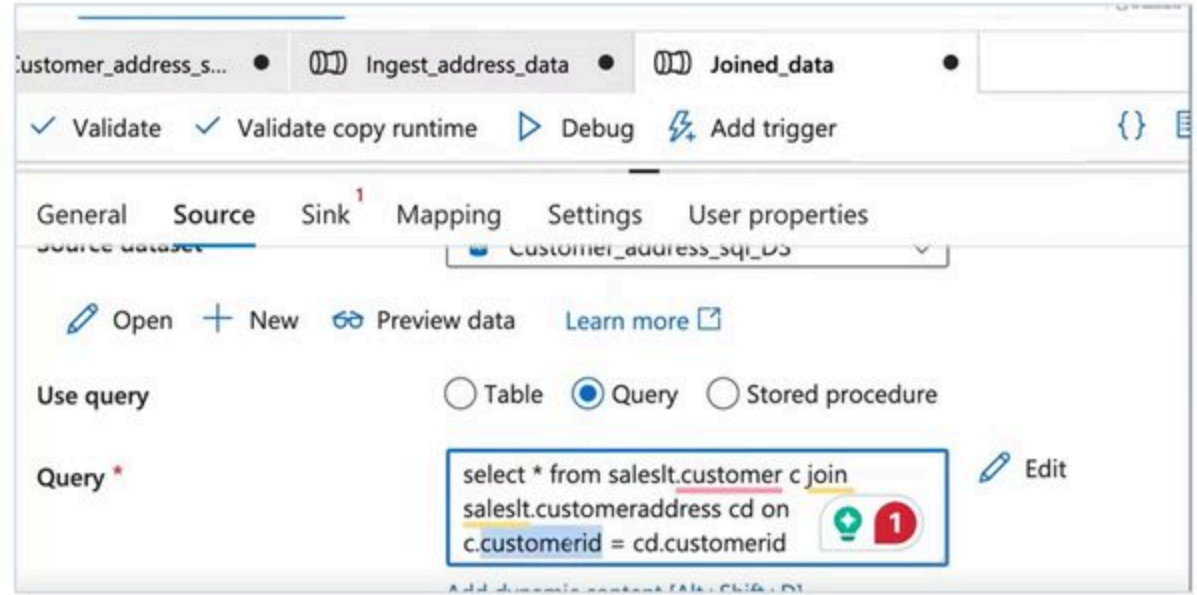
Joining two tables in the source query



PART 3

Lab 3 – Pipeline Joined_data and the join query



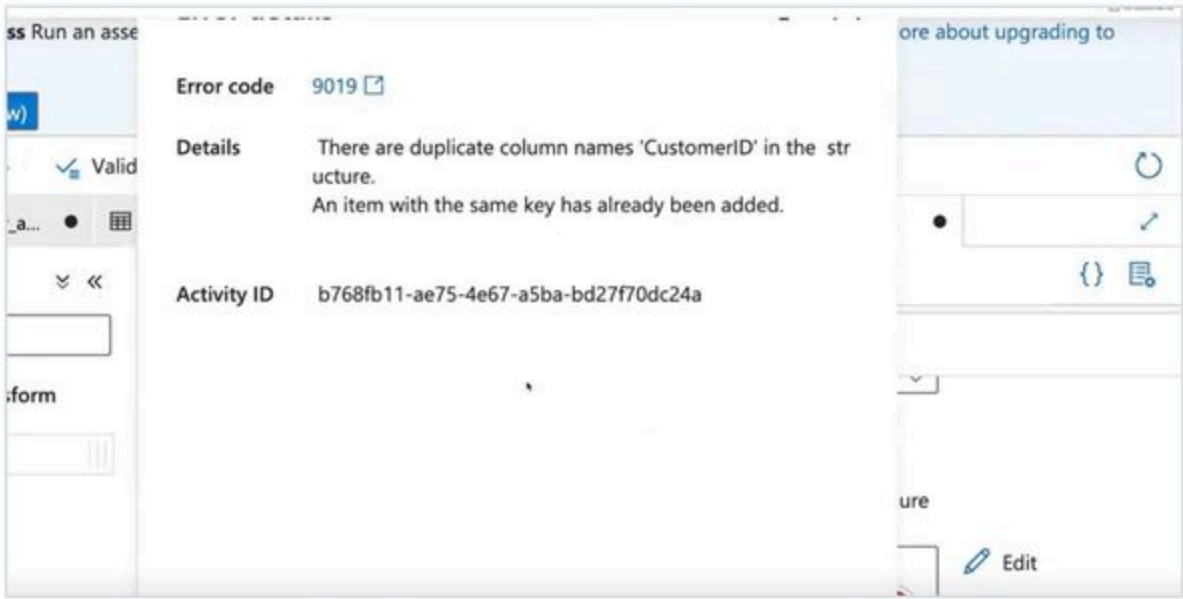


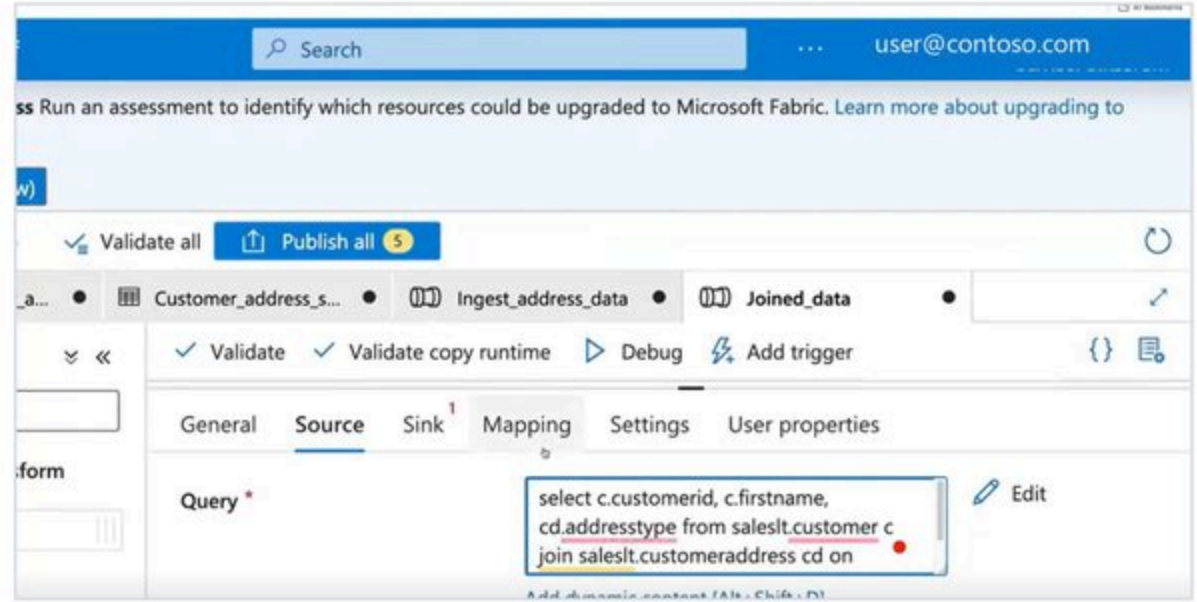
- New pipeline: Joined_data

- select * from salesLT.customer c join salesLT.customeraddress cd on c.customerid = cd.customerid

PART 3

Lab 3 – Error: duplicate column names





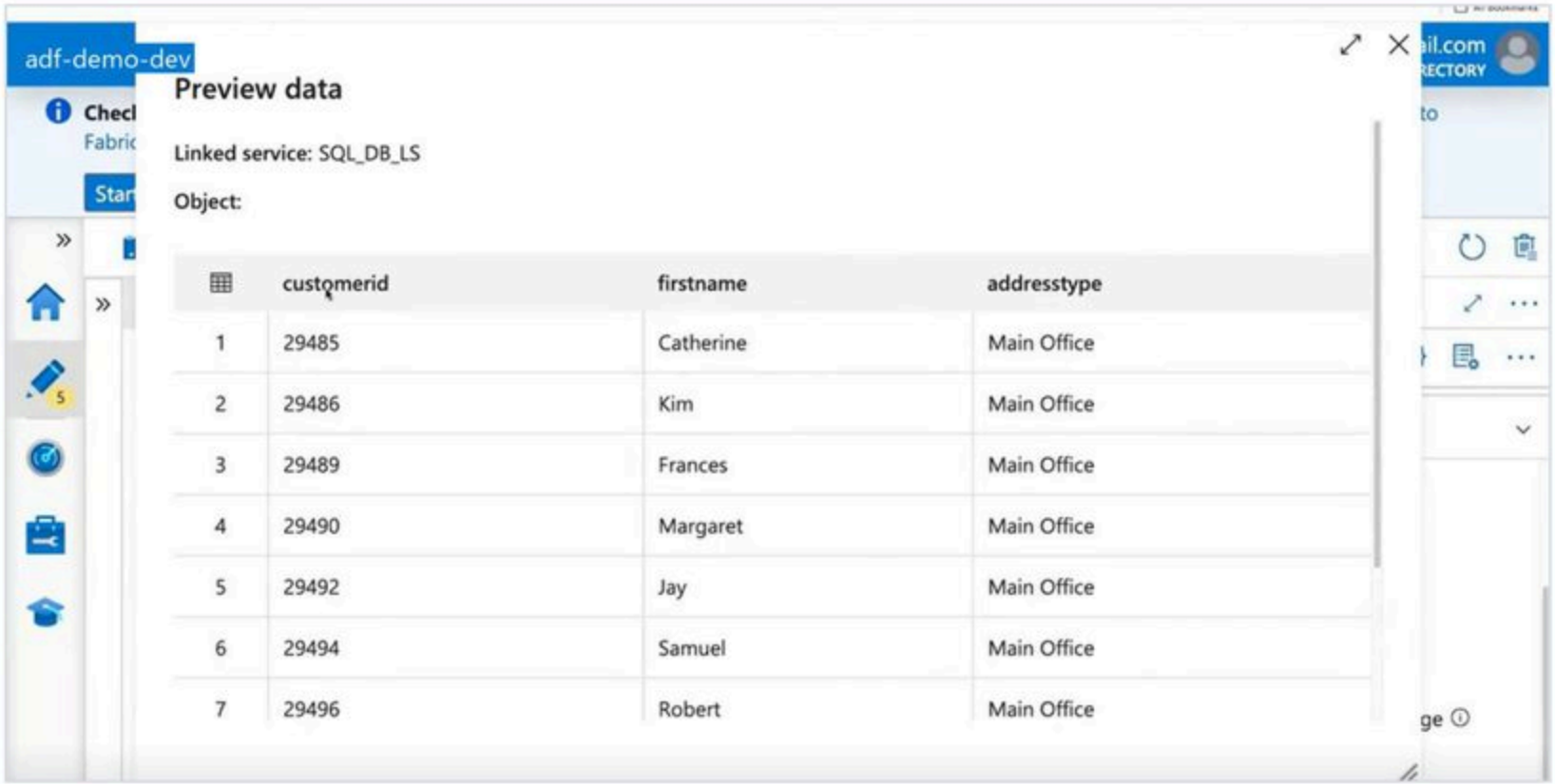
- Error code 9019: duplicate column names “CustomerID”
- Fix: select explicit columns

- select c.customerid, c.firstname, cd.addresstype …

⚠ Never use SELECT * on a join in a pipeline – list the columns.

PART 3

Lab 3 – Preview the joined result



	customerid	firstname	addresstype
1	29485	Catherine	Main Office
2	29486	Kim	Main Office
3	29489	Frances	Main Office
4	29490	Margaret	Main Office
5	29492	Jay	Main Office
6	29494	Samuel	Main Office
7	29496	Robert	Main Office

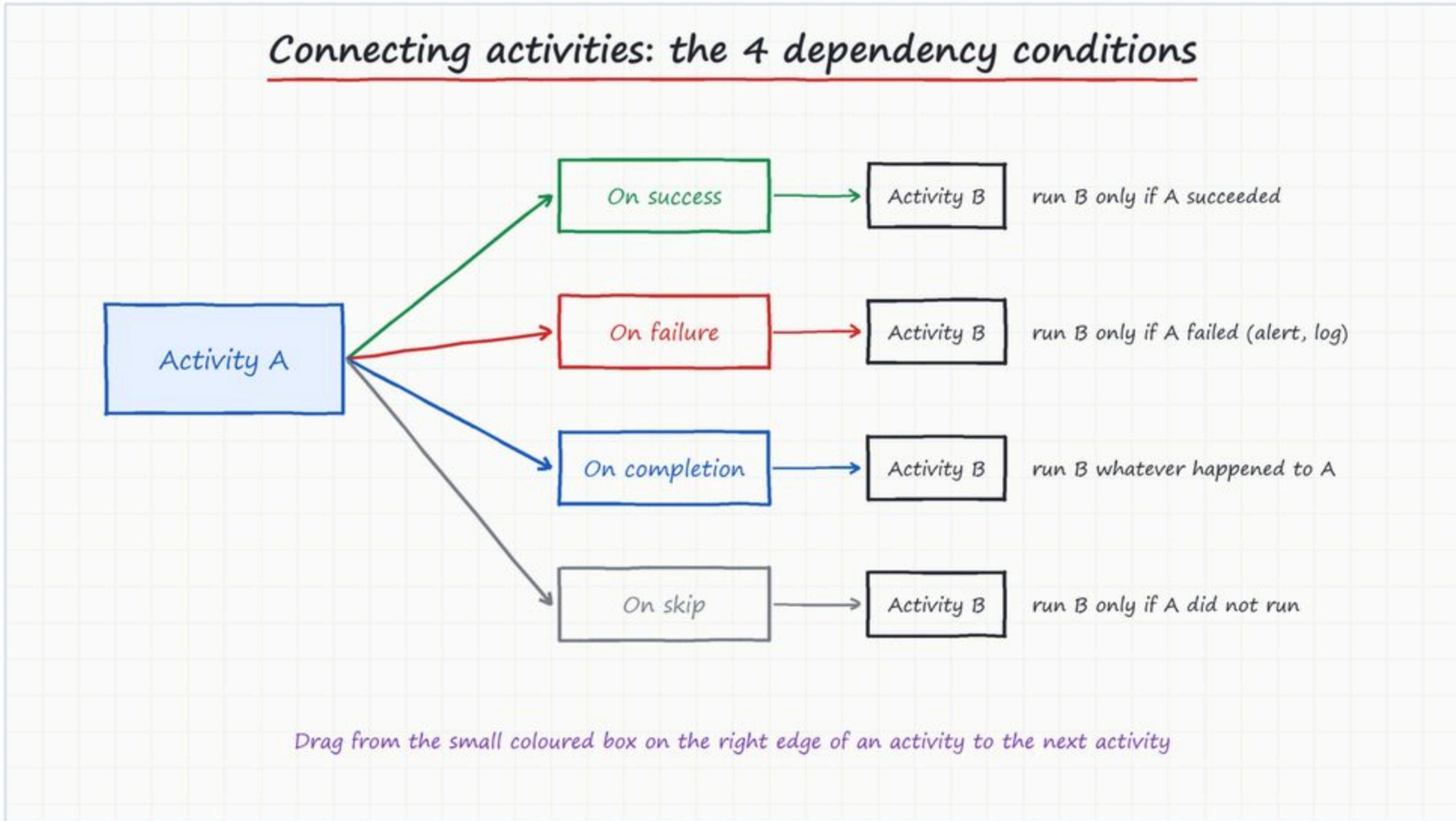
- Columns: customerid, firstname, addresstype
- Rows like 29485 – Catherine – Main Office
- Choose a sink (CSV/Parquet) and Debug

PART 4

Many tables in one pipeline

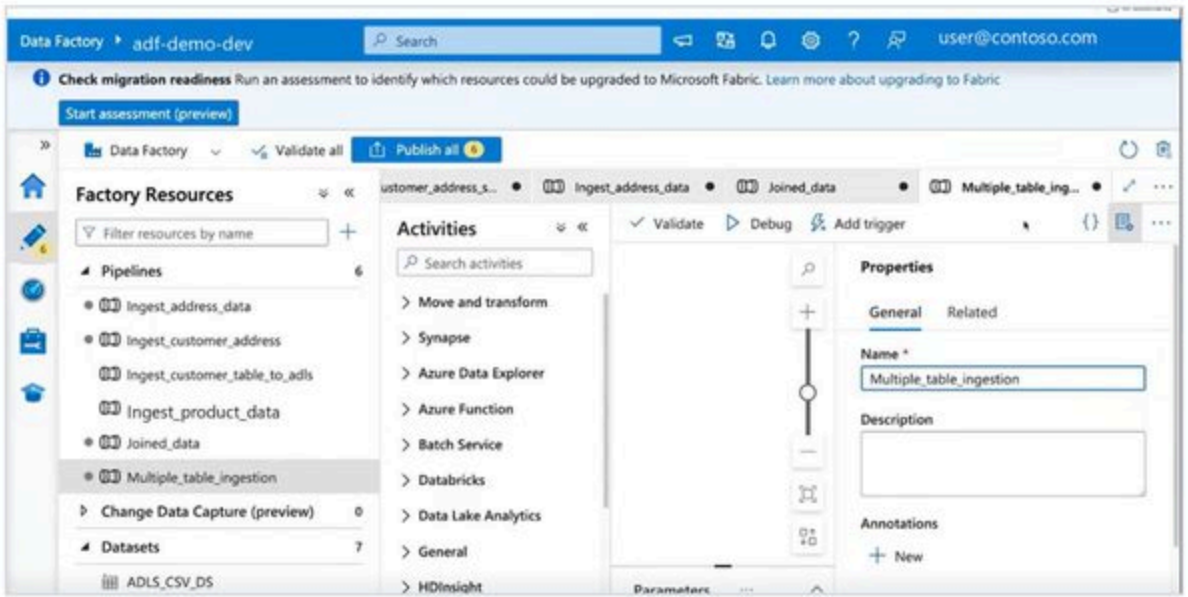
Lab 4: Customer + Address + Product • dependency conditions • sequential vs parallel

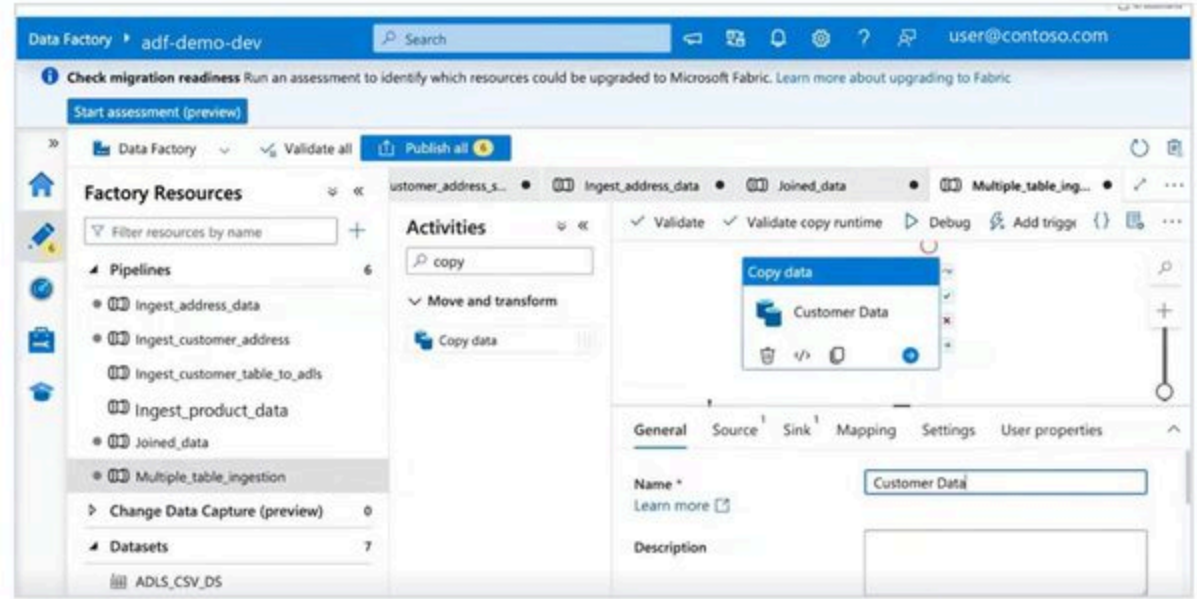
The 4 dependency conditions



PART 4

Lab 4 – Pipeline Multiple_table_ingestion

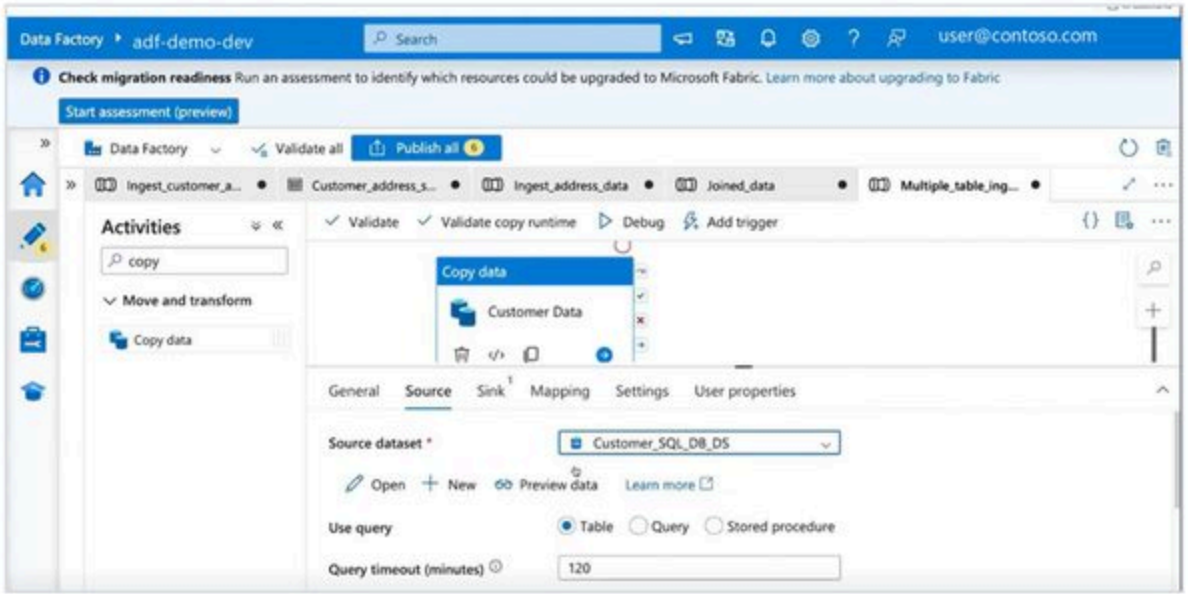


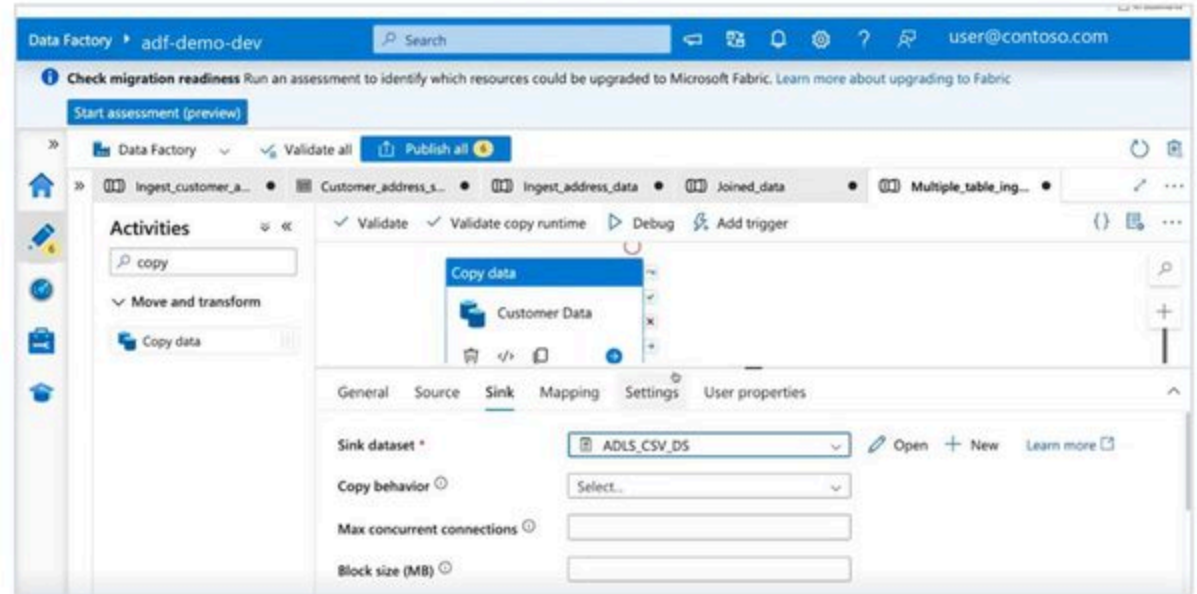


- New pipeline: Multiple_table_ingestion
- First Copy activity: Customer Data

PART 4

Lab 4 – Customer Data: source and sink

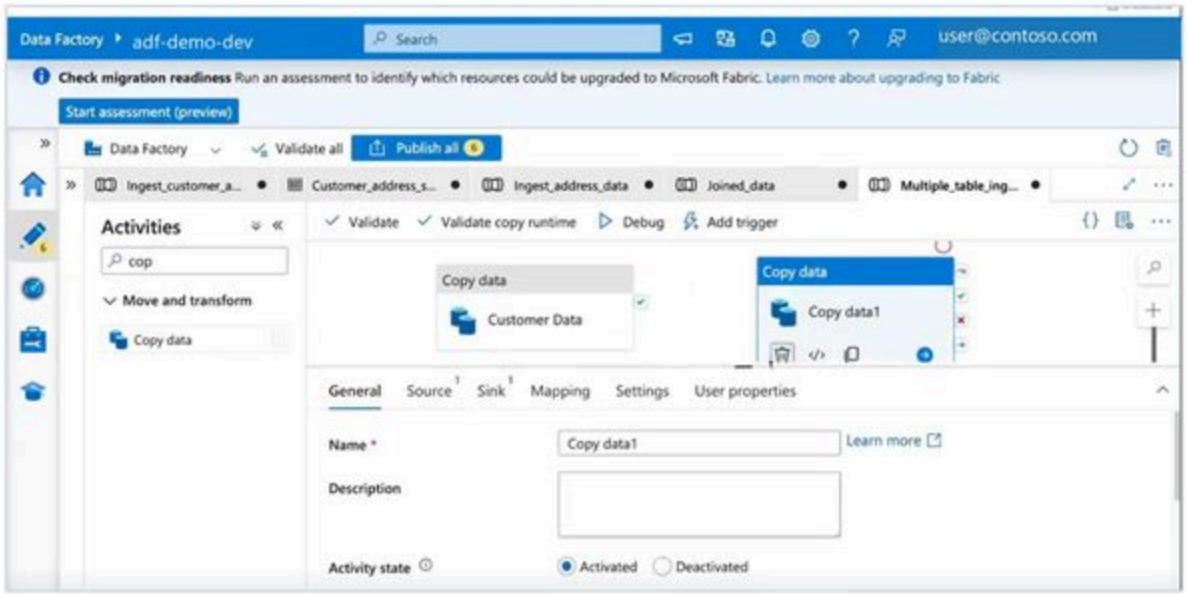


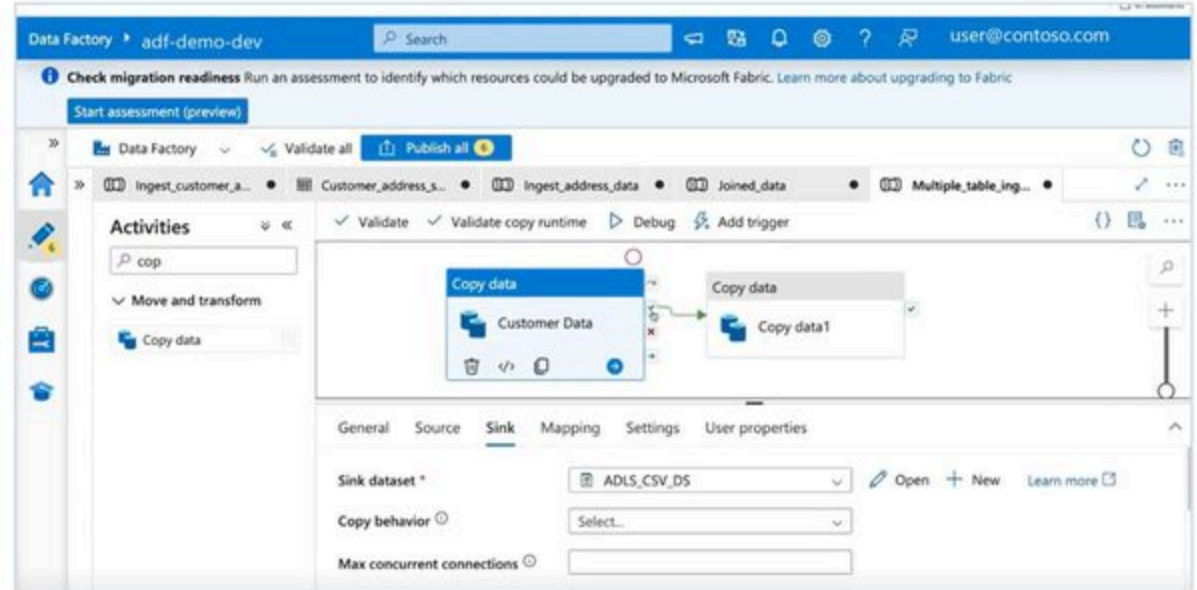


- Source: Customer_SQL_DB_DS (Table)
- Sink: ADLS_CSV_DS
- Reuse last week's datasets

PART 4

Lab 4 – Add a second activity and connect it

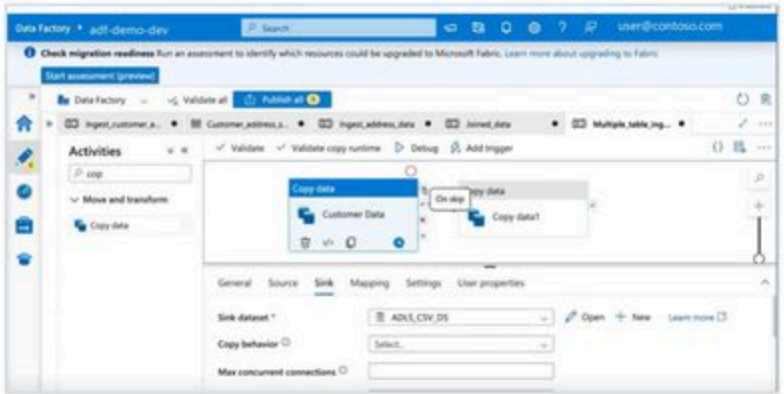


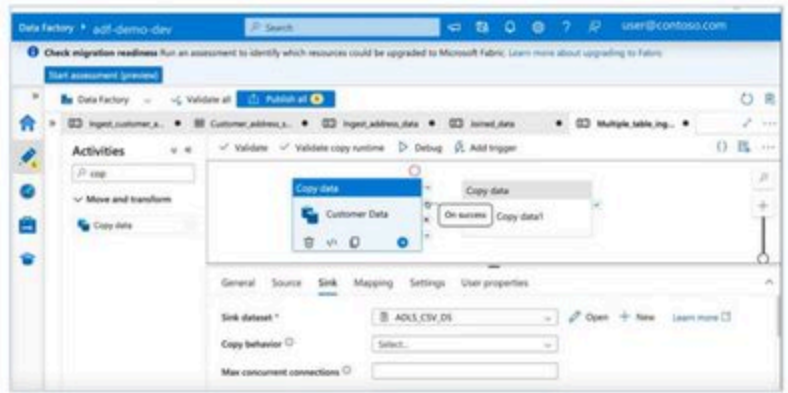


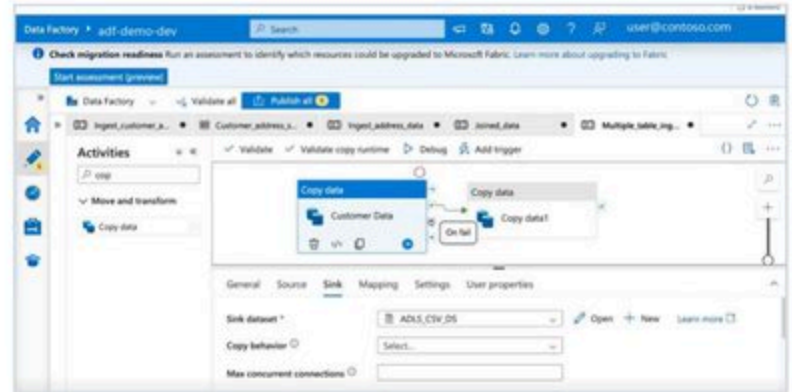
- Copy or drag a second Copy data activity
- Drag the green handle of Customer Data onto it
- Green arrow = On success

PART 4

The four handles in ADF Studio



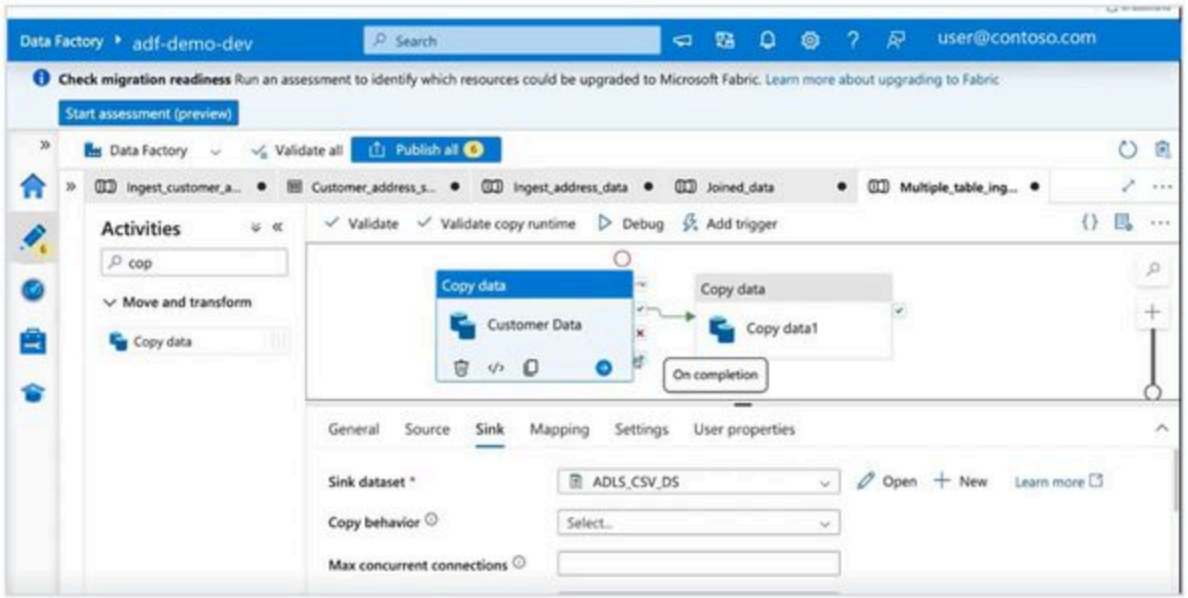


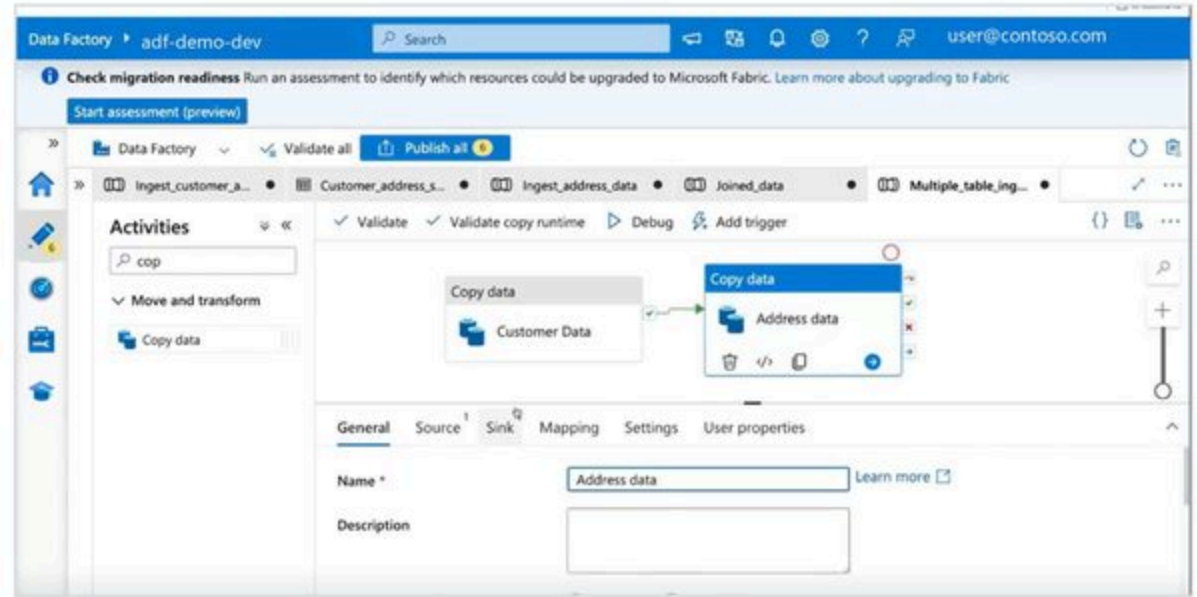


- On skip (grey arrow)
- On success (green)
- On fail (red)

PART 4

On completion and the Address activity

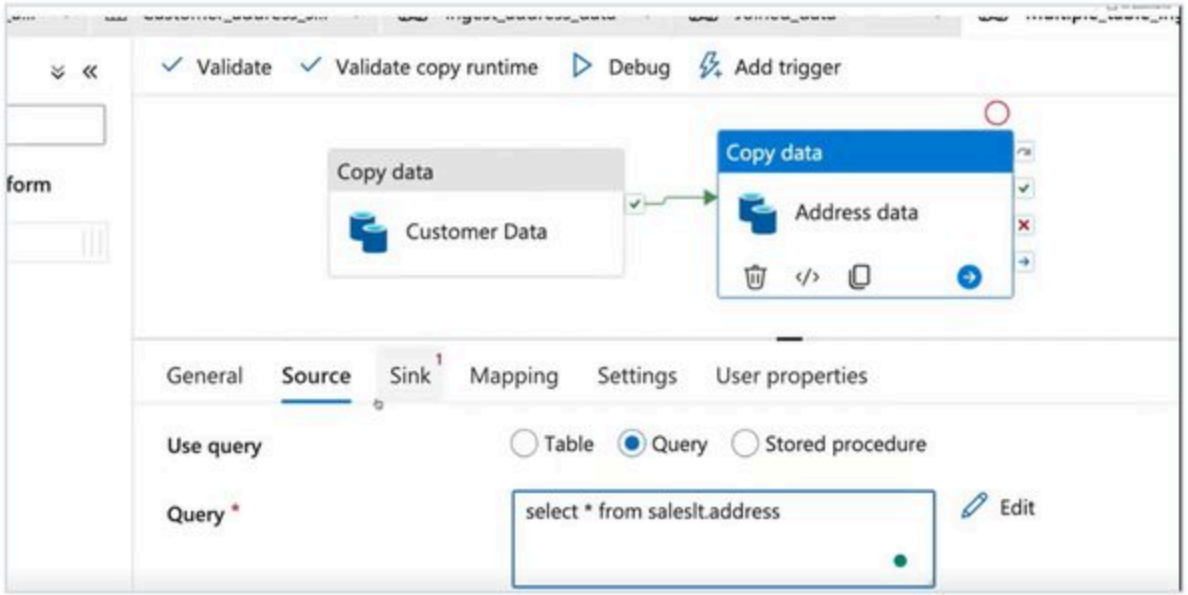


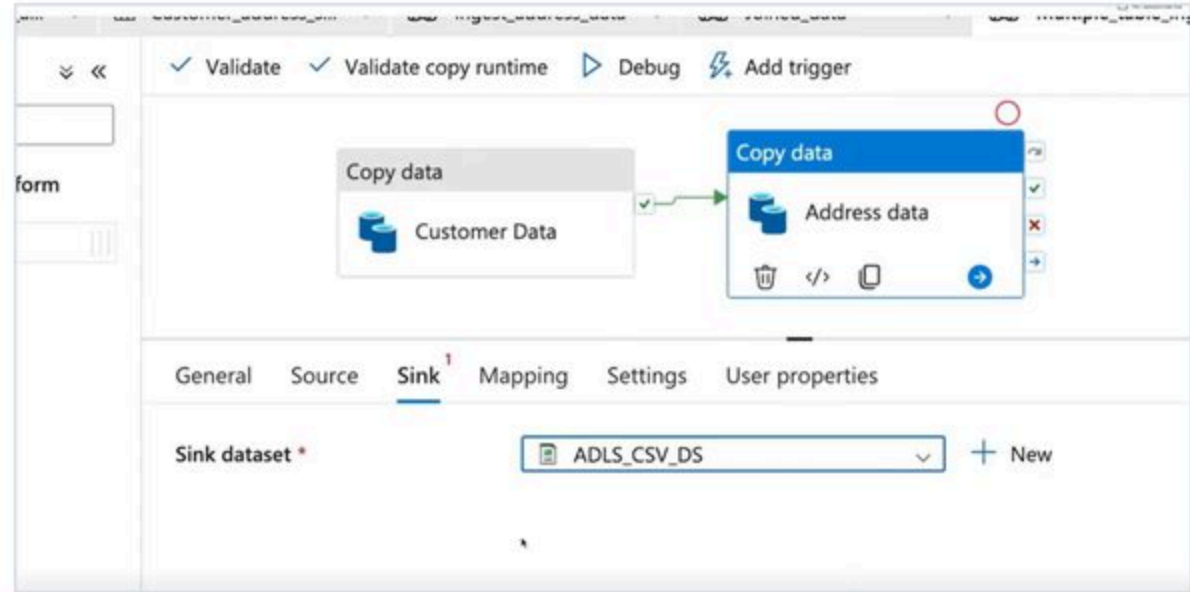


- On completion (blue): runs after success or failure
- Rename the second activity: Address data

PART 4

Lab 4 – Address data: query + CSV sink

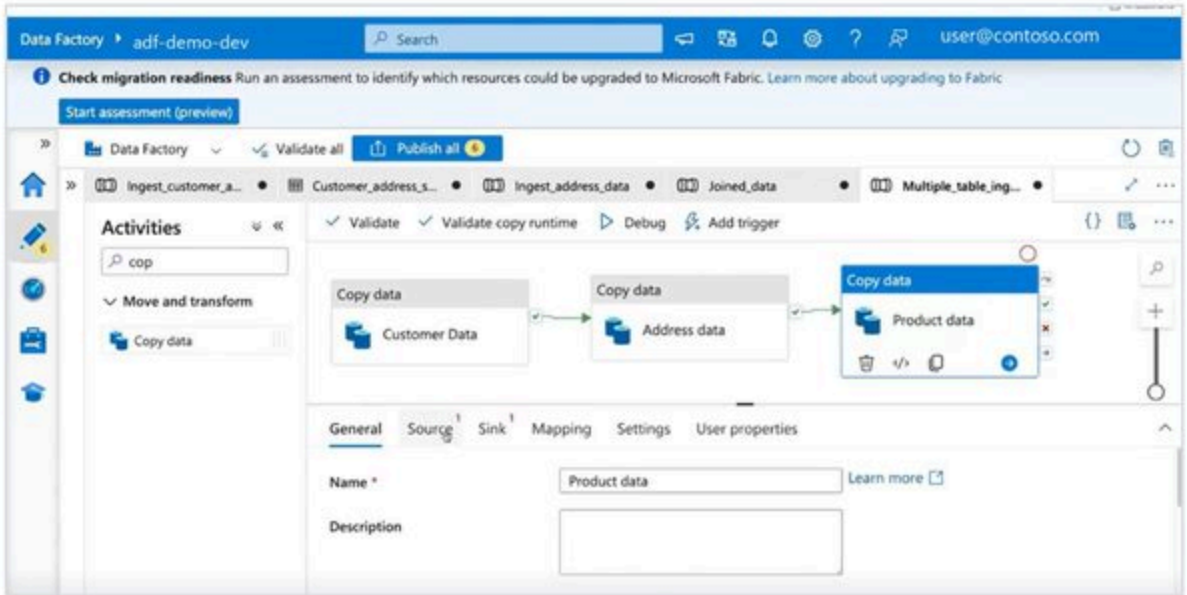


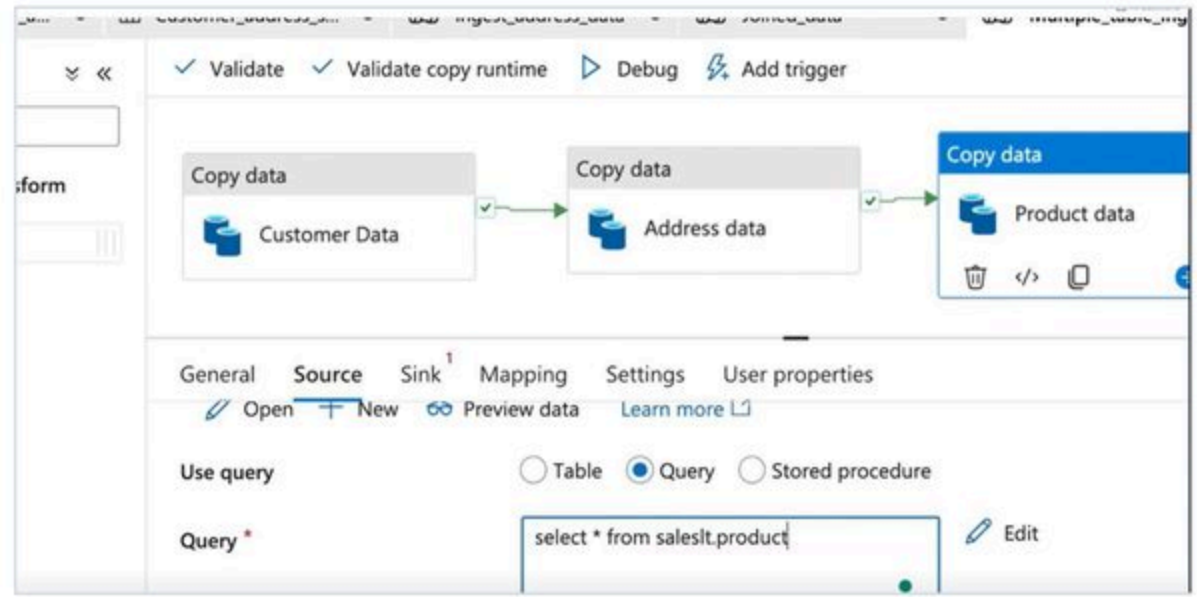


- Use query: select * from saleslt.address
- Sink: ADLS_CSV_DS

PART 4

Lab 4 – Third activity: Product data

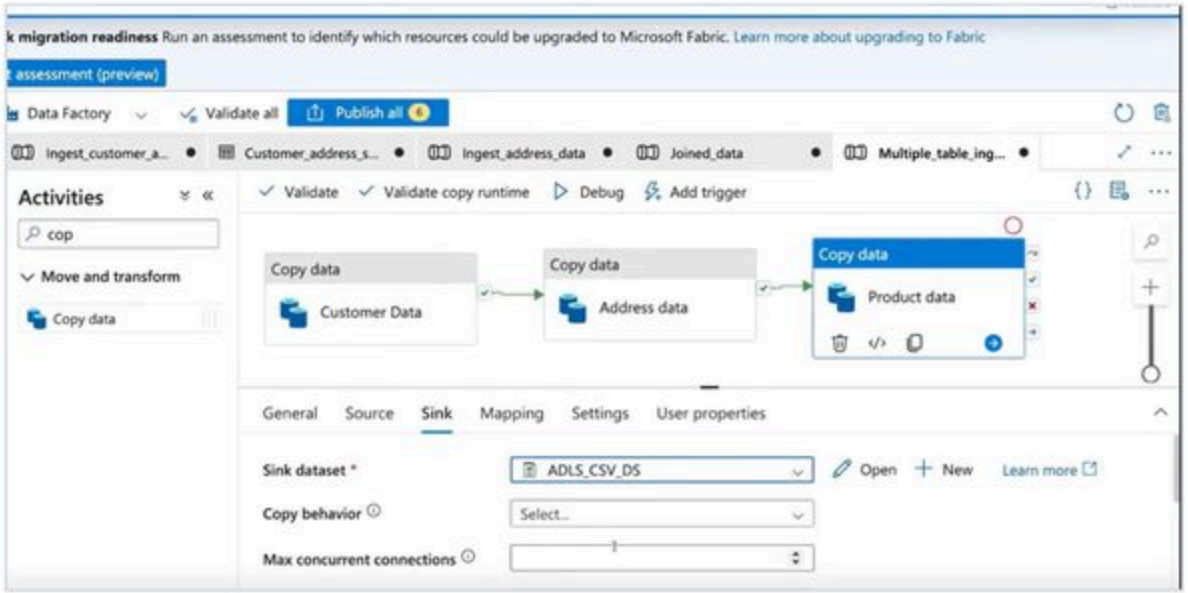


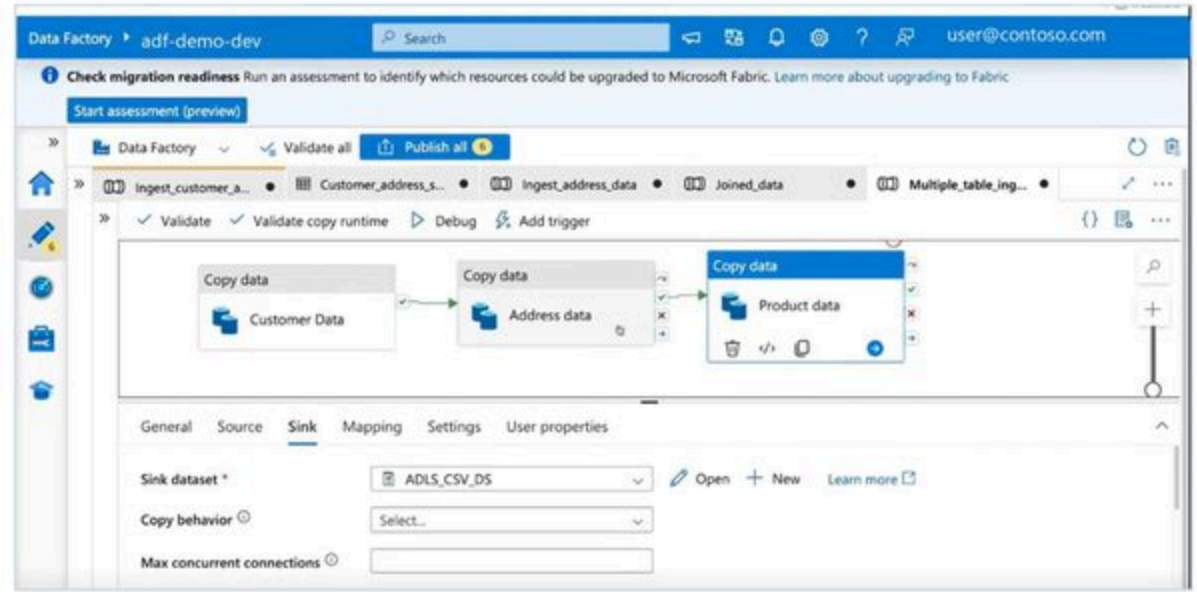


- Third Copy activity: Product data
- Connected On success from Address data
- Use query: select * from saleslt.product

PART 4

Lab 4 – Product data sink and the full chain

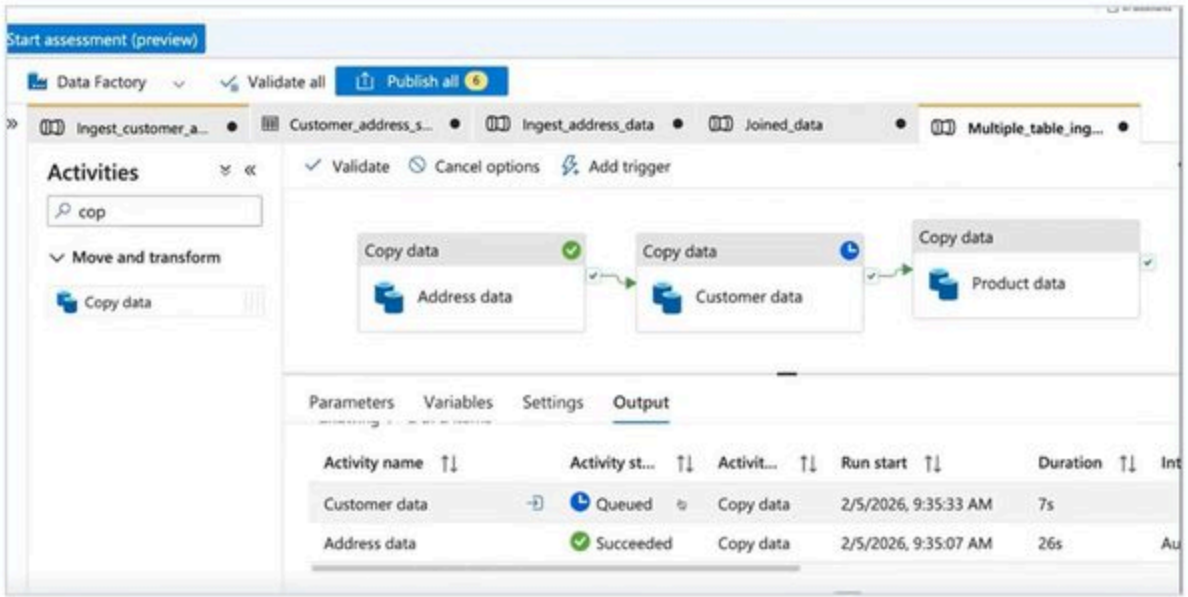


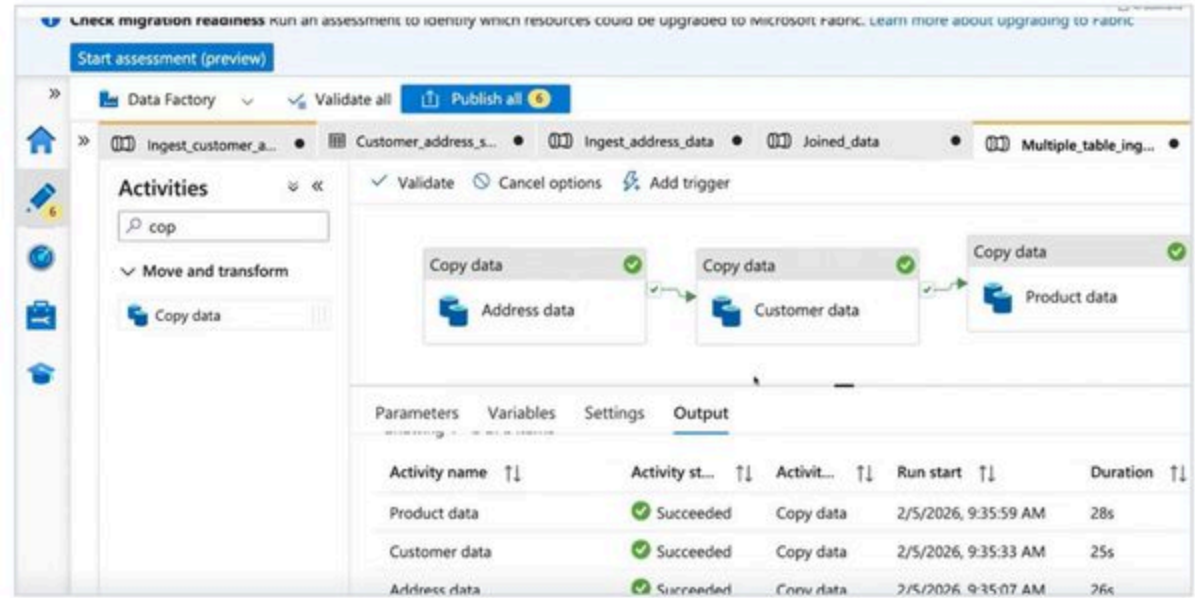


- Sink: ADLS_CSV_DS
- Customer → Address → Product, all On success

PART 4

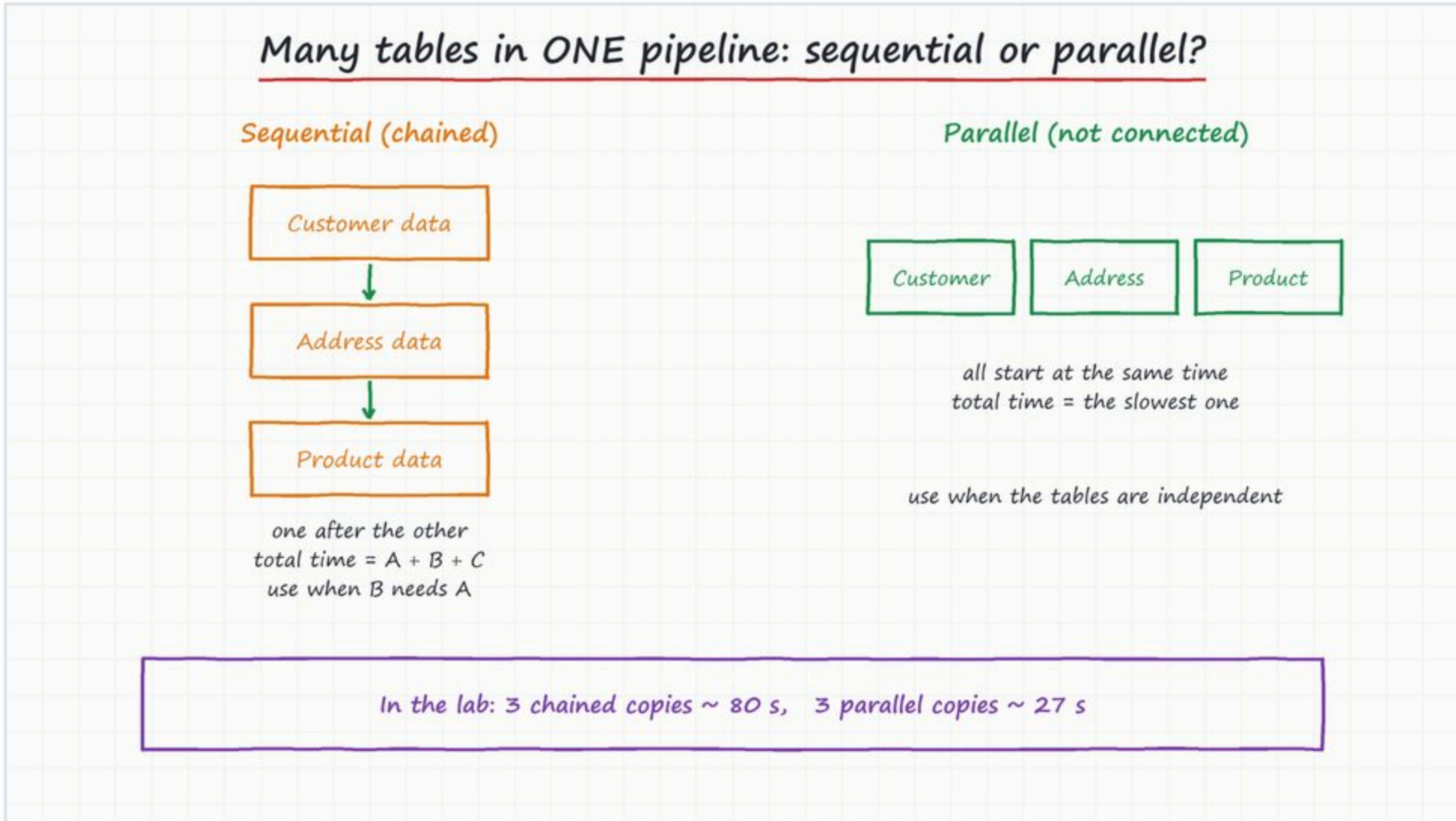
Run the chain – one after the other





- Activities start only when the previous one succeeds
- Address 26 s → Customer 25 s → Product 28 s
- Total ≈ 80 s

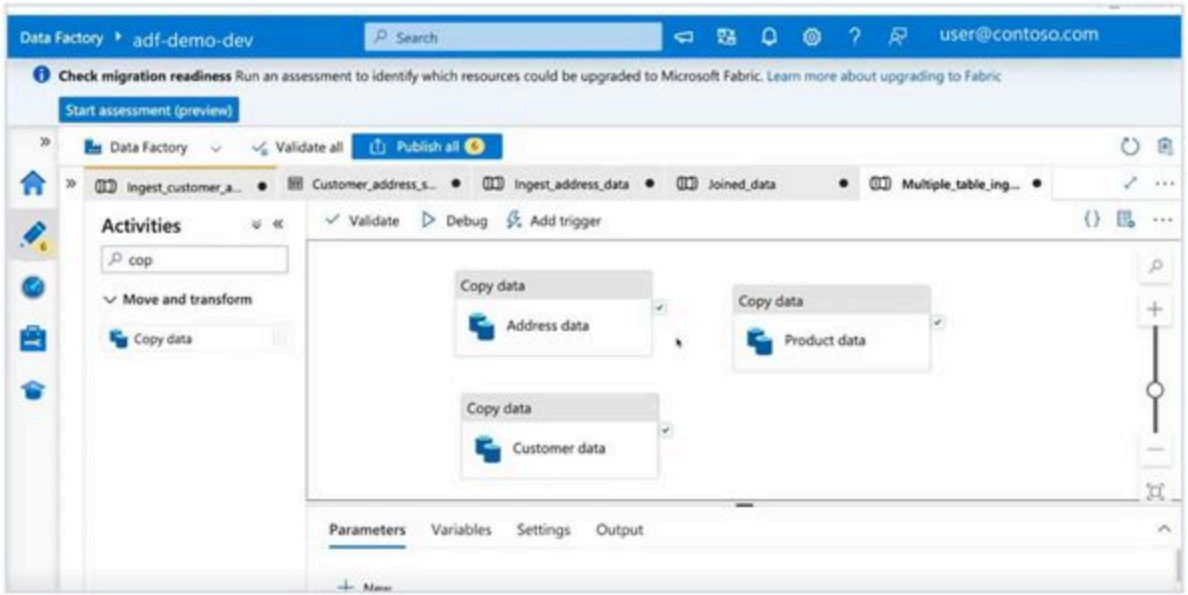
Sequential or parallel?

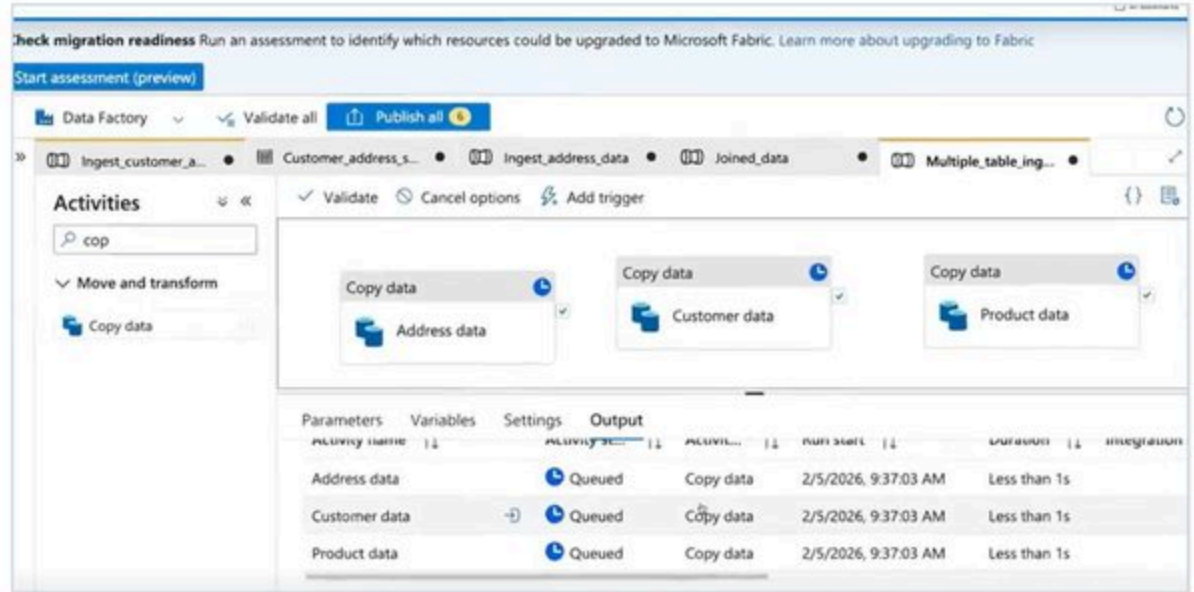


Prepared by Ir. UWAYO Jacques · Follow me on LinkedIn: linkedin.com/in/uwayo

PART 4

Run them in parallel



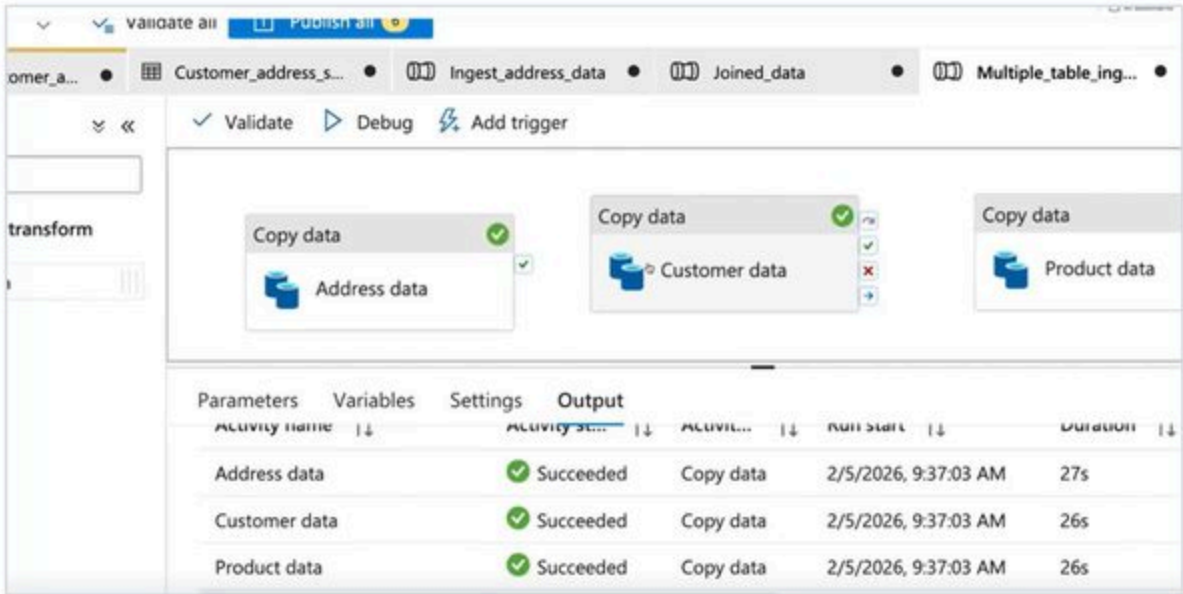


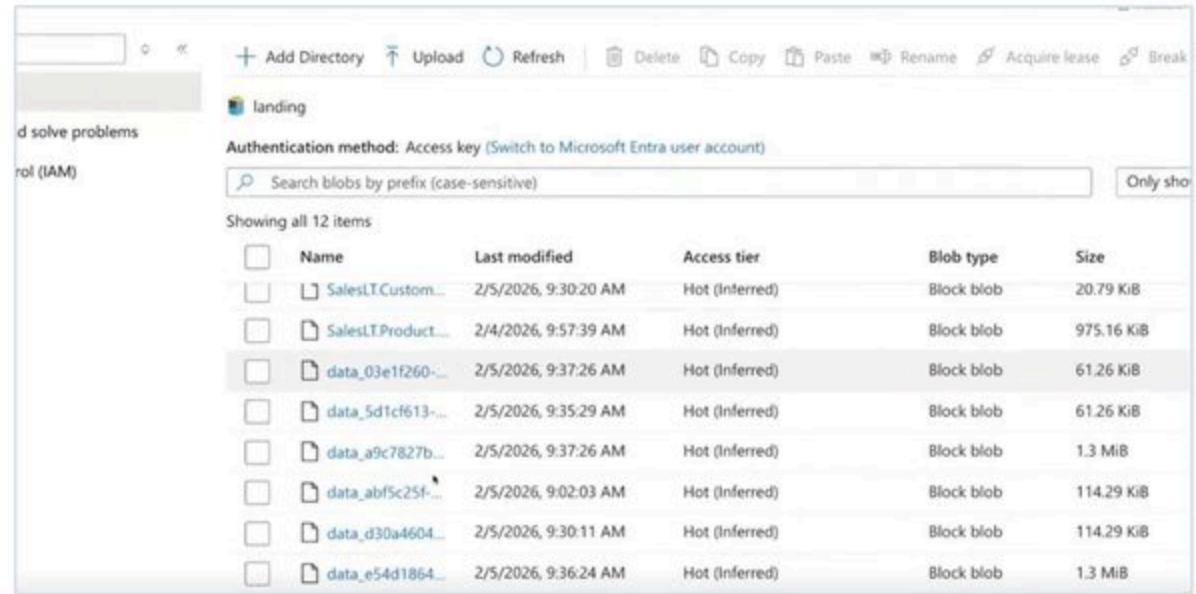
- Delete the arrows → 3 independent activities
- All three start at the same time

- Queued together at the same timestamp

PART 4

Parallel result – and the files





- All three succeeded, each ~26 s, total ≈ 27 s
- Files: data_<guid> names in landing
- Problem: one flat folder – which file is which?

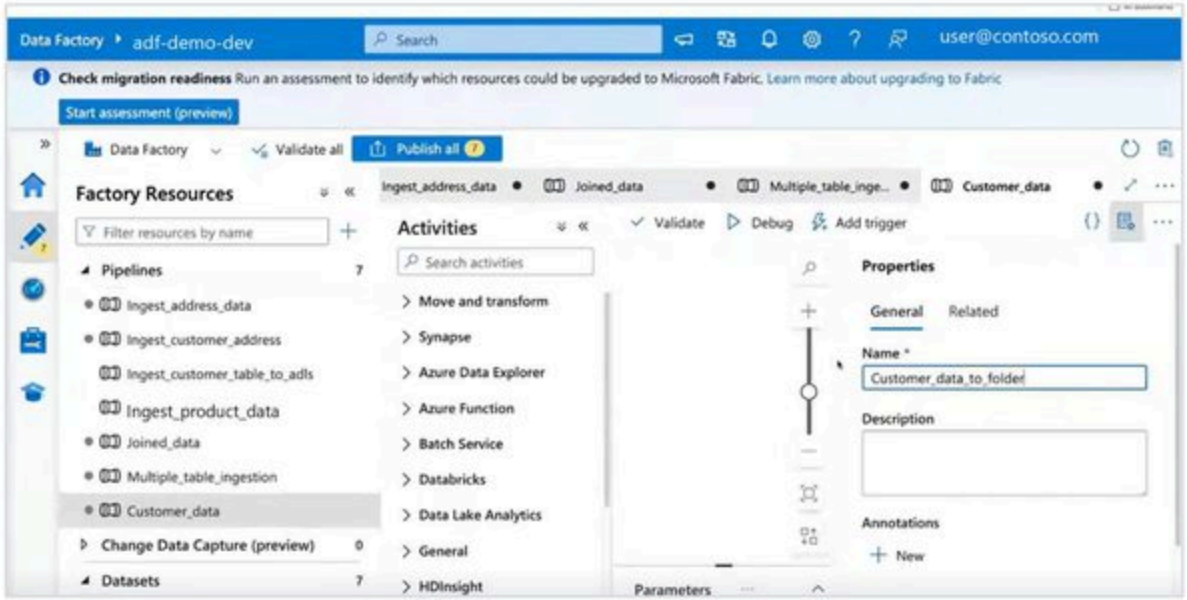
PART 5

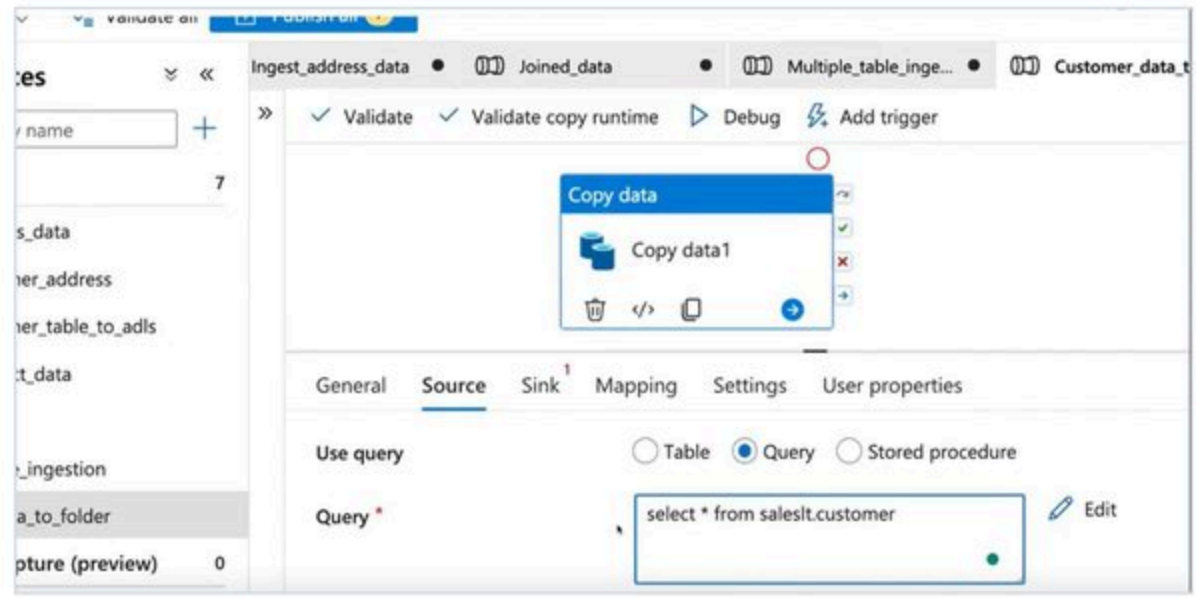
Dynamic folder creation

Lab 5: write each table into its own folder in the lake

PART 5

Lab 5 – Pipeline Customer_data_to_folder

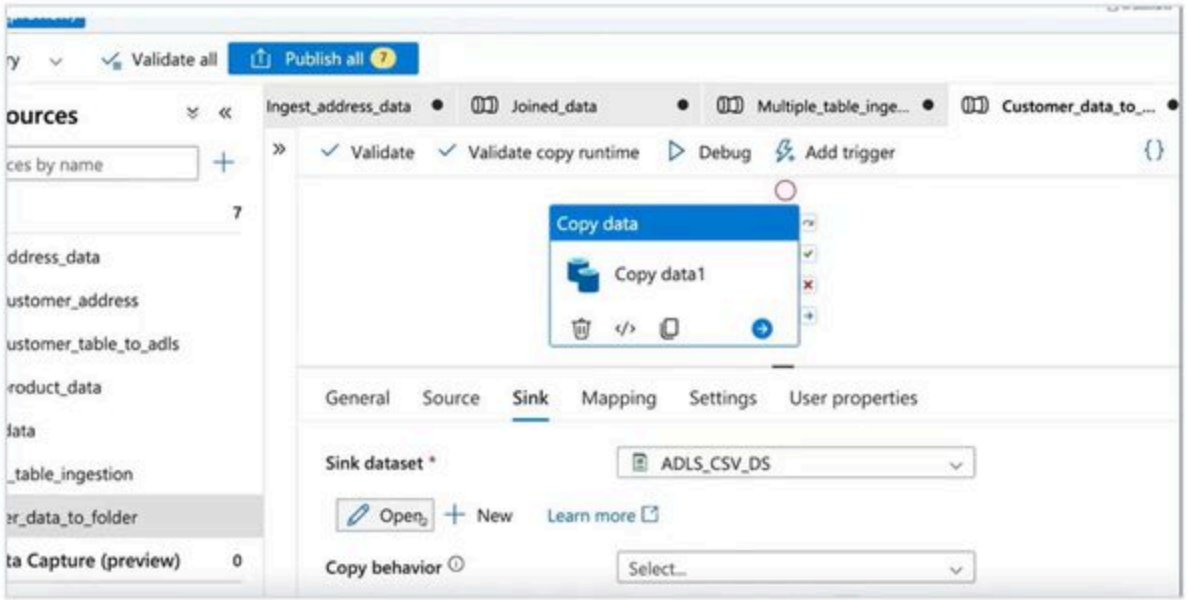


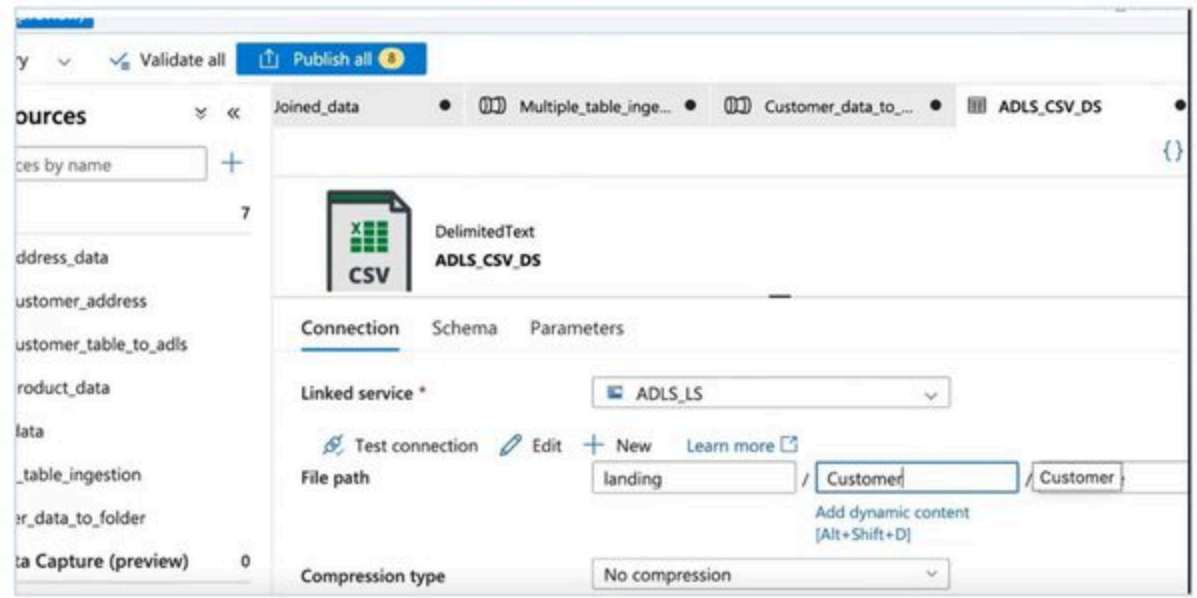


- New pipeline: Customer_data_to_folder
- Copy data1 source query: select * from saleslt.customer

PART 5

Set a directory on the sink dataset



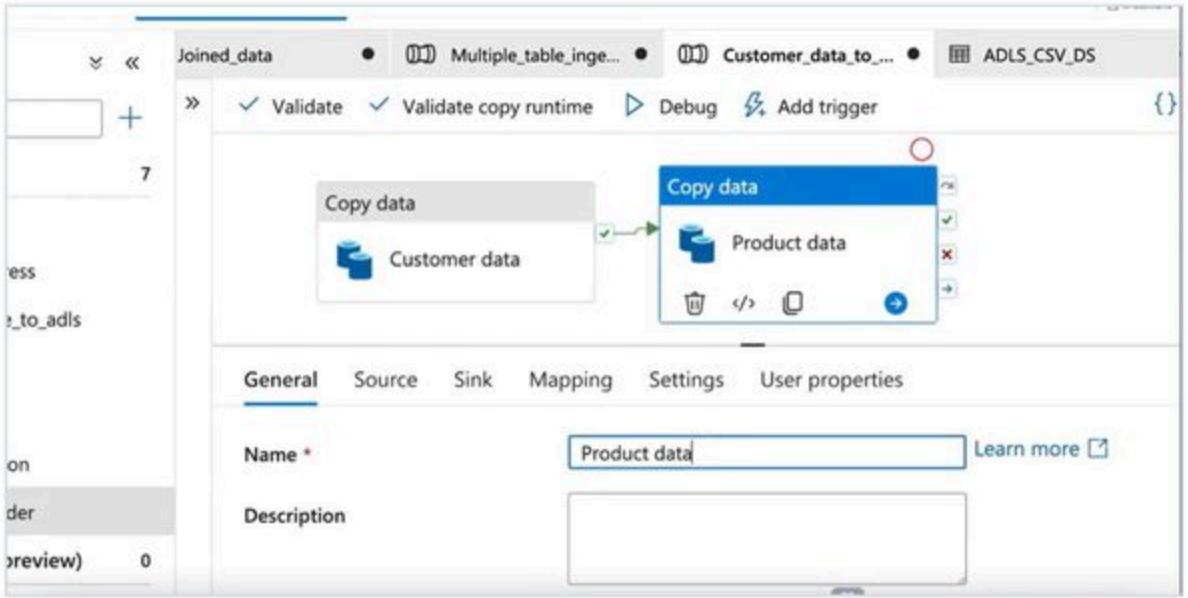


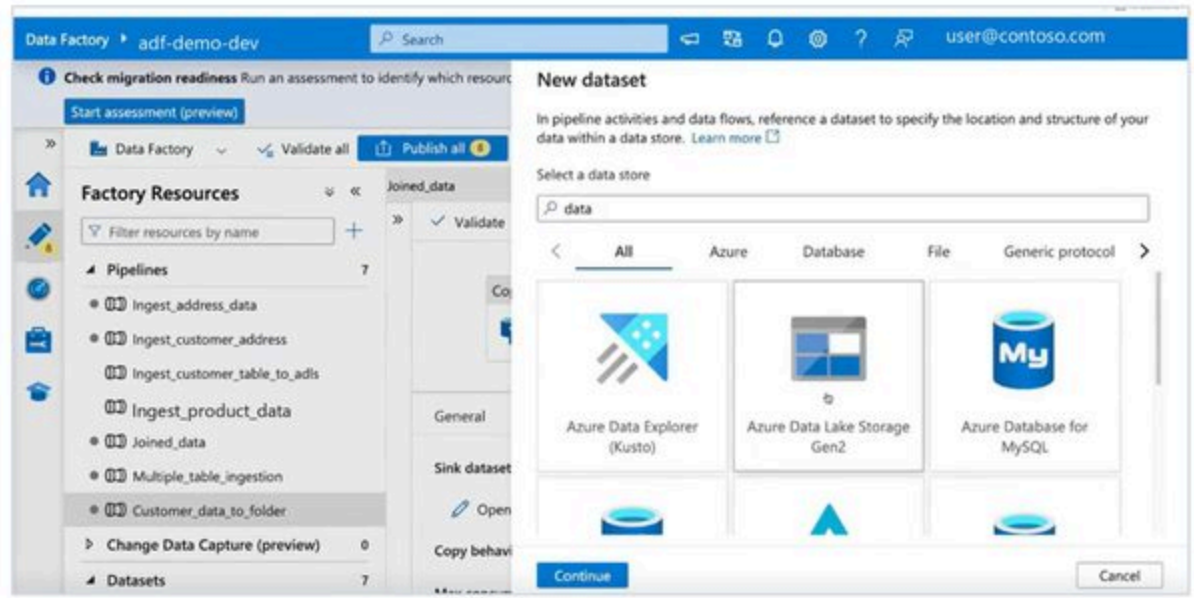
- Sink: ADLS_CSV_DS → Open
- File path: landing / Customer / (file name)
- ADF creates the folder automatically

⚠ Editing a shared dataset changes EVERY pipeline that uses it.

PART 5

Second activity: Product data

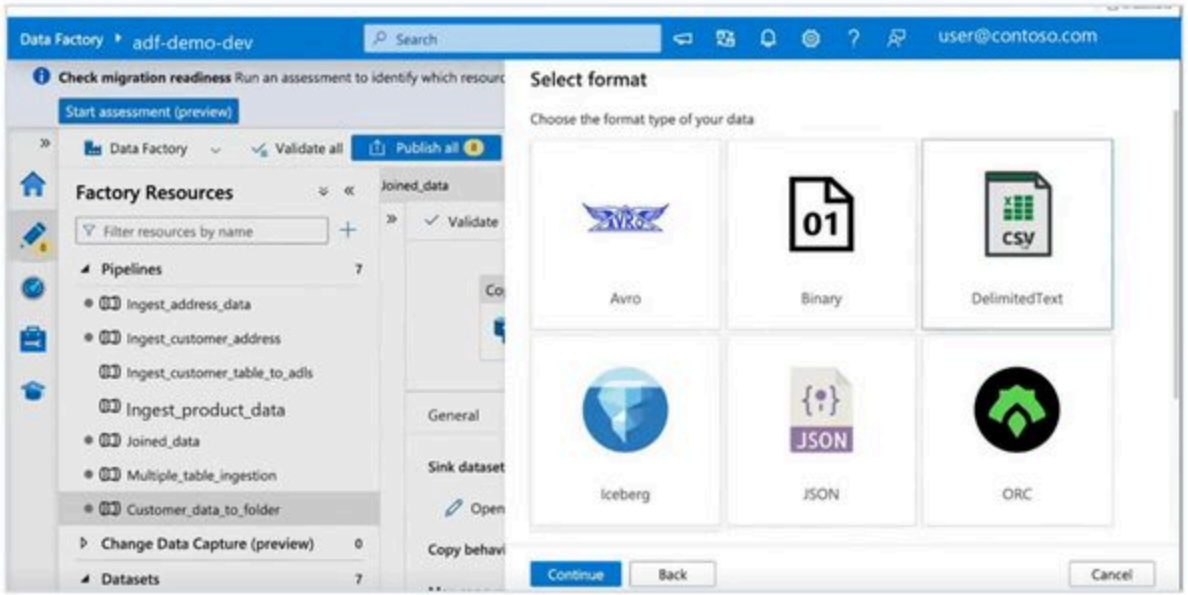


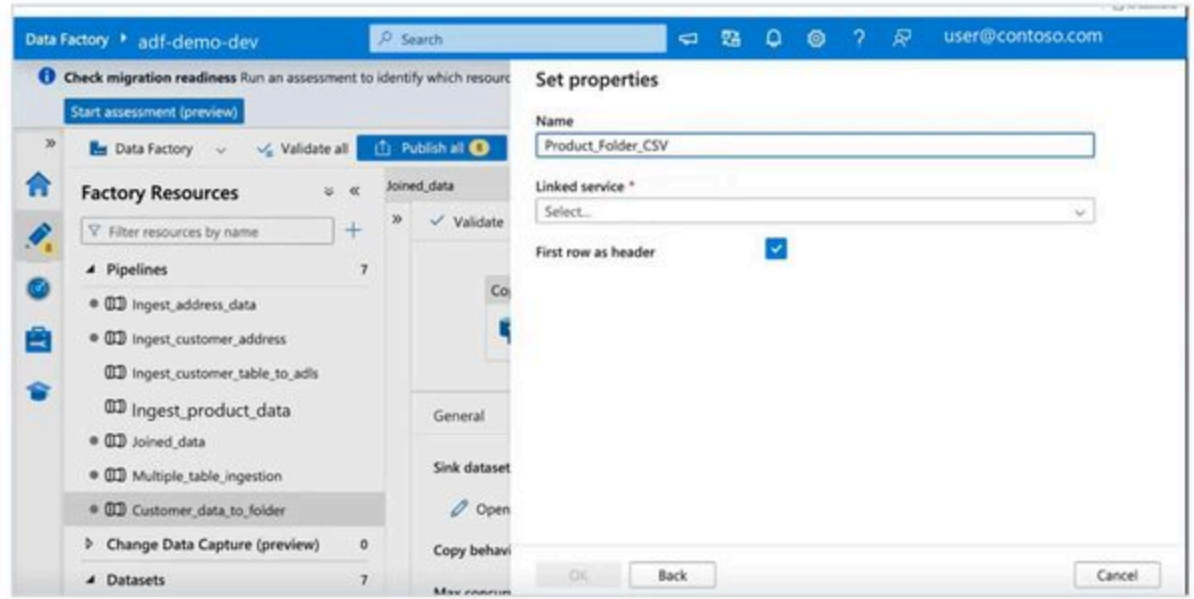


- Product data activity, connected On success
- Sink → + New → ADLS Gen2

PART 5

New dataset: Product_Folder_CSV_DS

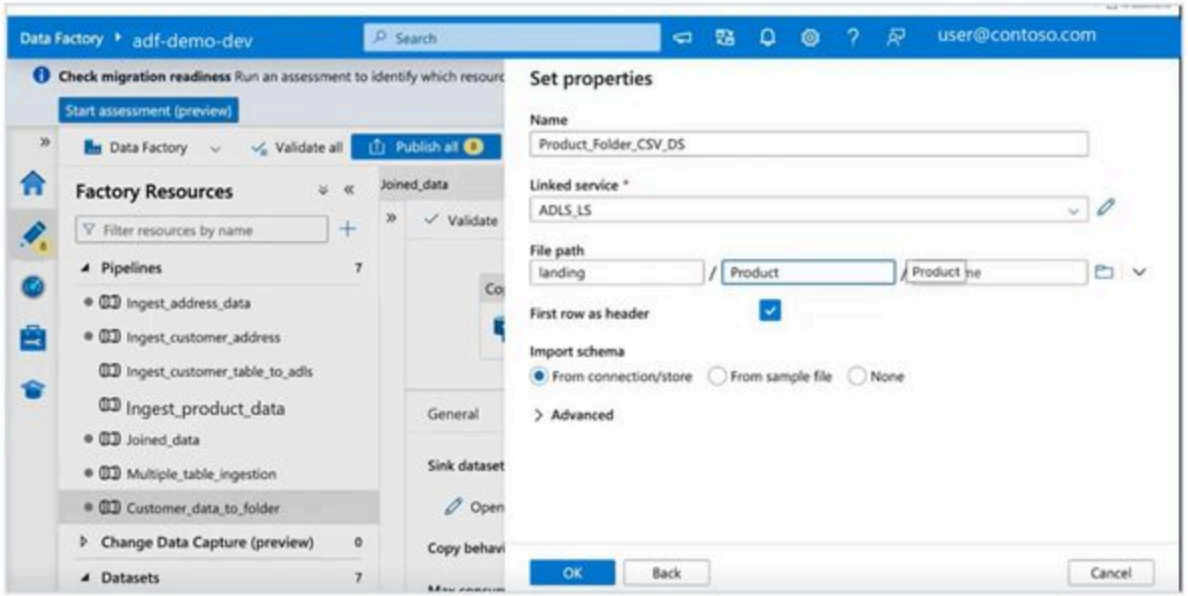


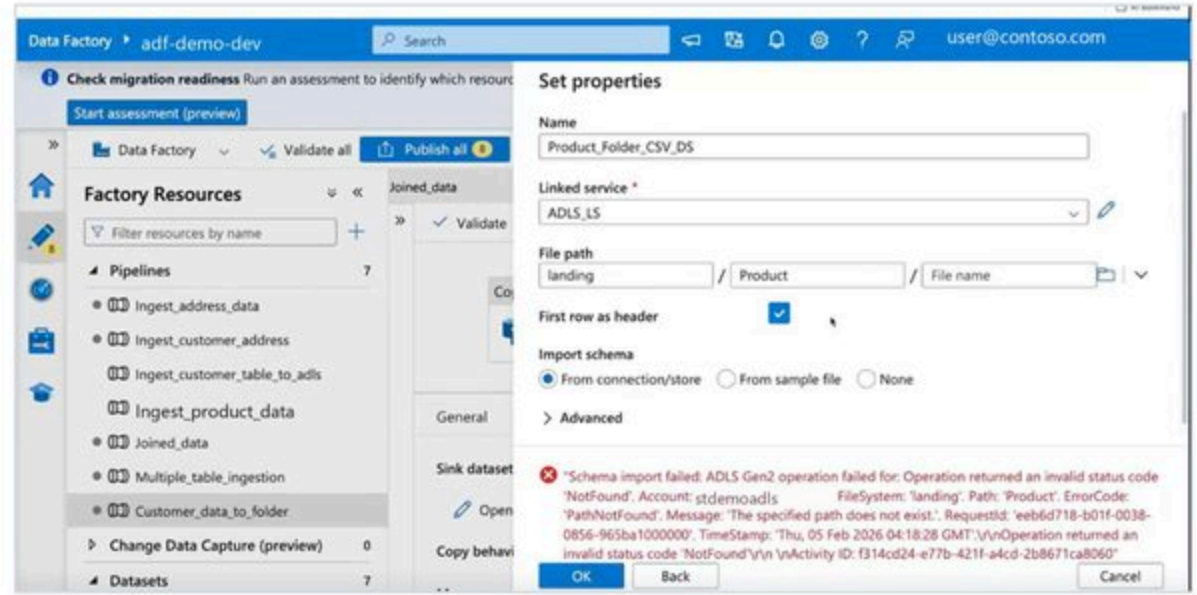


- Format: DelimitedText
- Name: Product_Folder_CSV_DS
- First row as header ✓

PART 5

File path landing / Product – schema import error





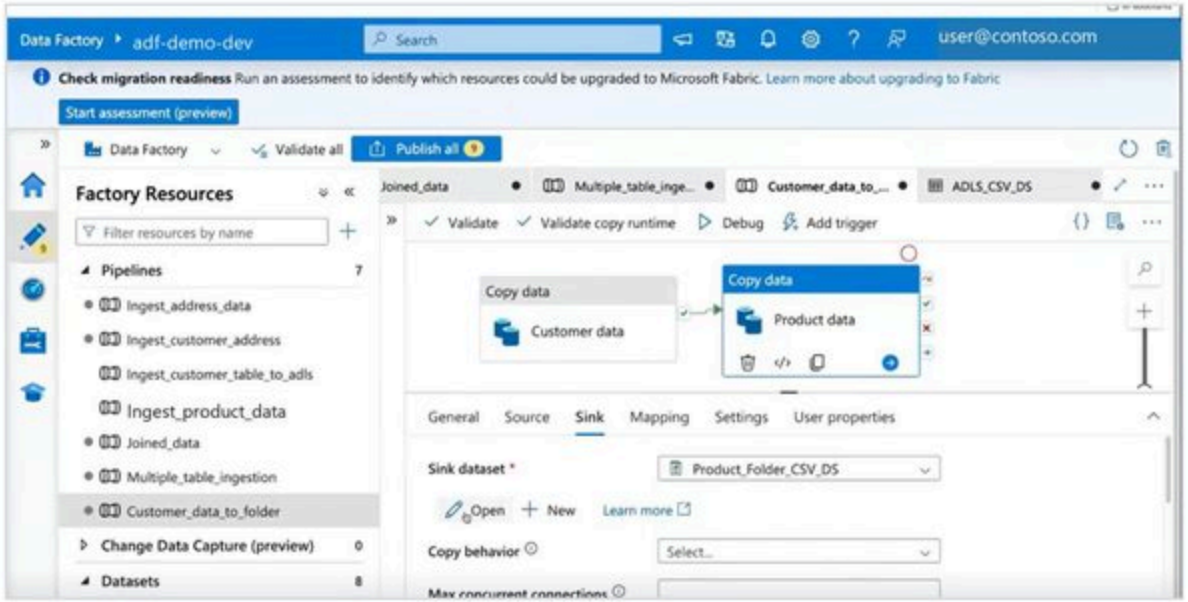
- File path: landing / Product
- Import schema: From connection/store → error

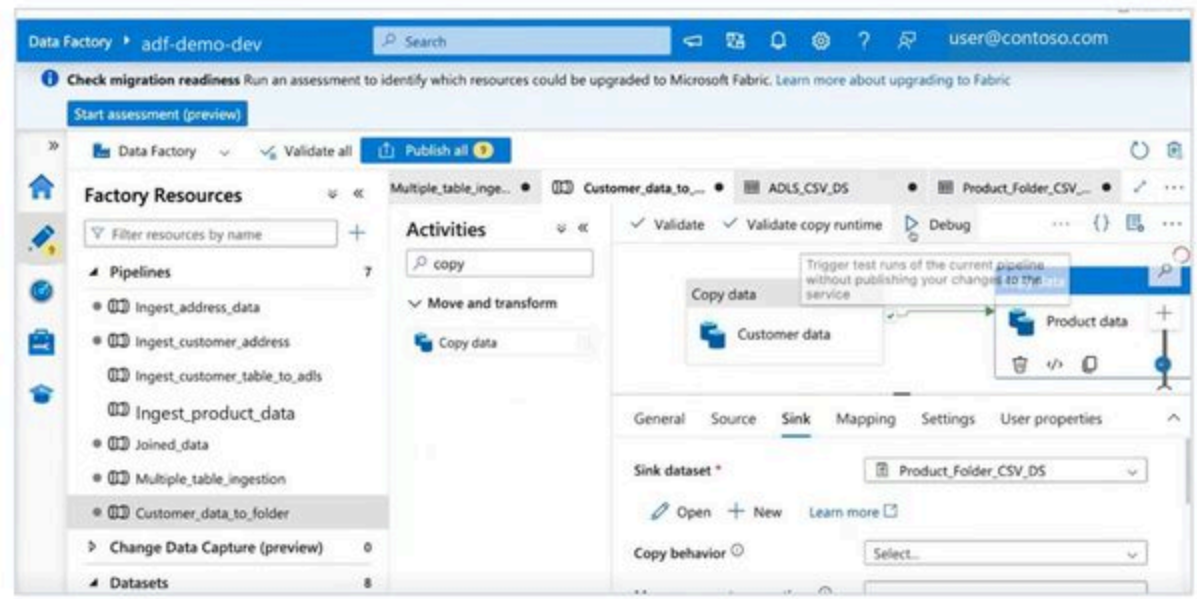
- "PathNotFound – the specified path does not exist"

⚠ New sink folder? Set Import schema to None.

PART 5

Debug the folder pipeline

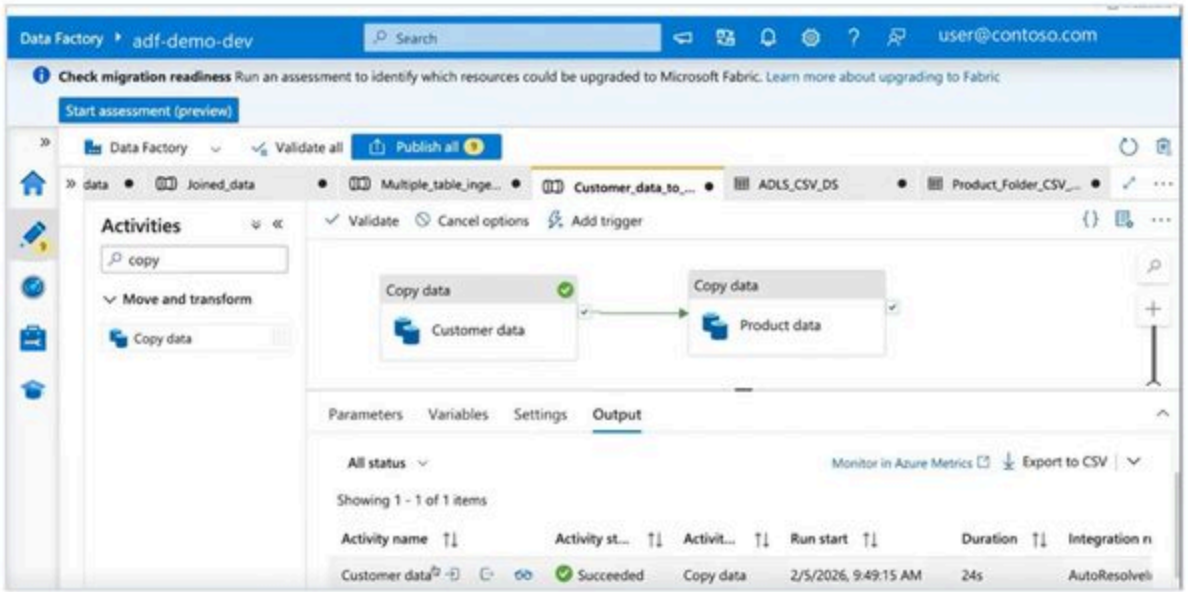


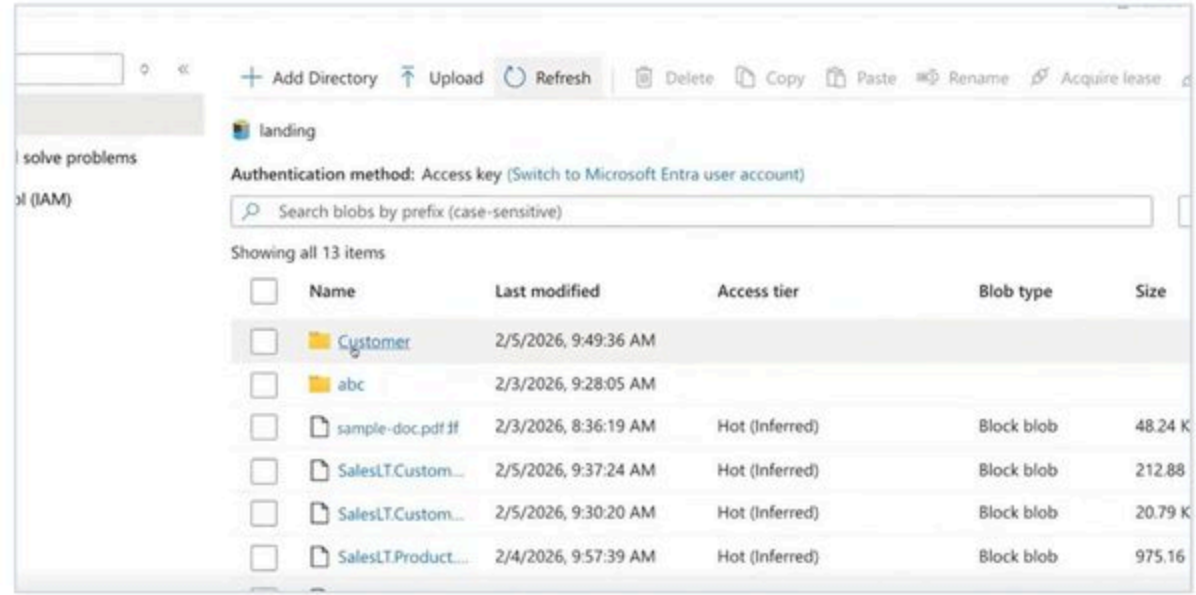


- Sink: Product_Folder_CSV_DS
- Debug → Customer data then Product data

PART 5

Validate: one folder per table

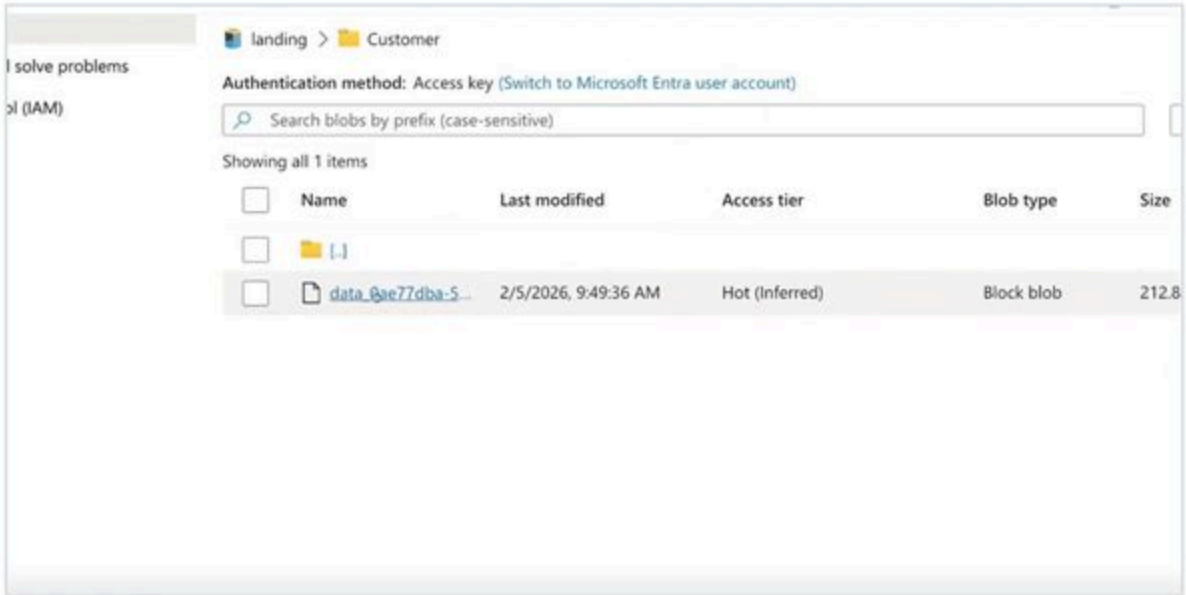


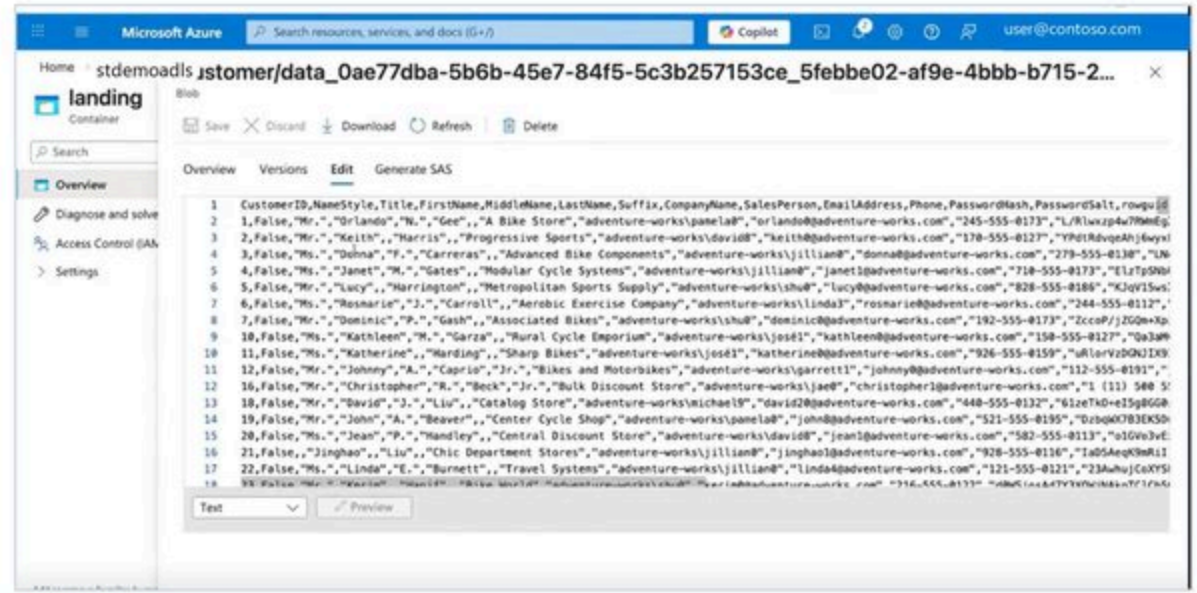


- Pipeline succeeded
- landing now has a Customer folder
- Folders sort above files

PART 5

Inside the Customer folder



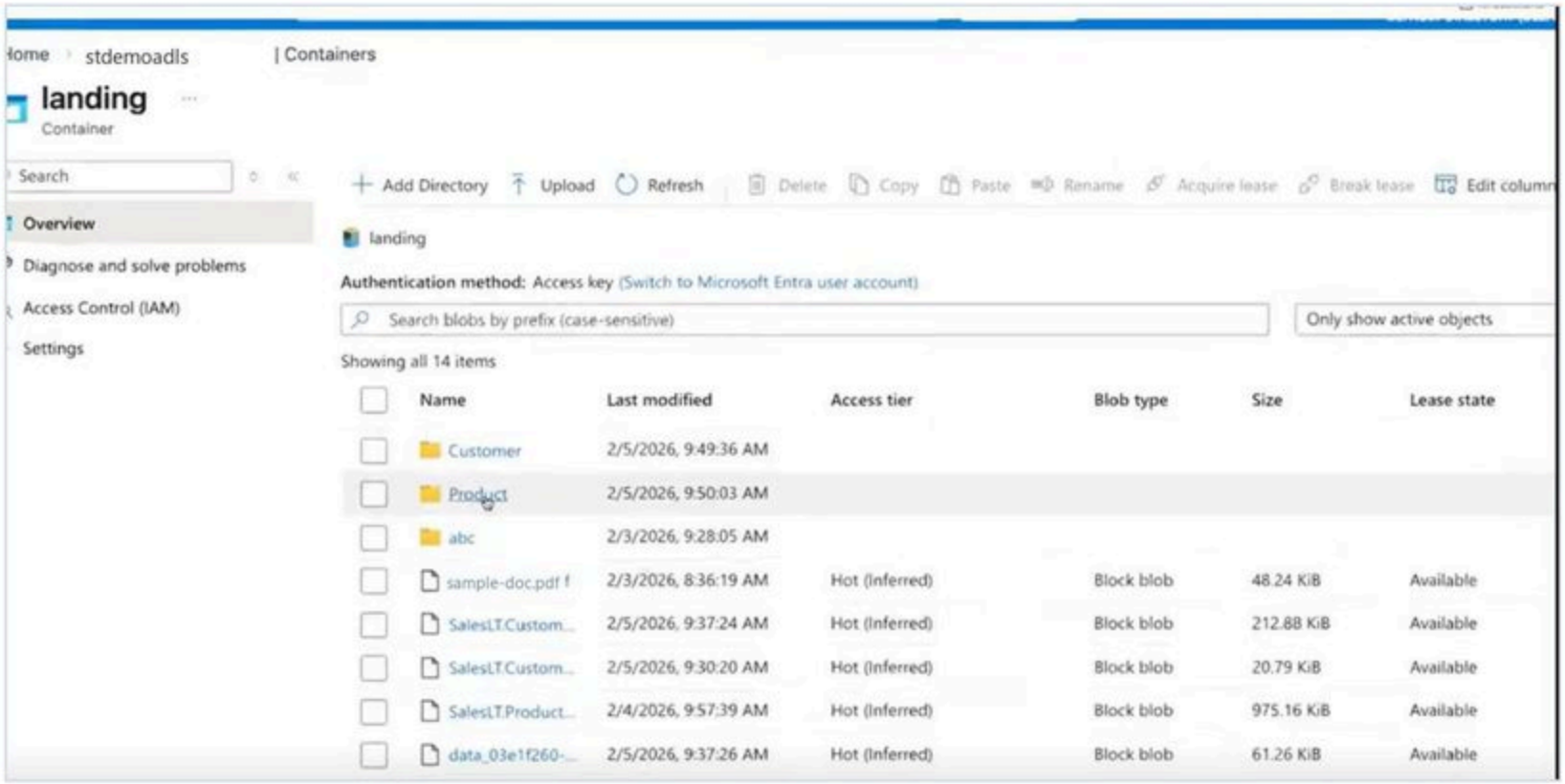


- landing/Customer/data_<guid>
- CSV with header row

- Customer columns: CustomerID, NameStyle, Title, FirstName
- ...

PART 5

Product folder – but two datasets for two folders



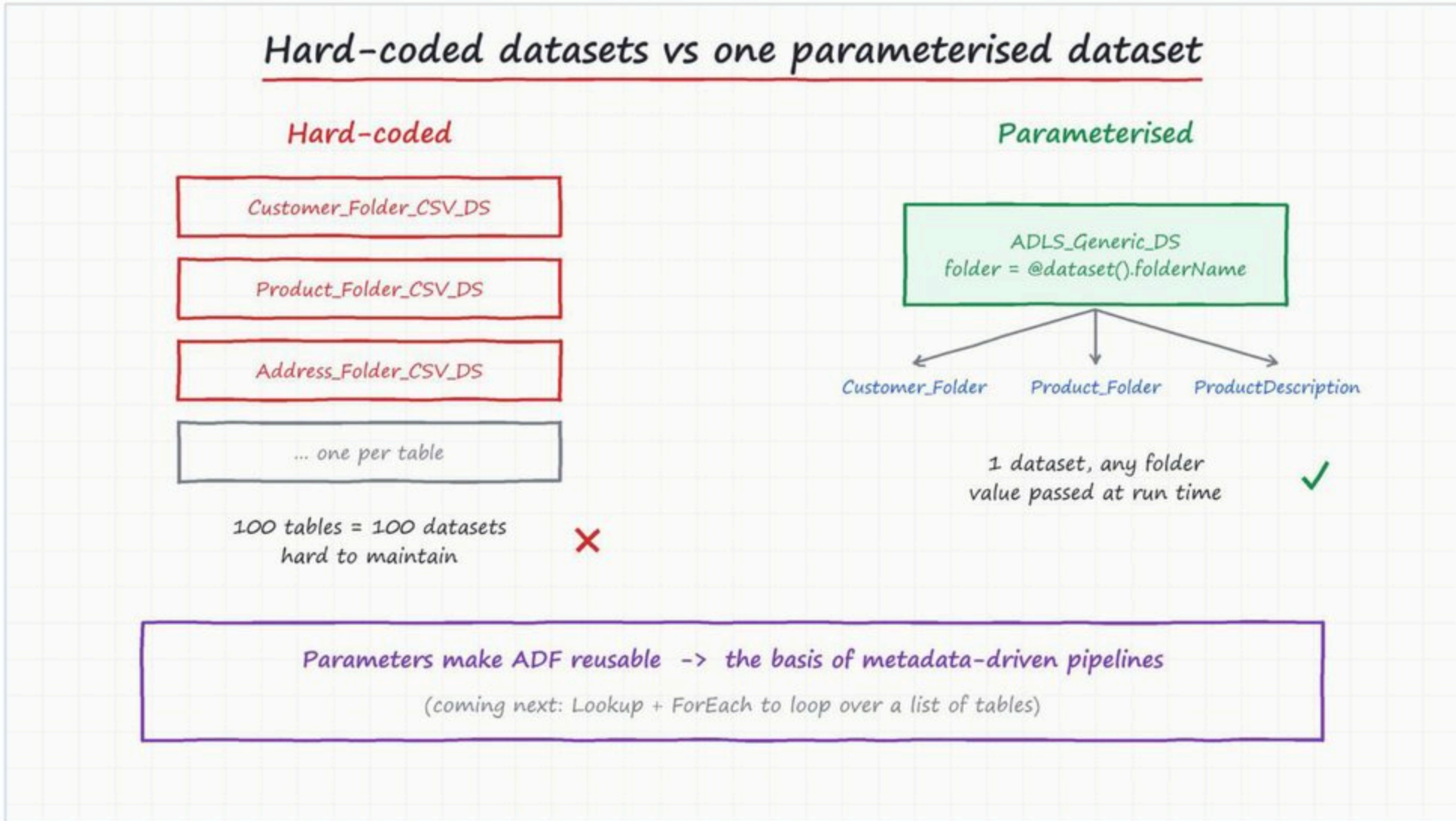
- landing: Customer and Product folders
- Works – but one dataset per folder
- 100 tables = 100 datasets?

PART 6

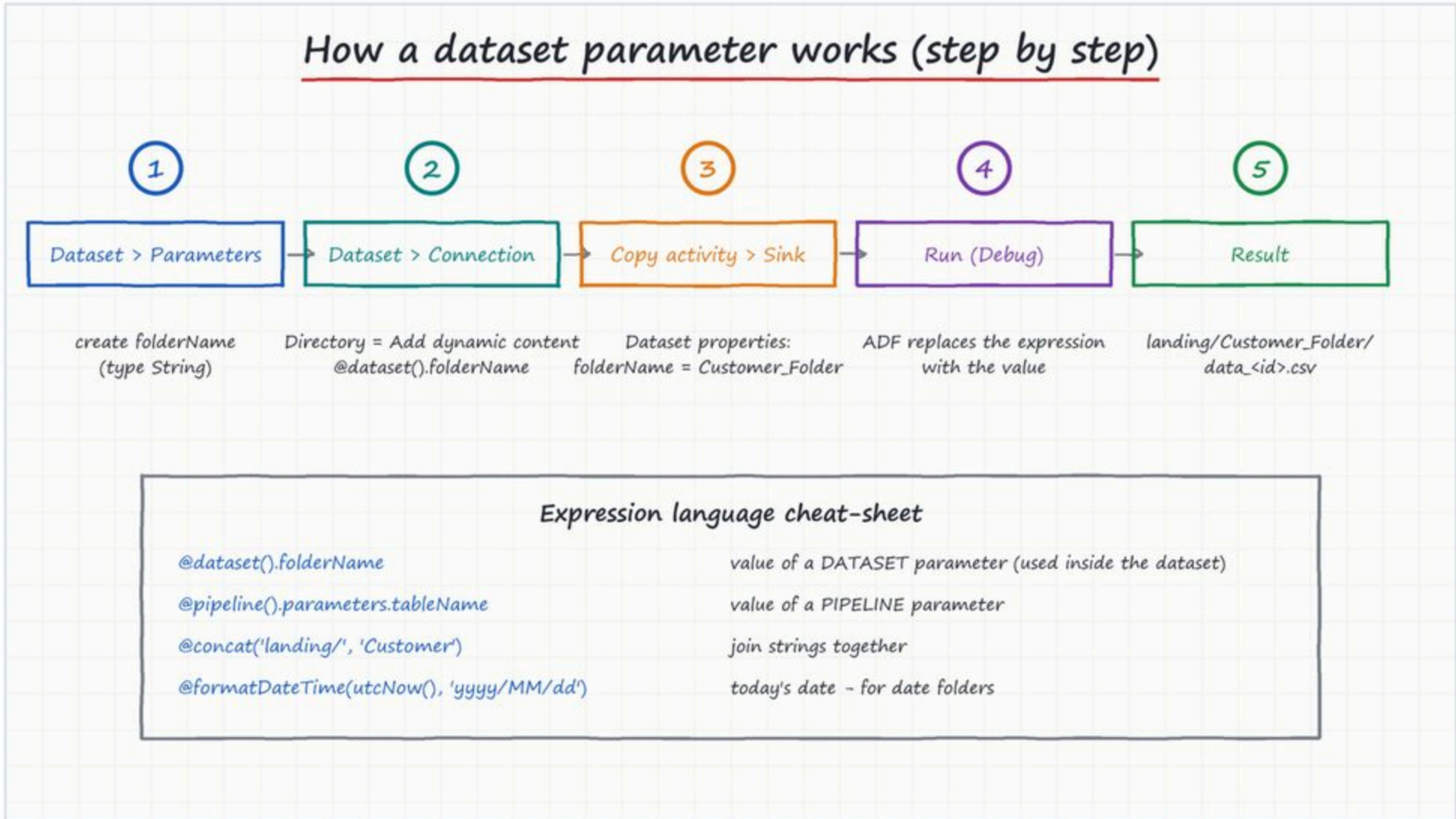
Dataset parameters

Lab 6: one generic dataset, folder name passed at run time

Hard-coded vs parameterised datasets

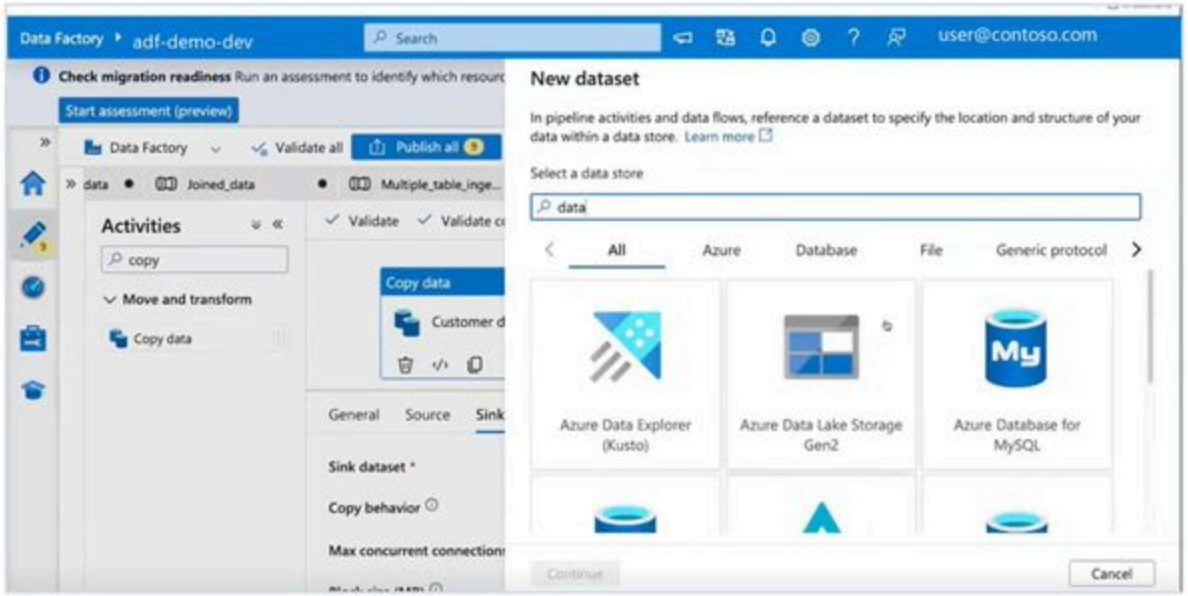


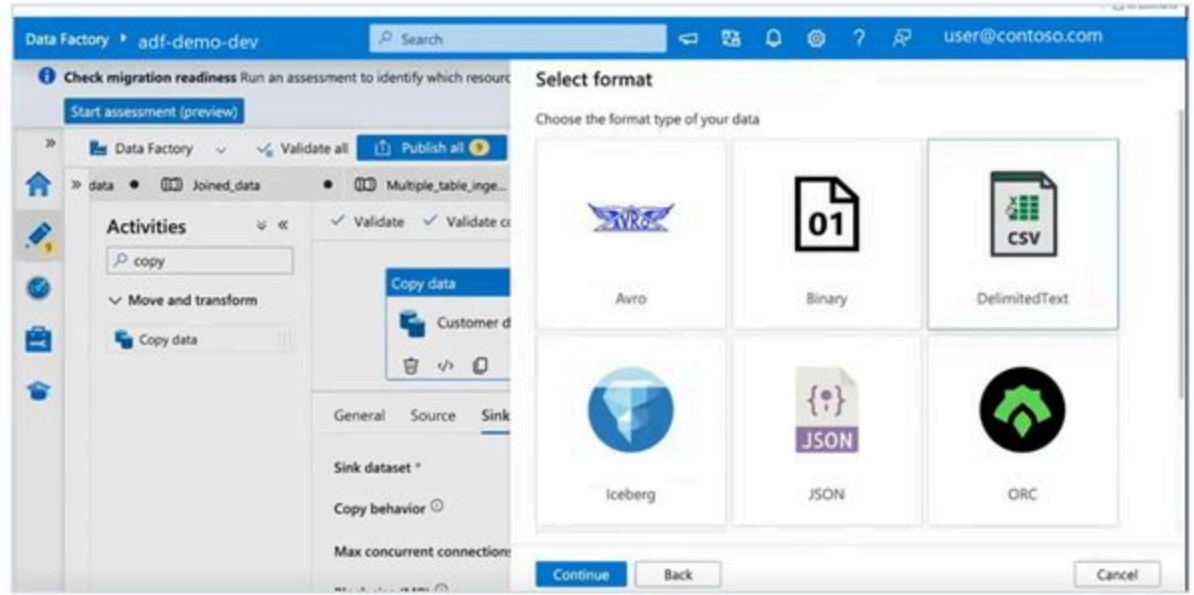
How a dataset parameter works



PART 6

Lab 6 – New dataset ADLS_Generic_DS

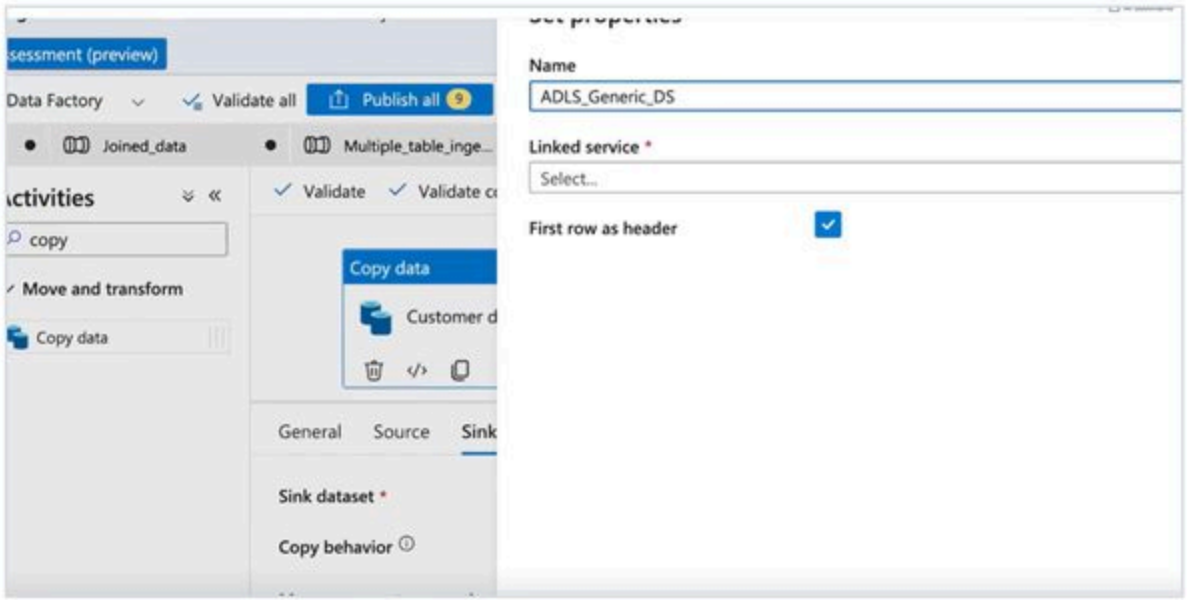


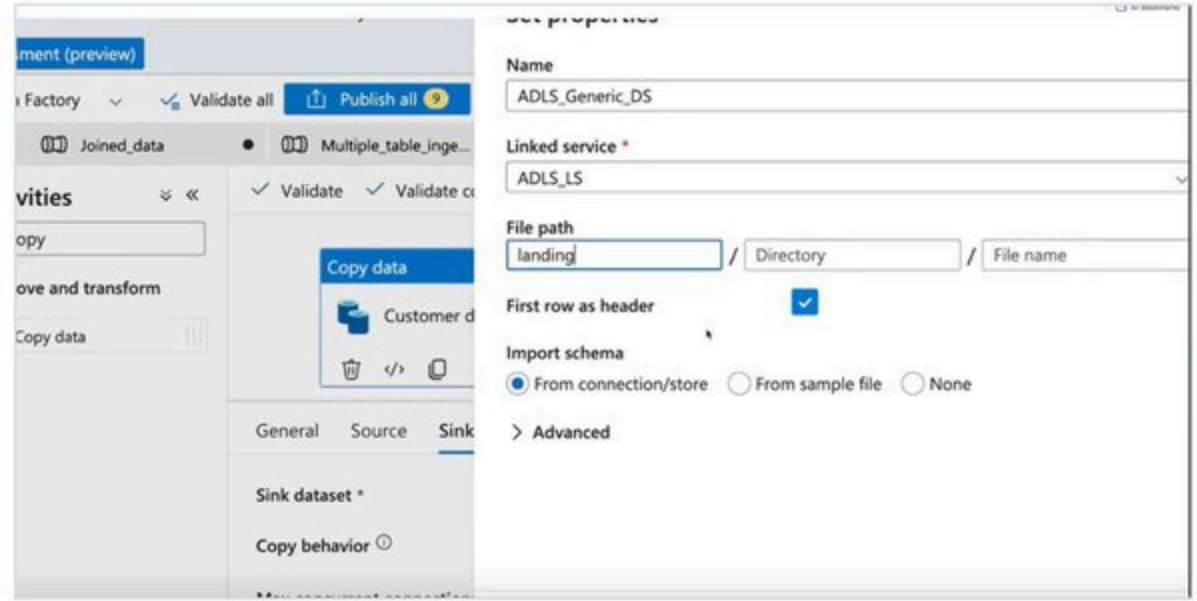


- + New dataset → ADLS Gen2
- Format: DelimitedText

PART 6

ADLS_Generic_DS – properties

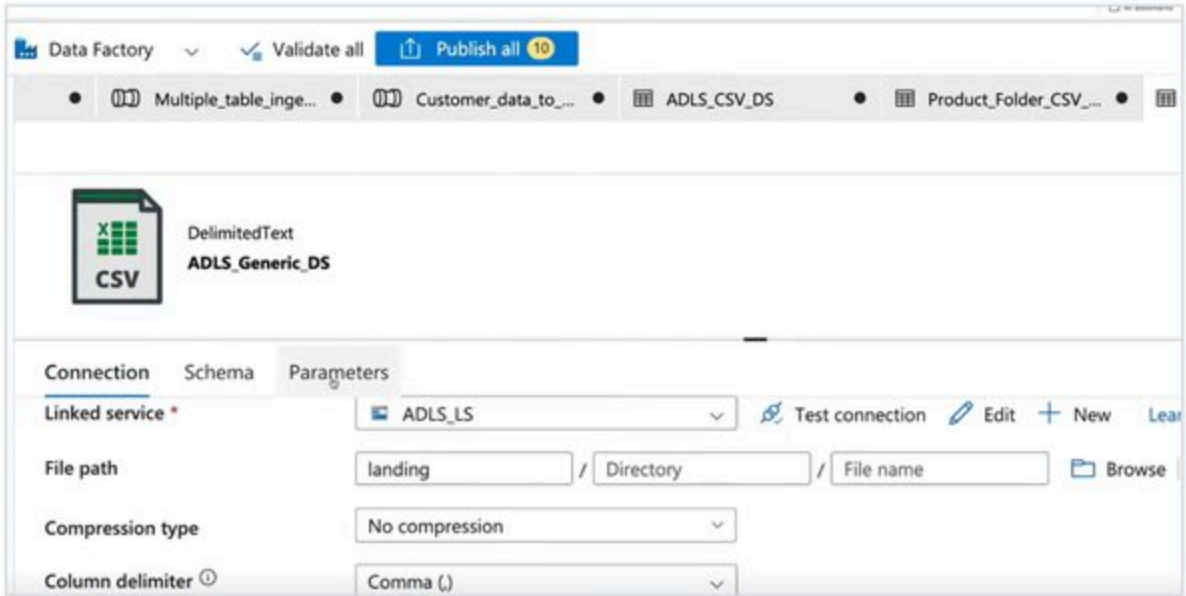


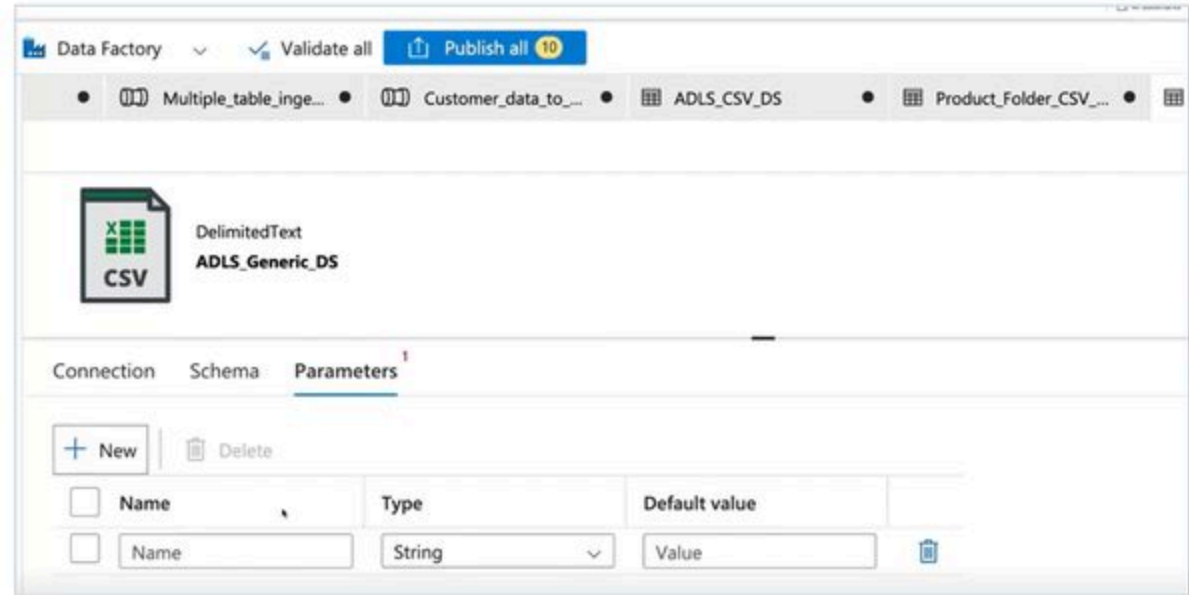


- Name: ADLS_Generic_DS
- Linked service: ADLS_LS
- File path: landing / (empty) / (empty)
- Import schema: None

PART 6

Parameters tab → + New

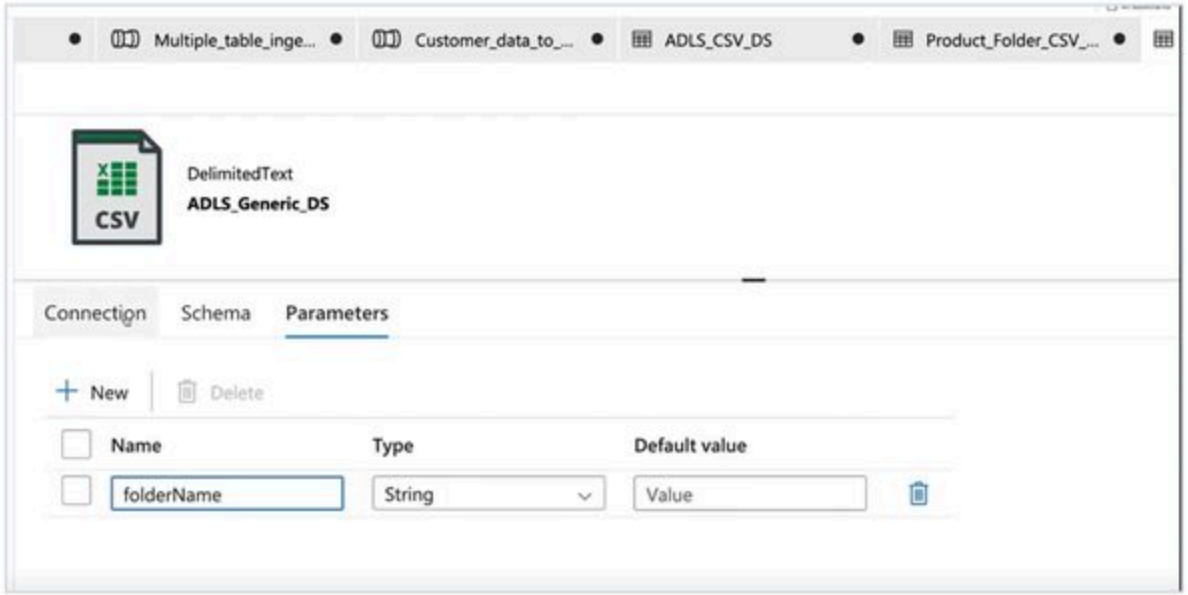


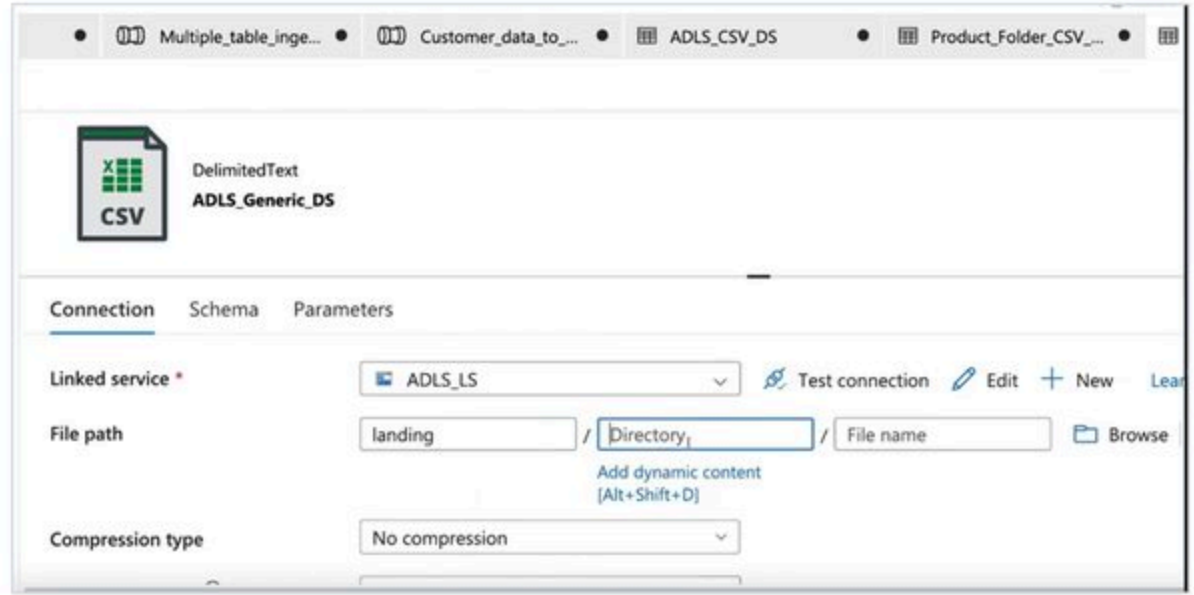


- Dataset → Parameters tab
- + New: Name, Type, Default value

PART 6

Create the folderName parameter

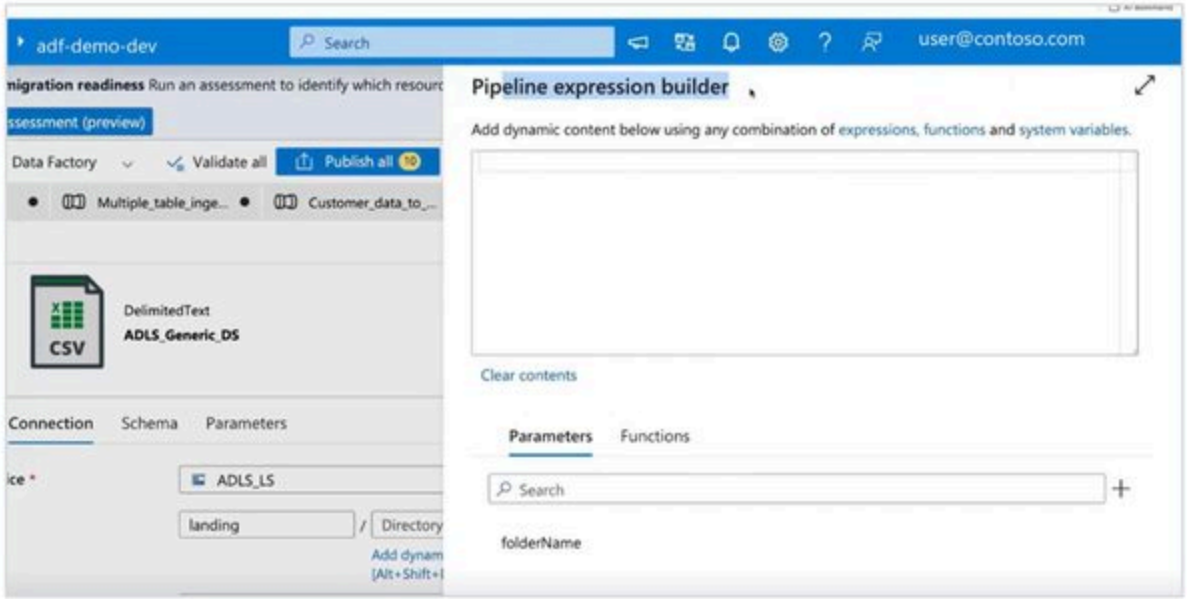


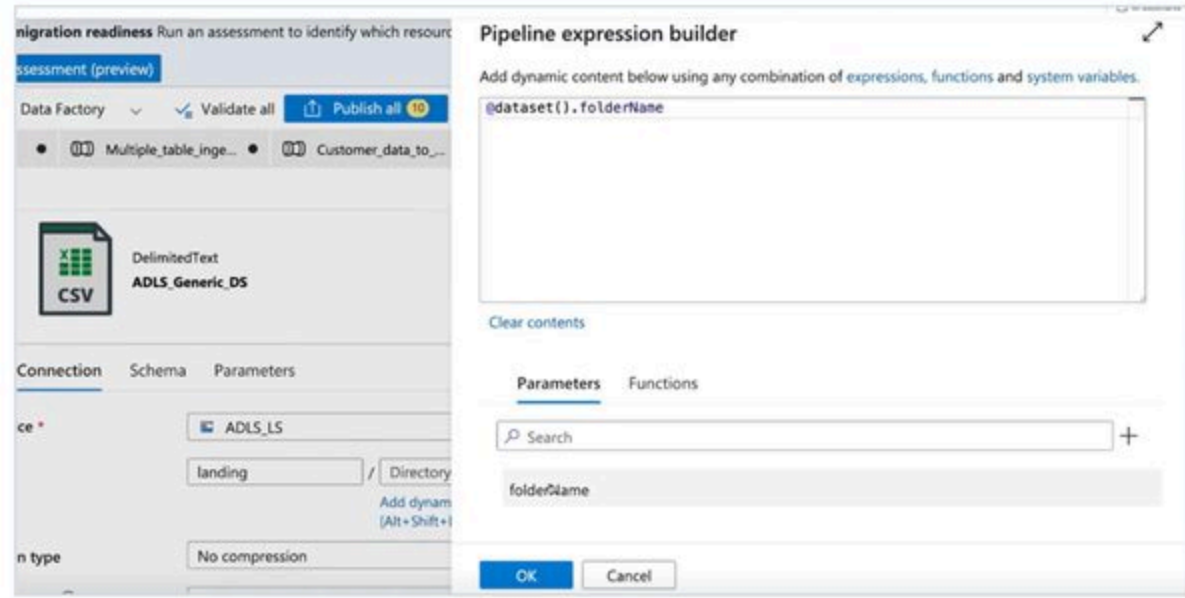


- Name: folderName, Type: String
- Back to Connection → click Directory
- "Add dynamic content [Alt+Shift+D]"

PART 6

Pipeline expression builder

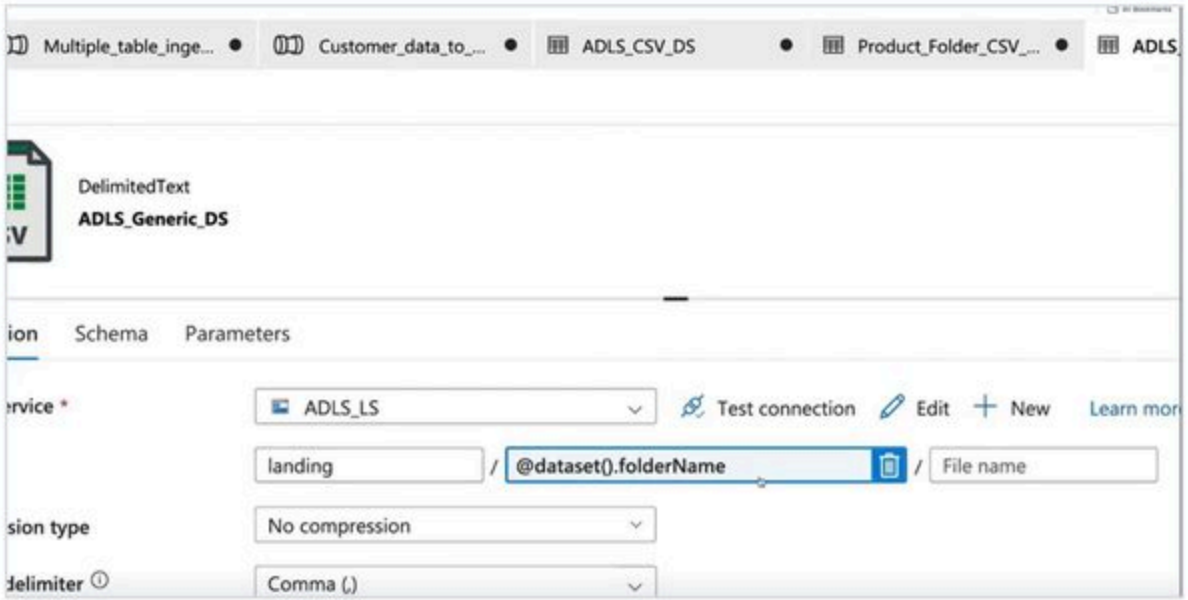


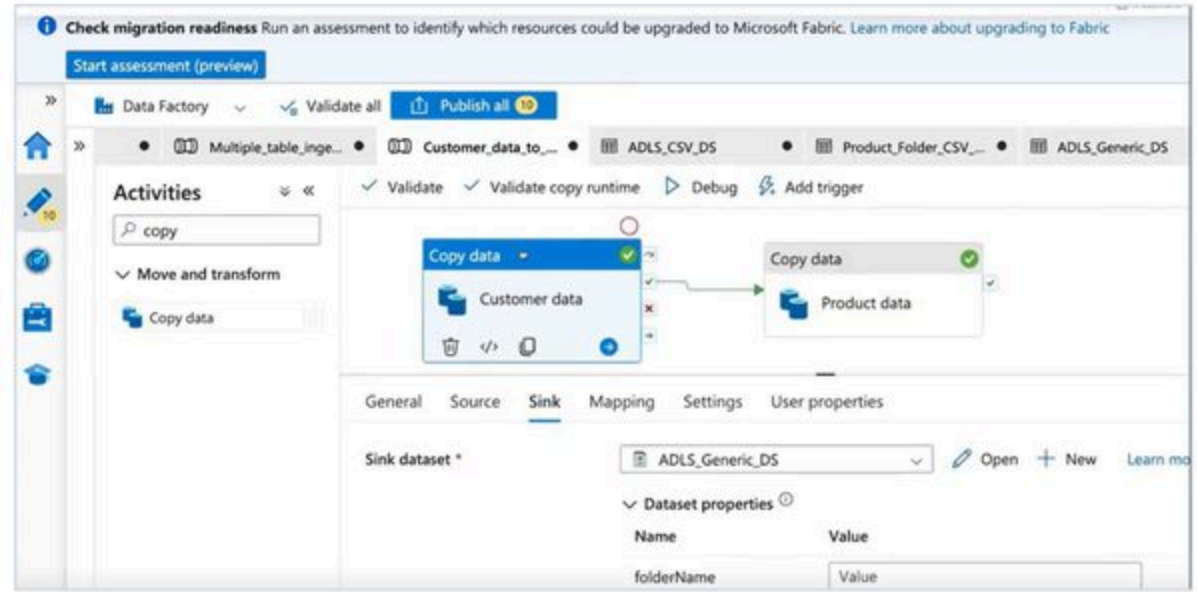


- Lists parameters, functions, system variables
- Click folderName → @dataset().folderName
- OK

PART 6

The Directory is now dynamic

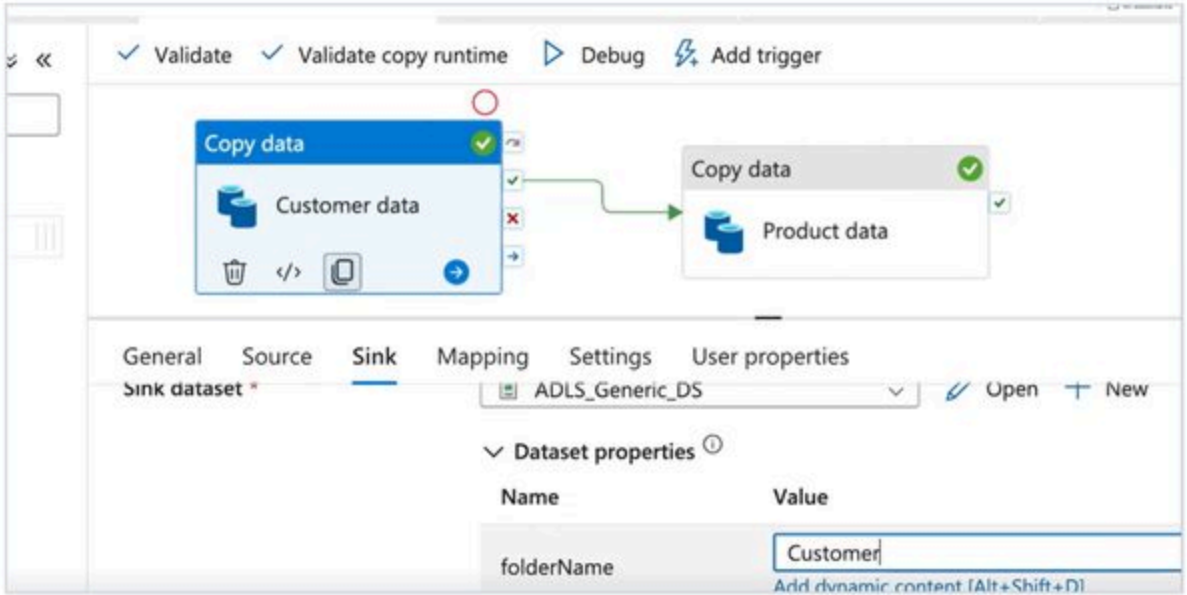


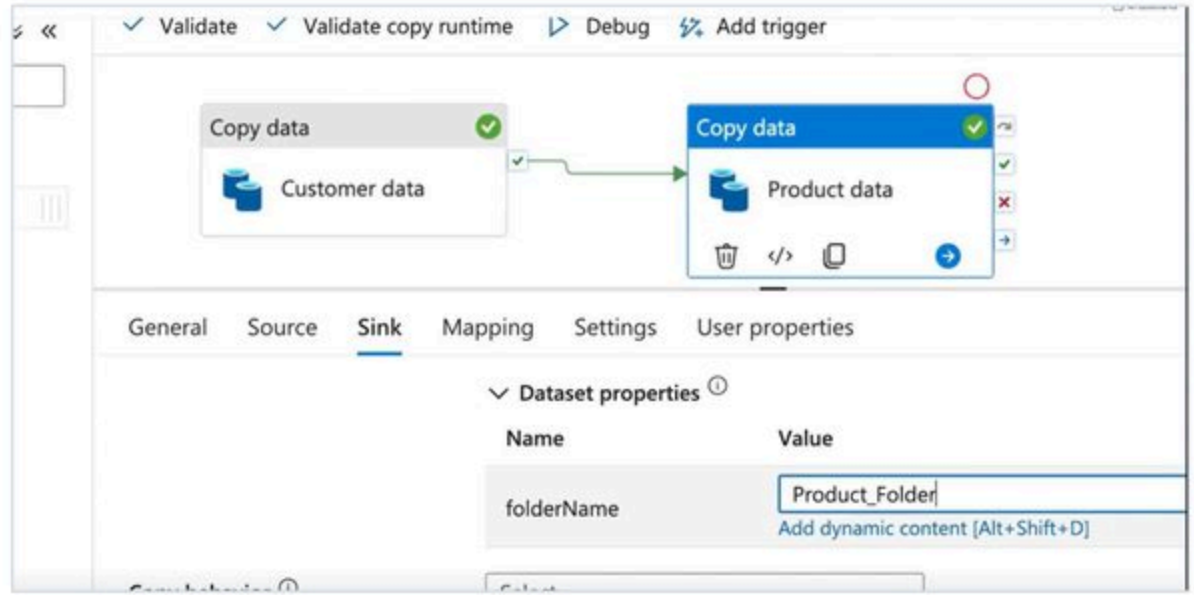


- Directory: @dataset().folderName
- Copy activity Sink: ADLS_Generic_DS
- Dataset properties → folderName = Value

PART 6

Pass a value from each activity

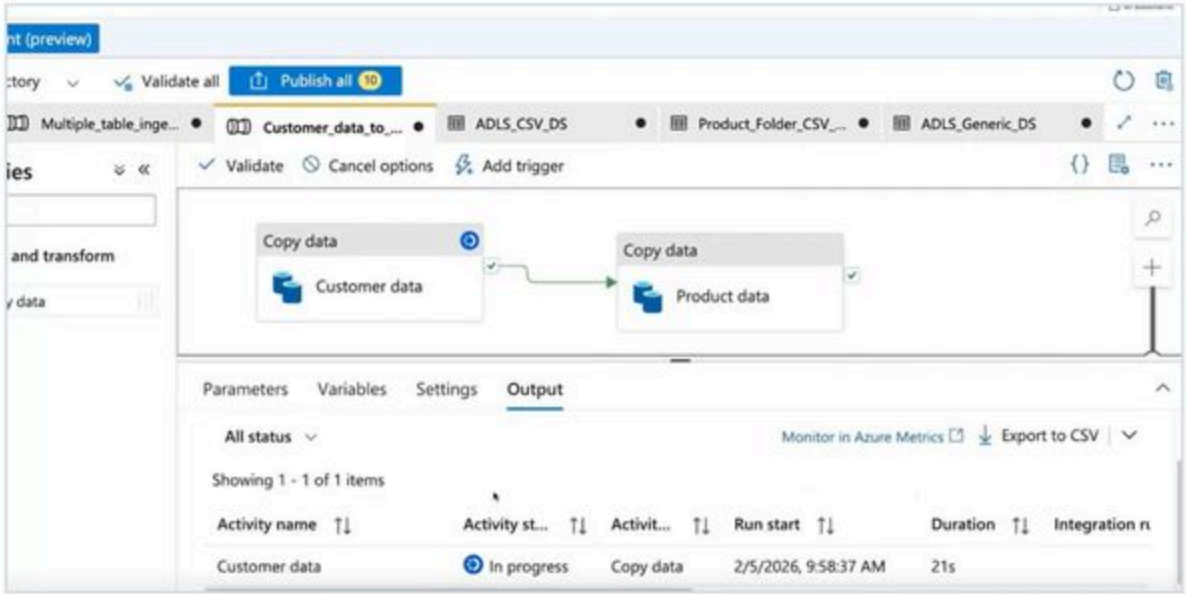


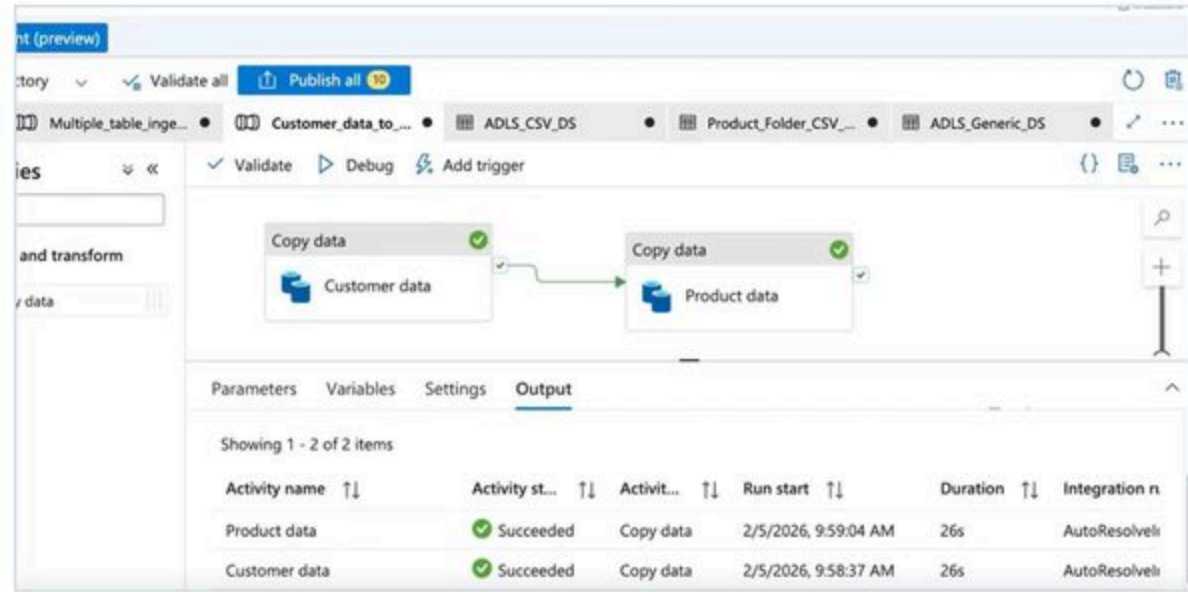


- Customer data → folderName = Customer_Folder
- Product data → folderName = Product_Folder
- Same dataset, different folders

PART 6

Debug and check the run

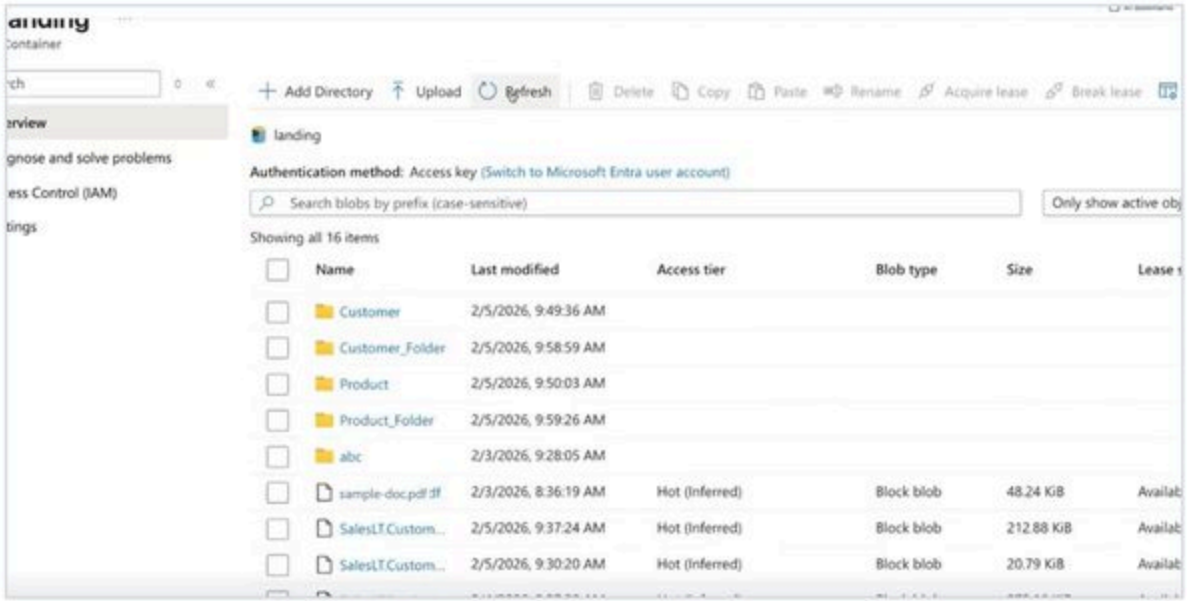


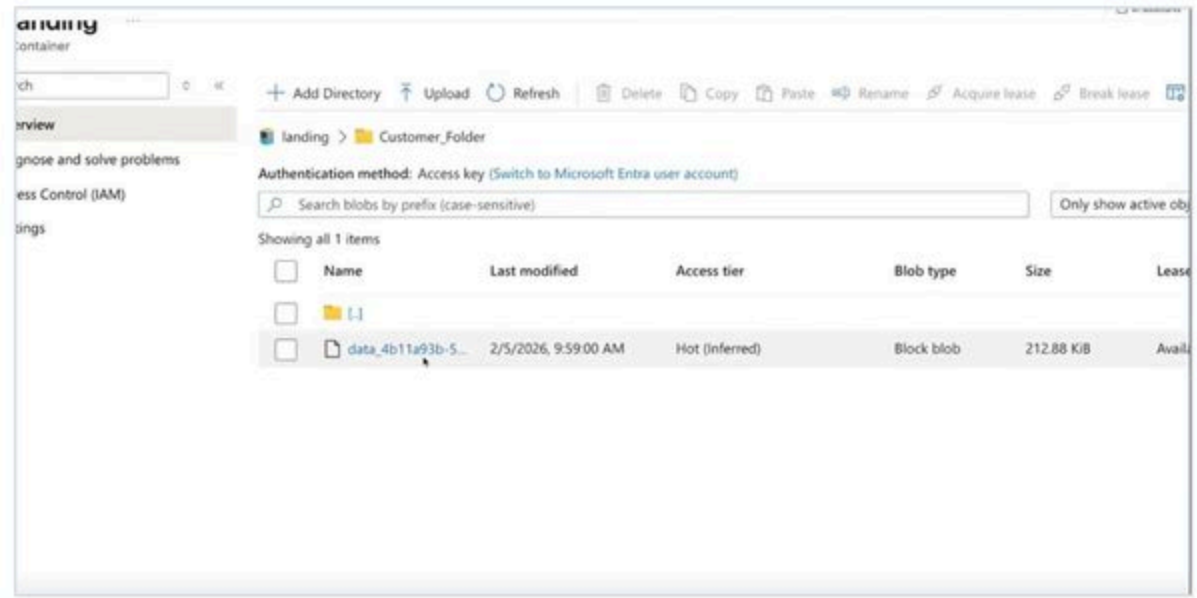


- Customer data → Product data
- Both succeeded (~26 s each)

PART 6

Validate: new folders from one dataset





- landing: Customer_Folder and Product_Folder
- File: landing/Customer_Folder/data_<guid>

- No new dataset needed for a new table

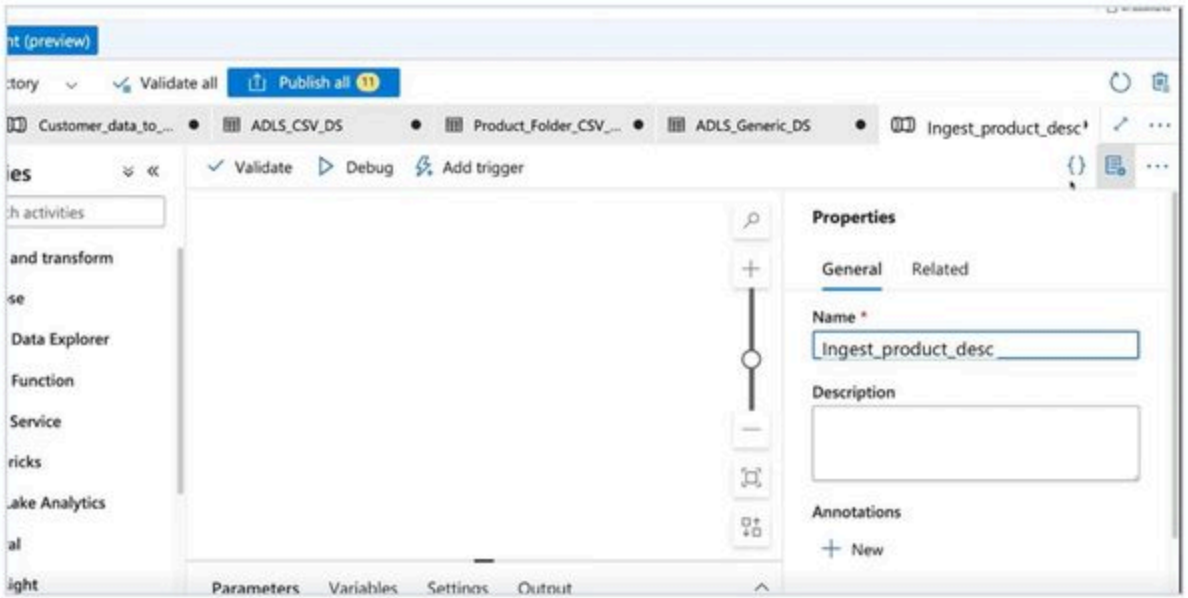
PART 7

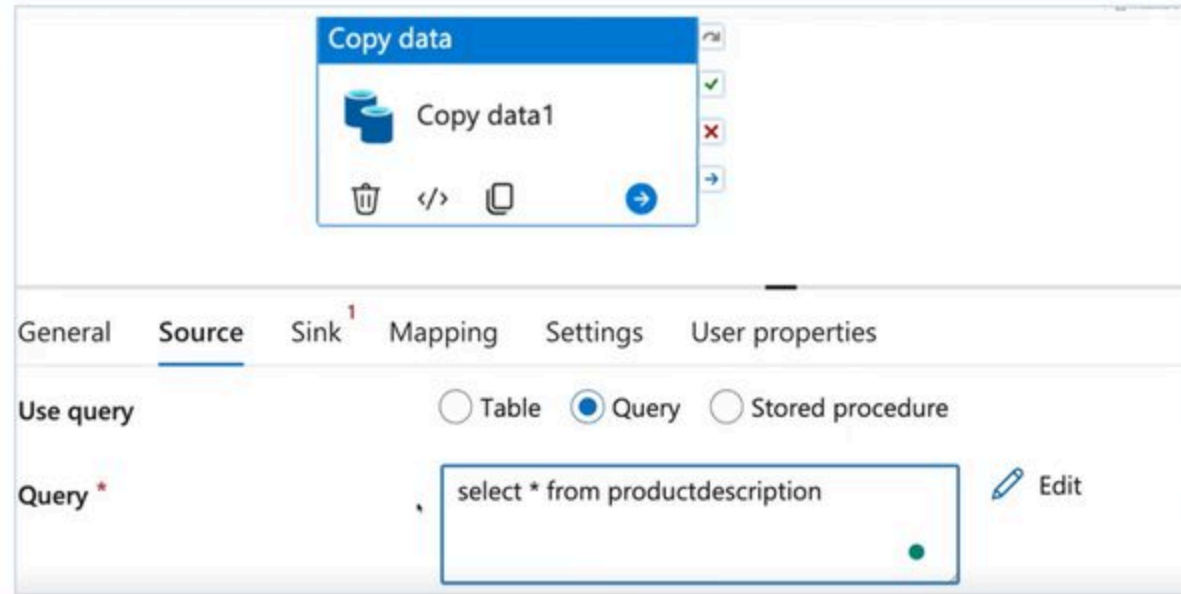
Lab 7: Practice

Copy SalesLT.ProductDescription into its own folder – on your own

PART 7

Lab 7 – Pipeline Ingest_product_desc

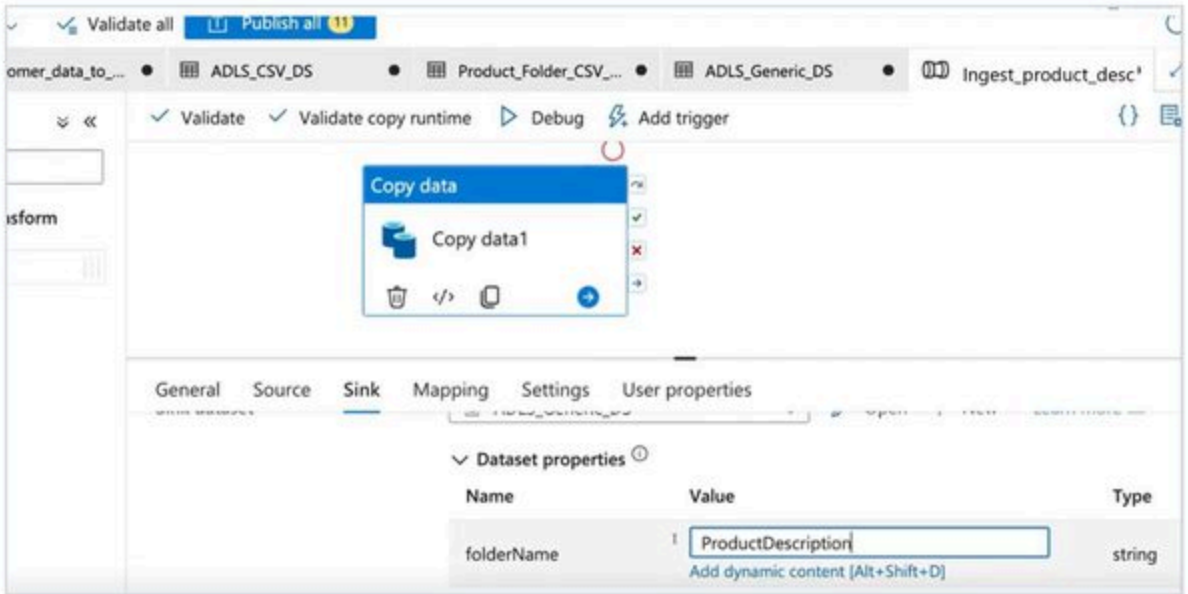


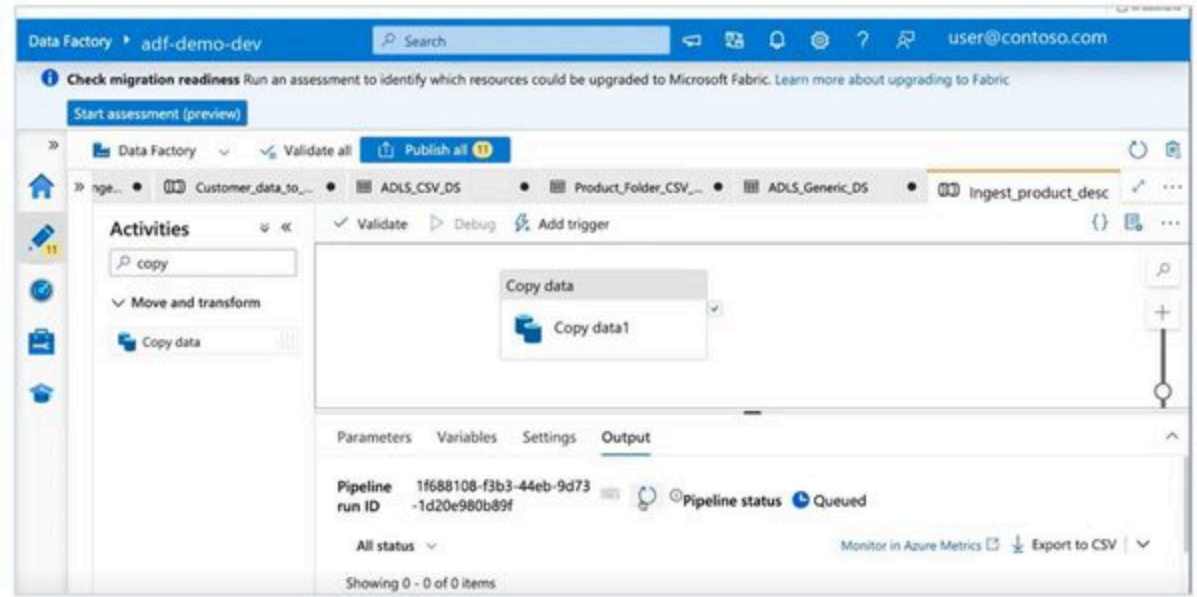


- New pipeline: Ingest_product_desc
- Source query: select * from productdescription

PART 7

Lab 7 – Sink: ADLS_Generic_DS

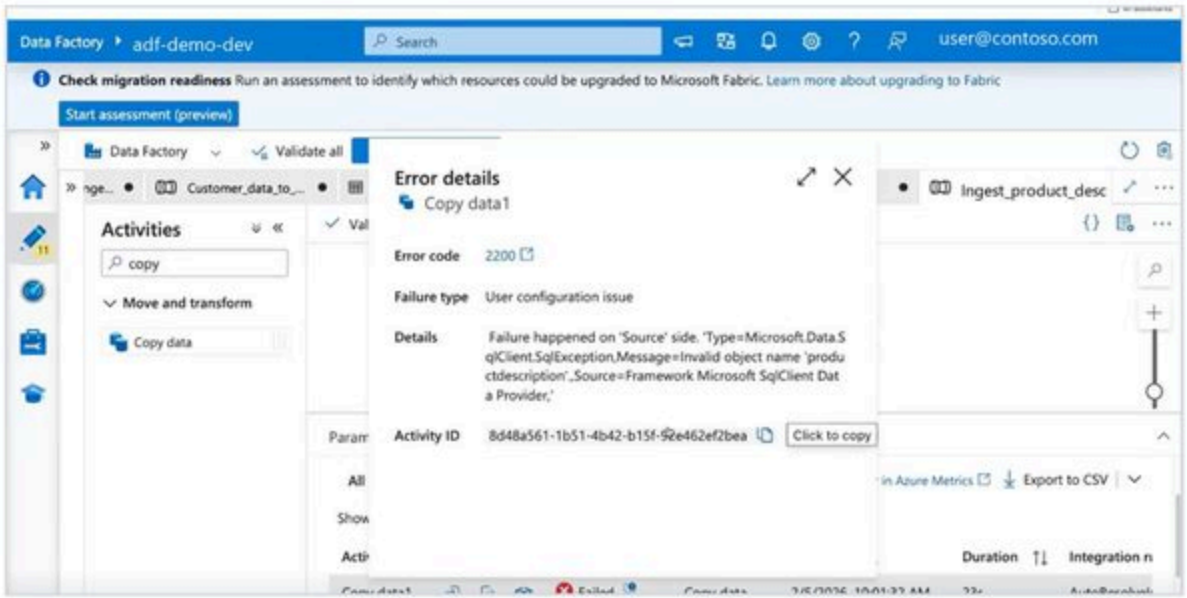


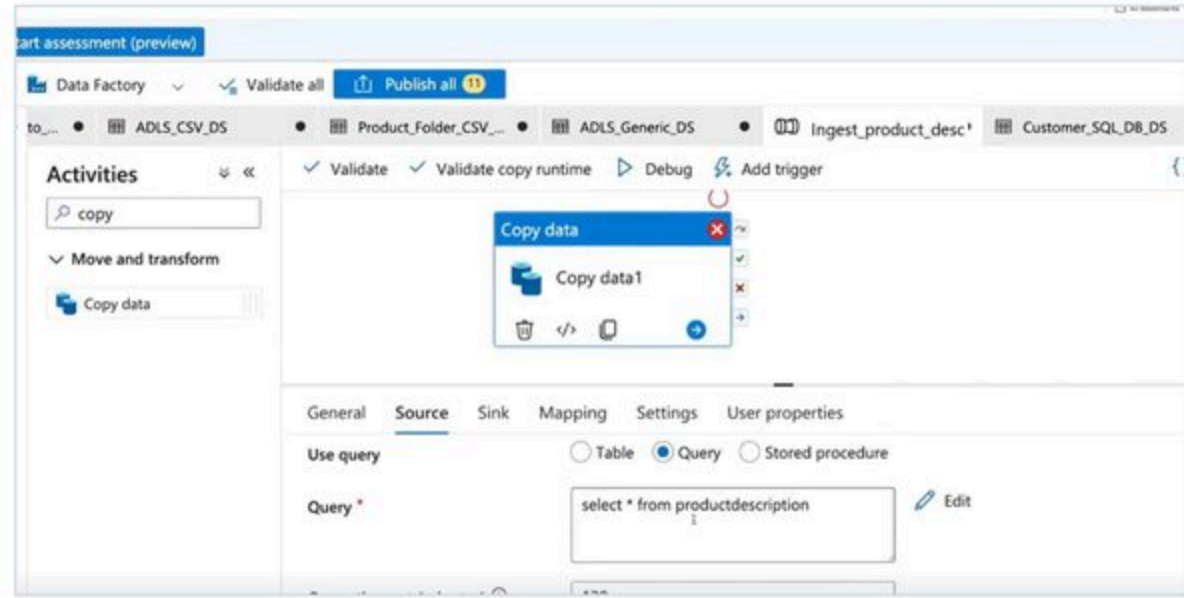


- Sink: ADLS_Generic_DS
- folderName = ProductDescription
- Debug → Queued

PART 7

Lab 7 – The run fails: read the error



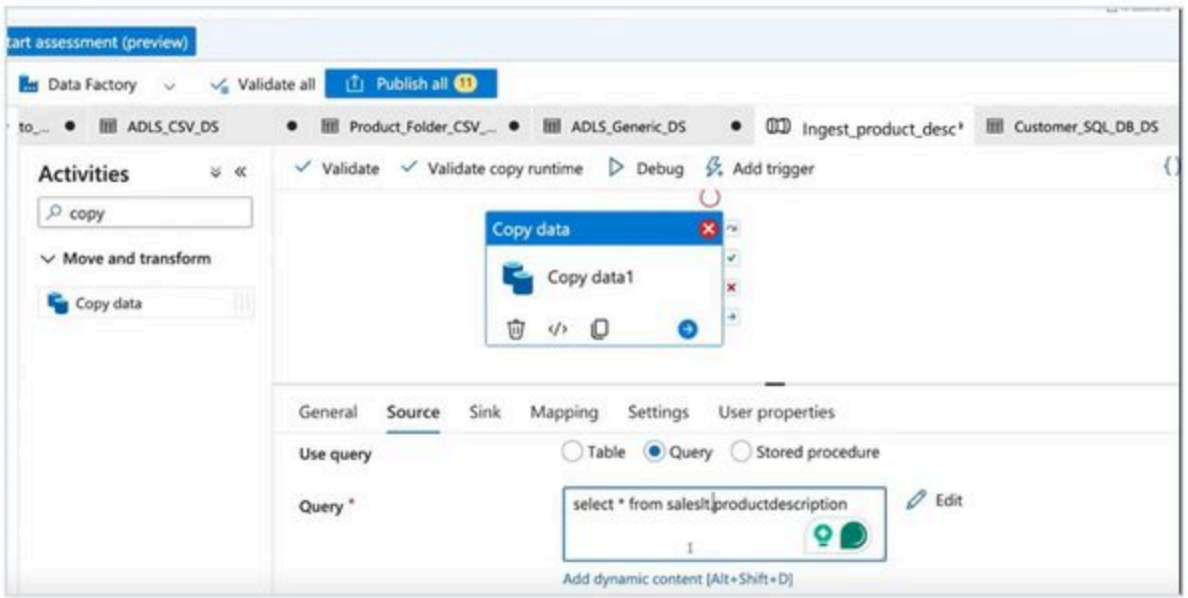


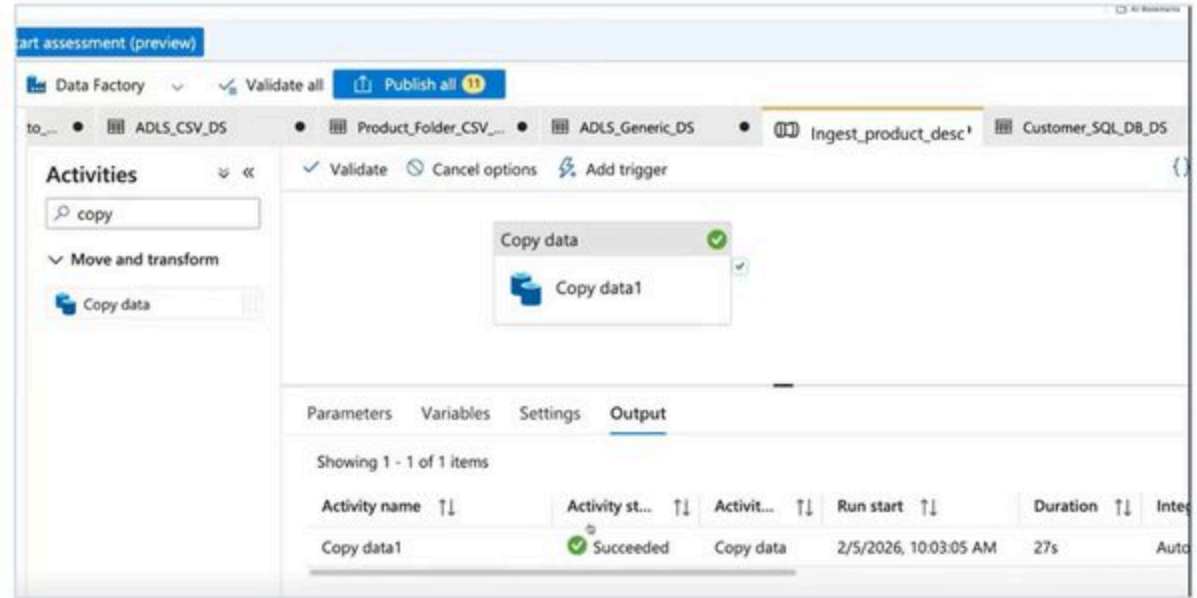
- Error code 2200 – User configuration issue
- "Invalid object name 'productdescription'"
- Failure happened on the Source side

⚠ Invalid object name = wrong table name or missing schema.

PART 7

Lab 7 – Fix and re-run

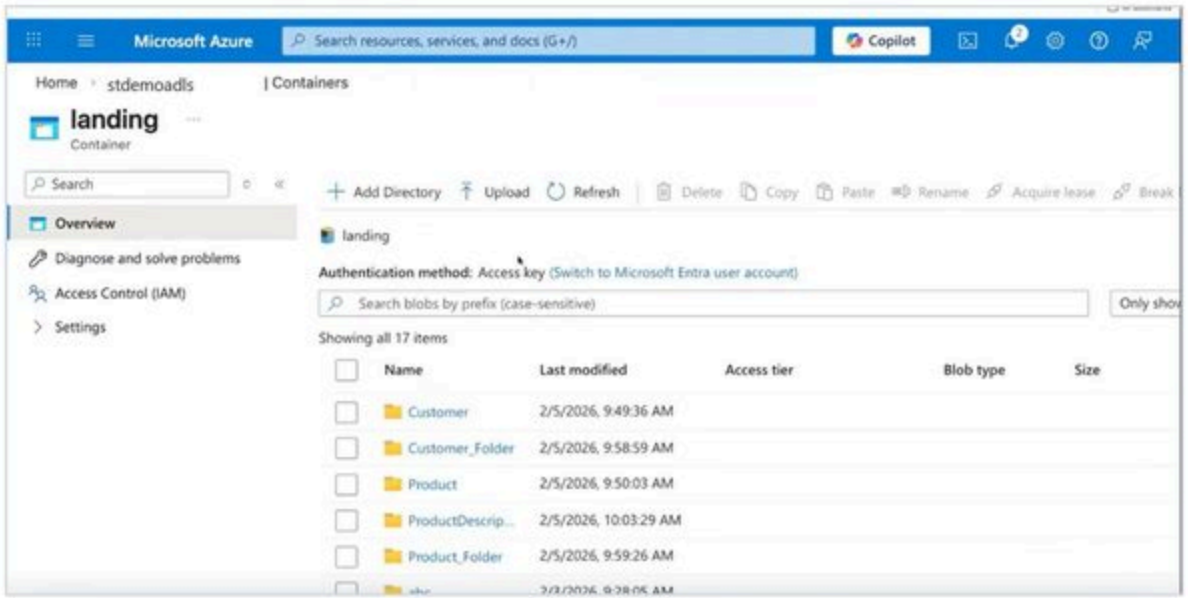


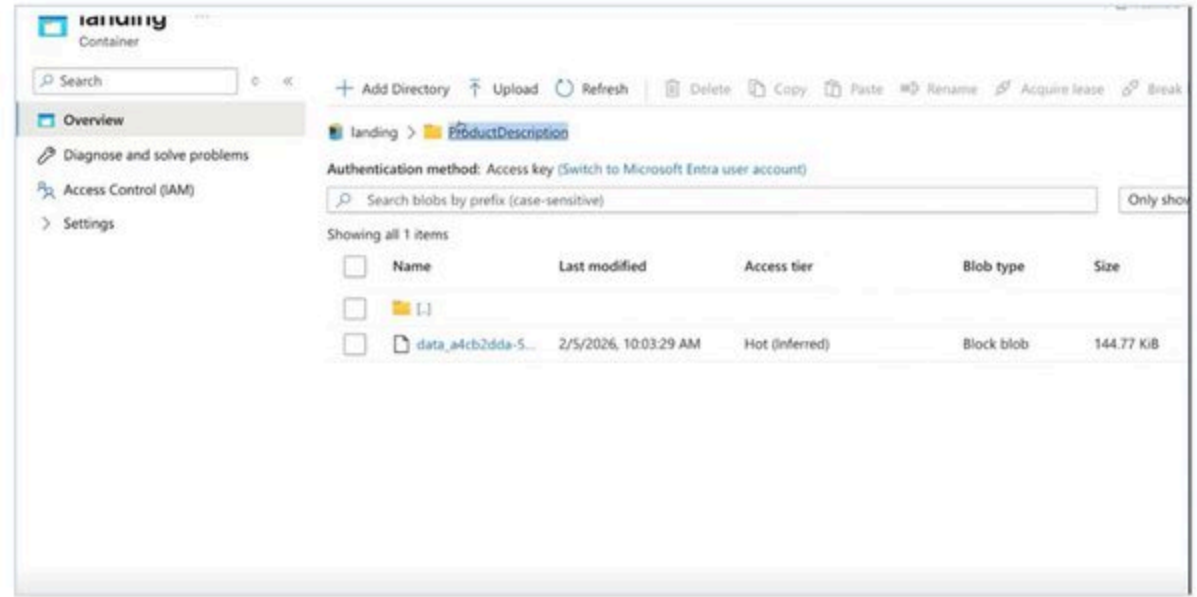


- select * from saleslt.productdescription
- Debug → Succeeded in ~27 s

PART 7

Lab 7 – Validate the new folder





- landing: ProductDescription folder
- landing/ProductDescription/data_<guid> (~145 KiB)
- Publish all when done

PART 8

Wrap-up

Troubleshooting • Best practices • Q&A • Homework

PART 8

Troubleshooting cheat-sheet

Symptom	Likely cause	Fix
Activity name should not have trailing space	Space at end of the name	Remove the space
Duplicate column names (9019)	SELECT * on a join	List columns; rename with AS
Schema import failed: PathNotFound	Sink folder does not exist yet	Import schema = None
Invalid object name (2200)	Missing schema (SalesLT.) or typo	Use schema.table
Client IP not allowed (Query editor)	Firewall / IP changed	Allowlist IP on the server
Files named data_<guid>	Query source + no file name	Set a file name or folder
Other pipelines write to a new folder	Shared dataset was edited	Use a parameterised dataset
Parquet file looks corrupted	It is binary	Read it with Spark / Synapse / Fabric

Best practices from today

- Parquet (or Delta) for analytics; CSV/JSON only for exchange
- Copy only what you need: explicit columns, WHERE filters
- Never SELECT * on a join; always prefix tables with the schema
- One folder per table (and later per load date)
- Parameterise datasets – avoid one dataset per table
- Run independent copies in parallel; chain only real dependencies
- Use On failure for alerts and logging

PART 8

Frequently asked questions

Question	Short answer
Parquet or Delta?	Delta = Parquet files + transaction log (ACID, time travel). Use Delta in Databricks/Fabric.
Why did my query file get a strange name?	Query source + no file name → data_<run id>.
Do I need to create folders first?	No – ADF creates them during the copy.
Max parallel activities?	Many; keep the source database load in mind.
Dataset vs pipeline parameter?	Dataset parameter makes the dataset reusable; pipeline parameter is set when the pipeline runs.
How do I loop over many tables?	Lookup (get the list) + ForEach (repeat a Copy) – later in the course.
Can I filter rows changed since yesterday?	Yes – a WHERE on ModifiedDate: incremental load.
Does Debug cost money?	Yes, like a normal run (activity runs + DIU time), but small.

Quick quiz

Question	Answer
Which format is columnar and binary?	Parquet
Which source option lets you join two tables?	Use query = Query
Which dependency runs only if the previous activity failed?	On failure (red)
Three unconnected Copy activities run …	In parallel
Expression for a dataset parameter called folderName?	@dataset().folderName
Why did “select * from productdescription” fail?	Missing the SalesLT schema

Prepared by Ir. UWAYO Jacques · Follow me on LinkedIn: linkedin.com/in/uwayo

Homework / practice

- Rebuild Labs 1–7 without the recording
- Copy SalesLT.SalesOrderHeader to ADLS as Parquet into folder SalesOrderHeader
- Add a fileName parameter to ADLS_Generic_DS; name files <table>.csv
- Pipeline with 3 parallel copies + an On failure Wait/Set variable activity
- Bonus: folder per date with @formatDateTime(utcNow(),'yyyy/MM/dd')
- Compare the size of one table as CSV, JSON and Parquet

Prepared by Ir. UWAYO Jacques · Follow me on LinkedIn: linkedin.com/in/uwayo

Next session preview – Session 05: Databricks

- Big data concept
- Types of data
- Spark vs PySpark vs Databricks
- How to create a Databricks account
- Read a CSV file in Databricks
- Create a table in Databricks
- Genie in Databricks
- Lots of Q&A

Thank you

Questions? • Publish your work • Rebuild the 7 labs on your own

Prepared by Ir. UWAYO Jacques · Follow me on LinkedIn: **linkedin.com/in/uwayo**